



Departamento de
Computação - **UFSCar**



Algoritmos para agrupamento de dados

Apostila sobre métodos não supervisionados para agrupamento de dados

Alexandre L. M. Levada
Departamento de Computação - UFSCar

20 de outubro de 2025

Prefácio

Em um mundo cada vez mais inundado por dados, a capacidade de descobrir estrutura, padrões e regularidades ocultas tornou-se uma das habilidades mais valiosas em ciência e tecnologia. O agrupamento de dados — ou clustering — está no coração dessa tarefa. Diferentemente dos métodos supervisionados, o agrupamento busca revelar, de forma autônoma, como os dados se organizam em grupos naturais, sem depender de rótulos ou classificações prévias. É, portanto, uma das formas mais puras e desafiadoras de aprendizado de máquina.

Esta apostila foi escrita com o propósito de introduzir o leitor ao fascinante universo dos algoritmos de agrupamento, abordando tanto os fundamentos teóricos quanto os aspectos práticos desses métodos. Aqui, são discutidas diferentes abordagens que refletem perspectivas complementares sobre o que significa “encontrar estrutura” em dados: dos modelos baseados em protótipos, como o k-médias, à modelagem probabilística dos Gaussian Mixture Models; das estratégias hierárquicas que constroem dendrogramas revelando relações entre grupos, às técnicas mais recentes de agrupamento espectral e métodos baseados em grafos, que exploram a geometria e a topologia dos dados.

Mais do que uma simples coletânea de algoritmos, este material busca despertar a intuição por trás de cada técnica: o que motiva seu uso, como ela interpreta a noção de similaridade e quais são suas vantagens e limitações. O leitor será convidado a refletir sobre as diferentes formas de medir distâncias, sobre a importância da representação dos dados e sobre como o conceito de cluster pode assumir múltiplas interpretações dependendo do contexto e do método utilizado.

Espera-se que, ao final desta leitura, o estudante não apenas compreenda o funcionamento dos algoritmos apresentados, mas também desenvolva uma visão crítica e comparativa sobre eles — uma competência essencial para quem deseja aplicar agrupamento de dados em pesquisa científica, negócios, engenharia, ou em qualquer área em que compreender a estrutura subjacente aos dados seja fundamental.

Bons estudos!

Sumário

1	Introdução ao agrupamento de dados	4
1.1	O problema do agrupamento de dados	5
1.2	Considerações finais	7
2	O algoritmo k-médias e suas variantes	8
2.1	Fundamentação teórica	8
2.2	Limitações do algoritmo k-médias	13
2.3	O algoritmo k-means++	14
2.3.1	Pseudocódigo	17
2.4	O algoritmo <i>Fuzzy C-Means</i>	17
2.5	Considerações finais	22
3	Agrupamento hierárquico	24
3.1	Abordagens do agrupamento hierárquico	24
3.2	Abordagem aglomerativa (Bottom-Up)	25
3.2.1	CrITÉrio de Ward para funções de ligação	27
3.3	Abordagem divisiva (Top-Down)	31
3.4	Comparação entre as abordagens	33
3.5	Índice Calinski-Harabasz	33
3.6	O método do cotovelo	35
3.7	Considerações finais	36
4	Modelos de mistura de Gaussianas	38
4.1	Formulação matemática	39
4.2	Comparação entre k-médias e GMM	40
4.3	Algoritmo EM: Expectation-Maximization	40
4.4	Modelo de mistura de Gaussianas com duas componentes	42
4.4.1	Algoritmo EM+GMM para agrupamento de dados	45
4.5	Seleção do número agrupamentos: critérios AIC e BIC	47
4.5.1	CrITÉrio de Informação de Akaike (AIC)	47
4.5.2	CrITÉrio de Informação Bayesiano (BIC)	48
4.6	Considerações finais	49
5	Agrupamento espectral (Spectral clustering)	51
5.1	A Matriz Laplaciana	52
5.2	Laplacianas normalizadas	54
5.3	Imersão Laplaciana 1-D (na reta)	55
5.4	Imersão Laplaciana Multidimensional	57
5.5	Relação entre cortes em grafos e agrupamento espectral	59

5.6	Considerações finais	64
6	Agrupamento de dados com em árvores geradoras mínimas	65
6.1	O problema da árvore geradora mínima (MST)	65
6.2	O Algoritmo de Kruskal	66
6.2.1	Análise da complexidade	72
6.3	O Algoritmo de Prim	73
6.3.1	Análise da Complexidade	78
6.4	Comparação entre os algoritmos de Kruskal e Prim	80
6.4.1	Diferenças conceituais e estruturais	81
6.4.2	Conclusão comparativa	81
6.5	Agrupamento baseado em MST's	83
6.5.1	Algoritmos divisivos	83
6.5.2	Agrupamento baseado em teoria da informação	84
6.5.3	Derivação matemática	86
6.5.4	Algoritmos aglomerativos	88
6.6	Considerações finais	89
7	Agrupamento de dados baseado em densidade	90
7.1	O algoritmo DBSCAN	90
7.1.1	Fundamentação matemática	91
7.1.2	Estrutura do algoritmo	93
7.1.3	Heurística para escolha de ε e MinPts	93
7.1.4	O conceito do gráfico <i>k-dist</i>	94
7.2	O algoritmo OPTICS	97
7.2.1	Fundamentação teórica	98
7.2.2	O gráfico de alcançabilidade	101
7.3	O algoritmo HDBSCAN	102
7.4	Considerações finais	105

1 Introdução ao agrupamento de dados

Reconhecer padrões é uma das capacidades cognitivas mais fundamentais do ser humano: identificar rostos em uma multidão, perceber grupos de sons semelhantes em uma conversa ou distinguir estilos em obras de arte são exemplos cotidianos dessa habilidade de **detectar regularidades e agrupar elementos similares**.

De forma análoga, no contexto computacional, o **agrupamento de dados** — ou *clustering* — busca desenvolver algoritmos capazes de identificar, de forma automática, **estruturas latentes e grupos naturais** em conjuntos de dados, sem a necessidade de rótulos ou supervisão explícita.

Formalmente, o problema pode ser descrito como o de **particionar o espaço de atributos** em subconjuntos coerentes (ou *clusters*), de modo que amostras dentro do mesmo grupo apresentem **alta similaridade entre si e baixa similaridade** em relação a amostras de outros grupos. Assim, dado um conjunto de amostras representadas por vetores em um espaço multidimensional, o objetivo é descobrir uma estrutura que revele a organização subjacente dos dados — uma tarefa essencial para compreensão, análise exploratória e modelagem de fenômenos complexos.

O agrupamento de dados tem papel central em inúmeras aplicações modernas, tais como:

- **Visão computacional:** segmentação de imagens, detecção de padrões visuais e agrupamento de características locais;
- **Processamento de sinais e fala:** análise de timbres, descoberta de tópicos sonoros e separação de fontes;
- **Ciência de dados e bioinformática:** agrupamento de perfis gênicos, descoberta de subtipos em dados clínicos e mineração de padrões em grandes bases biomédicas;
- **Sistemas inteligentes:** recomendação de conteúdos, análise de comunidades em redes sociais e segmentação de mercados.

A disciplina de agrupamento de dados é inerentemente interdisciplinar, envolvendo conceitos de estatística, aprendizado de máquina, geometria computacional, teoria dos grafos, otimização e teoria da informação. Essa riqueza de fundamentos reflete-se na variedade de algoritmos existentes e nas diferentes interpretações possíveis sobre o que constitui um *cluster*.

De maneira geral, o agrupamento de dados se insere no contexto do **aprendizado não supervisionado**, em contraste com o aprendizado supervisionado, em que as classes ou rótulos das amostras são previamente conhecidos. No agrupamento, o desafio é justamente **descobrir** essa estrutura sem qualquer conhecimento prévio sobre os grupos subjacentes.

- **Aprendizado supervisionado:** as amostras possuem rótulos conhecidos, e o objetivo é aprender uma função que generalize para novos dados;
- **Aprendizado não supervisionado:** os rótulos são desconhecidos, e o objetivo é identificar automaticamente agrupamentos ou representações latentes nos dados.

O agrupamento é, portanto, um dos pilares do aprendizado não supervisionado, e serve frequentemente como ponto de partida para etapas posteriores de análise, como classificação, redução de dimensionalidade, detecção de anomalias e extração de características.

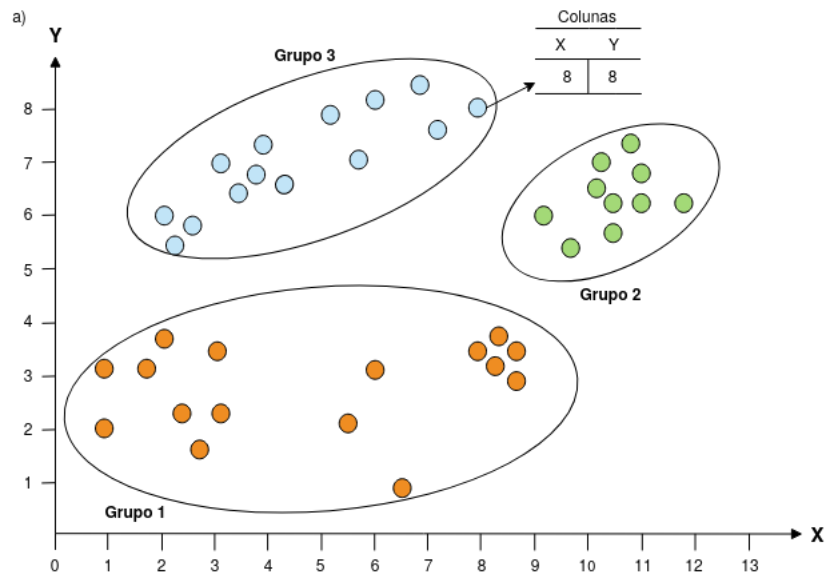


Figura 1: Exemplo ilustrativo de agrupamento de dados em um espaço bidimensional. Fonte: <https://dataat.github.io/introducao-ao-machine-learning/>

1.1 O problema do agrupamento de dados

Dado um conjunto de amostras $X = \{\vec{x}_1, \vec{x}_2, \dots, \vec{x}_n\}$, com $\vec{x}_i \in \mathbb{R}^d$, o objetivo do agrupamento é encontrar uma partição $C = \{C_1, C_2, \dots, C_k\}$ de X , tal que:

- amostras dentro de um mesmo grupo C_j sejam similares segundo alguma métrica ou medida de proximidade;
- amostras pertencentes a grupos distintos sejam dissimilares.

O número de grupos k pode ser conhecido ou não, dependendo do método utilizado. De maneira geral, um algoritmo de agrupamento procura estimar implicitamente uma **função de atribuição**

$$f : X \rightarrow \{1, 2, \dots, k\}$$

que associa a cada vetor $\vec{x}_i$ um índice de cluster.

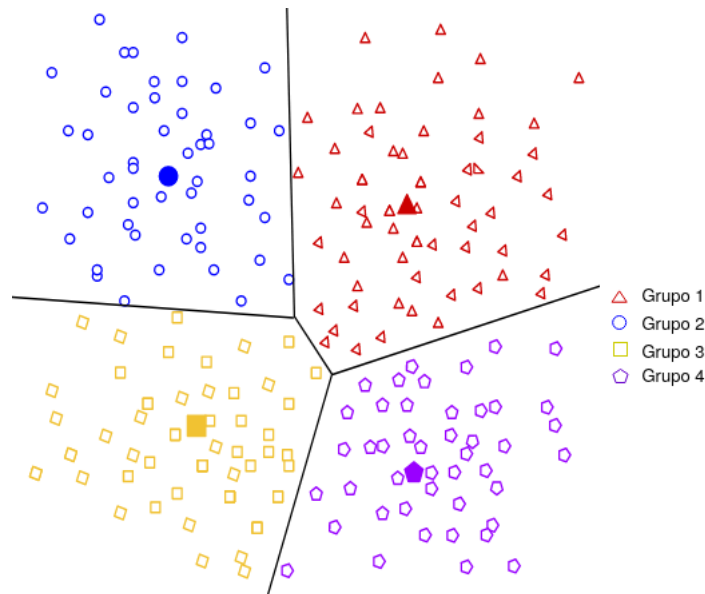


Figura 2: Partição do espaço de atributos em regiões correspondentes aos diferentes grupos. Fonte: <https://dataat.github.io/introducao-ao-machine-learning/>

Observações

- X : conjunto de dados observado (o *espaço de características*);
- C : conjunto de clusters ou grupos descobertos;
- **Classificação supervisionada:** requer conhecimento prévio de rótulos Y ;
- **Agrupamento de dados:** busca inferir a estrutura de C sem acesso a Y .

Questões Fundamentais

- O que caracteriza um *cluster* em um dado conjunto de dados?
- Que medidas de similaridade ou distância são mais adequadas?
- Quantos grupos existem nos dados?
- Como avaliar a qualidade de um agrupamento obtido?

Essas perguntas estão no cerne do estudo do **agrupamento de dados** e motivam o desenvolvimento de diversas abordagens, desde modelos baseados em centroides, como o k -médias, até métodos probabilísticos, hierárquicos, espectrais e baseados em grafos.

De forma geral, os algoritmos de agrupamento podem ser classificados segundo diferentes critérios:

- **Particionais** ou **hierárquicos**, conforme produzam uma única partição ou uma estrutura em múltiplos níveis;

- **Paramétricos** ou **não paramétricos**, dependendo de assumirem (ou não) um modelo explícito para a distribuição dos dados;
- **Baseados em distância, densidade** ou **similaridade estrutural**, conforme a forma como definem coesão entre amostras.

1.2 Considerações finais

O estudo do agrupamento de dados é essencial para compreender como padrões emergem em conjuntos de dados complexos e de alta dimensionalidade. Neste capítulo introdutório, discutimos o conceito geral de agrupamento, destacando a ideia de partição do espaço de características em grupos coerentes e a diferença fundamental entre aprendizado supervisionado e não supervisionado.

Foi enfatizado que o objetivo central do agrupamento é **revelar a estrutura oculta dos dados**, oferecendo uma forma de explorá-los e interpretá-los de maneira mais informada. Além disso, abordamos brevemente as principais famílias de métodos e as questões centrais envolvidas em sua aplicação.

Nos capítulos seguintes, exploraremos detalhadamente os algoritmos mais representativos dessa área — incluindo k -médiãs, *Gaussian Mixture Models*, agrupamento hierárquico, *spectral clustering* e métodos baseados em grafos —, discutindo suas formulações matemáticas, interpretações geométricas e implicações práticas.

Em síntese, o agrupamento de dados não se resume à execução de algoritmos: trata-se de uma disciplina que busca compreender **como a estrutura do mundo se manifesta nos dados**, e como essa estrutura pode ser descoberta, quantificada e utilizada para gerar conhecimento.

2 O algoritmo k -médias e suas variantes

O algoritmo k -médias (k -means) é um dos métodos mais conhecidos e amplamente utilizados para agrupamento de dados. Sua popularidade se deve à simplicidade conceitual, à eficiência computacional e à boa capacidade de gerar resultados interpretáveis em uma ampla gama de aplicações. A ideia central do k -médias é particionar um conjunto de dados em k grupos distintos, de forma que cada amostra pertença ao cluster cujo centro (ou média) esteja mais próximo dela, segundo uma métrica de distância — tipicamente a distância euclidiana.

De forma intuitiva, o k -médias busca encontrar uma representação compacta dos dados, onde cada cluster é resumido por um protótipo (ou centróide) que corresponde à média das amostras que o compõem. O processo consiste em alternar entre duas etapas principais:

1. atribuir cada amostra ao centróide mais próximo;
2. recalculando os centróides como a média das amostras atribuídas a cada grupo.

Essas duas etapas são repetidas até que o algoritmo convirja, ou seja, até que as atribuições dos dados não mudem mais ou que a variação do critério de otimização se torne desprezível.

Do ponto de vista matemático, o k -médias procura minimizar a soma das distâncias quadráticas entre as amostras e os centróides correspondentes, o que equivale a resolver um problema de otimização não convexo. Embora o método garanta convergência para um mínimo local, a solução final depende fortemente das posições iniciais dos centróides, tornando a inicialização uma etapa importante para o bom desempenho do algoritmo.

O k -médias é particularmente adequado para dados numéricos e bem separados, nos quais os clusters possuem formato aproximadamente esférico e tamanhos semelhantes. Apesar de suas limitações — como a sensibilidade a outliers e a dificuldade em lidar com clusters de forma arbitrária —, ele permanece um dos métodos mais influentes e serve de base conceitual para várias extensões e algoritmos mais sofisticados, como o k -medoids, o *fuzzy c -means* e os modelos de mistura gaussiana.

2.1 Fundamentação teórica

O algoritmo K -médias é um método de aprendizado não supervisionado, no sentido de que o processo de aprendizado é totalmente autônomo. Sua primeira versão foi proposta em 1957 por Lloyd e ficou conhecida por bastante tempo como *Lloyd's algorithm*.

A ideia básica desse algoritmo consiste em agrupar pontos de acordo com uma medida de similaridade, sendo que os resultados obtidos (agrupamentos) dependem fortemente da medida de similaridade escolhida. Na sua formulação original, o algoritmo K -médias adota a distância Euclidiana como métrica de similaridade.

A distância Euclidiana entre dois vetores $\vec{p}_1, \vec{p}_2 \in \mathbb{R}^d$ é dada pela seguinte equação:

$$d(\vec{p}_1, \vec{p}_2) = \|\vec{p}_1 - \vec{p}_2\| = \sqrt{(p_1^{(1)} - p_2^{(1)})^2 + (p_1^{(2)} - p_2^{(2)})^2 + \dots + (p_1^{(d)} - p_2^{(d)})^2}. \quad (1)$$

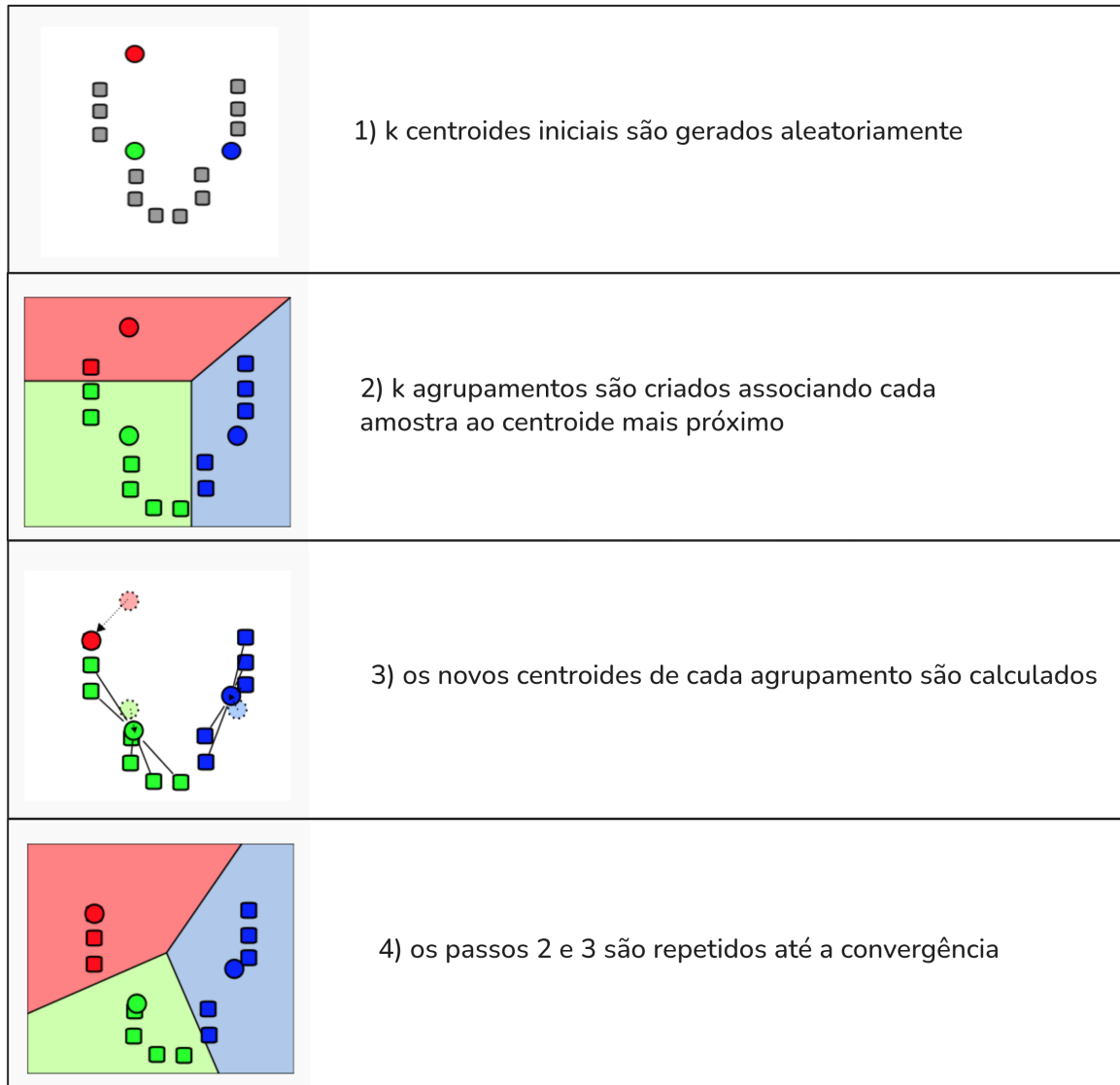
Formulação do Problema

Dado um conjunto de vetores de dados $X = \{\vec{x}_1, \vec{x}_2, \dots, \vec{x}_n\}$, com $\vec{x}_i \in \mathbb{R}^d$, o objetivo do algoritmo é particionar os n vetores em $k < n$ grupos $S = \{s_1, s_2, \dots, s_k\}$ de modo a minimizar o espalhamento intra-*cluster*. Formalmente, o problema pode ser descrito como a busca pela partição ótima S^* que satisfaz o seguinte critério:

$$S^* = \operatorname{argmin}_S \sum_{i=1}^k \sum_{\vec{x} \in s_i} \|\vec{x} - \vec{\mu}_i\|^2, \quad (2)$$

onde $\vec{\mu}_i$ representa o centróide (ponto médio) do agrupamento s_i .

A função objetivo acima determina que o centro de cada *cluster* deve estar o mais próximo possível dos dados que compõem aquele grupo. No entanto, encontrar a solução ótima para esse problema é *NP-hard* para um número arbitrário de dimensões d . A heurística adotada pelo *K-médias* para simplificar o problema consiste em fixar previamente o número de agrupamentos K . Assim, o parâmetro K (número de *clusters*) deve ser definido antes da execução do algoritmo.



Em resumo, o resultado a seguir mostra como encontrar o centro ótimo de um dado agrupamento.

Teorema 1 (Centro ótimo para um agrupamento). *Dado um agrupamento não vazio s_k , seu centróide ou média é a **única** escolha de centro que minimiza o espalhamento interno:*

$$\sum_{\vec{x} \in s_k} \|\vec{x} - \vec{c}^{(k)}\|^2. \quad (3)$$

Demonstração. Note que o critério acima é equivalente a:

$$\sum_{\vec{x} \in s_k} \sum_{i=1}^d (x_i - c_i^{(k)})^2 = \sum_{\vec{x} \in s_k} \sum_{i=1}^d \left[x_i^2 - 2x_i c_i^{(k)} + (c_i^{(k)})^2 \right]. \quad (4)$$

Intercambiando as somas, podemos reescrever:

$$\sum_{\vec{x} \in s_k} \sum_{i=1}^d (x_i - c_i^{(k)})^2 = \sum_{i=1}^d \left[\sum_{\vec{x} \in s_k} x_i^2 - 2 \left(\sum_{\vec{x} \in s_k} x_i \right) c_i^{(k)} + |s_k| (c_i^{(k)})^2 \right]. \quad (5)$$

A expressão acima é uma função quadrática em $c_i^{(k)}$. Derivando-a em relação a $c_i^{(k)}$ e igualando a zero, obtemos:

$$-2 \sum_{\vec{x} \in s_k} x_i + 2|s_k| c_i^{(k)} = 0, \quad (6)$$

o que implica:

$$c_i^{(k)} = \frac{1}{|s_k|} \sum_{\vec{x} \in s_k} x_i. \quad (7)$$

Trata-se de uma função convexa, o que garante que o ponto encontrado é um ótimo global. Assim, o centróide $\vec{c}^{(k)}$ corresponde à média aritmética dos atributos das amostras pertencentes ao agrupamento s_k . $\square$

O teorema a seguir mostra que a regra do centróide mais próximo — isto é, atribuir cada ponto $\vec{x}$ ao grupo cujo centro está mais próximo de $\vec{x}$ — produz uma solução ótima para o problema de agrupamento do algoritmo k -médias, quando os centros são fixos.

Teorema 2 (Agrupamentos ótimos para um conjunto de centros). *Dado um conjunto X de n amostras e uma sequência de K centros $\vec{c}^{(1)}, \vec{c}^{(2)}, \dots, \vec{c}^{(K)}$, uma partição em agrupamentos $s_1, s_2, \dots, s_K$ minimiza a função objetivo*

$$\sum_{i=1}^K \sum_{\vec{x} \in s_i} \|\vec{x} - \vec{c}^{(i)}\|^2, \quad (8)$$

se o algoritmo associa cada amostra $\vec{x}$ ao agrupamento s_i que minimiza a distância

$$\|\vec{x} - \vec{c}^{(i)}\|^2. \quad (9)$$

Demonstração. A prova é direta. Cada amostra $\vec{x}$ contribui exatamente uma única vez ao somatório definido em (8). Assim, ao escolher alocar $\vec{x}$ no agrupamento cujo centro $\vec{c}^{(i)}$ é o mais próximo, minimiza-se a contribuição individual dessa amostra à função objetivo. Portanto, a regra do centróide mais próximo leva a uma partição ótima para um conjunto fixo de centros. $\square$

O algoritmo a seguir ilustra o pseudocódigo do k-médias.

Algoritmo 1 Algoritmo k-médias para agrupamento de dados

```
// Parâmetros:
// X: a matriz de dados  $n \times d$  (cada linha é uma amostra)
// n: o número de amostras
// k: o número de agrupamentos
function K-MÉDIAS( $X, n, k$ )
     $\vec{s}_1, \vec{s}_2, \dots, \vec{s}_k \leftarrow \text{select\_random\_seeds}(X, k)$  ▷ Selecione k centroides aleatórios
    for  $i \leftarrow 1; i \leq k; i++$  do
         $\vec{\mu}_i \leftarrow \vec{s}_i$ 
    end for
    while not convergence do
        for  $i \leftarrow 1; i \leq k; i++$  do
             $\omega_i \leftarrow \{\}$  ▷ Inicie com  $k$  partições vazias
        end for
        for  $i \leftarrow 1; i \leq n; i++$  do
             $j \leftarrow \arg \min_l \|\vec{\mu}_l - \vec{x}_i\|$  ▷ Centroide mais próximo
             $\omega_j \leftarrow \omega_j \cup \{\vec{x}_i\}$  ▷ Associe amostra ao agrupamento
        end for
        for  $i \leftarrow 1; i \leq k; i++$  do
             $\vec{\mu}_i \leftarrow \frac{1}{|\omega_i|} \sum_{\vec{x} \in \omega_i} \vec{x}$  ▷ Recalcular os centroides
        end for
    end while
    return  $\omega_1, \omega_2, \dots, \omega_k$ 
end function
```

A convergência do algoritmo k-médias é geralmente medida de três maneiras diferentes:

1. os centróides dos clusters recém-formados não se movem ou seu movimento é menor que um limite;
2. os pontos de dados permanecem nos mesmos clusters;
3. um número máximo de iterações é atingido.

Análise da complexidade

A complexidade do algoritmo K-médias depende essencialmente dos seguintes fatores:

- N : número de amostras;

- K : número de centros (ou clusters);
- D : dimensão das amostras;
- T : número de iterações até a convergência (geralmente desconhecido a priori).

Primeiramente, note que tanto a função `select_random_seeds(X, K)` quanto o laço `for` inicial (fora do `while`) são executados em tempo $O(K)$. A parte crítica do algoritmo é a execução do laço `while`, que analisaremos a seguir.

Para determinar o centróide mais próximo de uma amostra $\vec{x}_i$, é necessário compará-la com todos os K centróides. Cada comparação envolve o cálculo de uma distância Euclidiana, cujo custo é $O(D)$, já que o vetor tem D componentes.

Assim, para decidir qual é o centróide mais próximo de $\vec{x}_i$, o custo computacional é:

$$O(KD).$$

Como esse processo é repetido para todas as N amostras, o custo total por iteração é:

$$O(NKD).$$

Por fim, como o processo completo se repete por um número desconhecido T de iterações até a convergência, a complexidade total do algoritmo K -médias é dada por:

$$O(TNKD).$$

Apesar de envolver múltiplos fatores, observa-se que o algoritmo é linear em relação ao número de amostras N , o que justifica sua ampla utilização em aplicações de grande escala.

Tabela 1: Resumo dos principais componentes de custo do algoritmo K -médias.

Etapas	Descrição	Custo Assintótico
Inicialização	Seleção centróides iniciais	$O(K)$
Atribuição	Cálculo das distâncias e atribuição das amostras aos centróides	$O(NKD)$
Atualização	Recomputar os centróides com base nas novas atribuições	$O(ND)$
Iterações	Repetição das etapas até a convergência	$O(TNKD)$

Em síntese, o custo total do algoritmo é dominado pela etapa de atribuição, que cresce linearmente com o número de amostras N e com o número de iterações T até a convergência.

Estratégias de aceleração

Embora o algoritmo K -médias seja eficiente para conjuntos de dados de tamanho moderado, sua aplicação direta pode se tornar proibitiva quando N e D são grandes. Diversas estratégias foram

propostas para reduzir o custo computacional sem comprometer significativamente a qualidade dos agrupamentos. Entre as principais, destacam-se:

- **Uso de estruturas de indexação (KD-trees, Ball-trees):** Essas estruturas permitem acelerar a busca pelo centróide mais próximo, reduzindo o número de distâncias efetivamente calculadas. Em vez de comparar cada ponto com todos os K centróides, as árvores espaciais possibilitam podar regiões do espaço onde nenhum centróide mais próximo possa existir.
- **Mini-Batch K-means:** Proposto por Sculley (2010), este método processa pequenos lotes (*mini-batches*) de dados em cada iteração, atualizando incrementalmente os centróides. Essa abordagem reduz drasticamente o custo por iteração e permite lidar com grandes volumes de dados em ambientes de fluxo contínuo.
- **Inicialização aprimorada (K-means++):** A inicialização aleatória tradicional pode levar a convergência lenta ou a mínimos locais ruins. O método *K-means++* (Arthur & Vassilvitskii, 2007) seleciona centróides iniciais de forma probabilística, priorizando pontos mais distantes entre si, o que tende a reduzir o número de iterações T .
- **Aproximações baseadas em amostragem:** Em cenários de altíssimo volume de dados, pode-se realizar o agrupamento sobre uma amostra representativa e, posteriormente, refinar os centróides com base no conjunto completo.
- **Paralelização e computação distribuída:** Implementações modernas do K -médias (como no Apache Spark) distribuem as amostras entre múltiplos nós de processamento, tornando o custo efetivo sublinear em N e viabilizando sua aplicação em escala massiva.

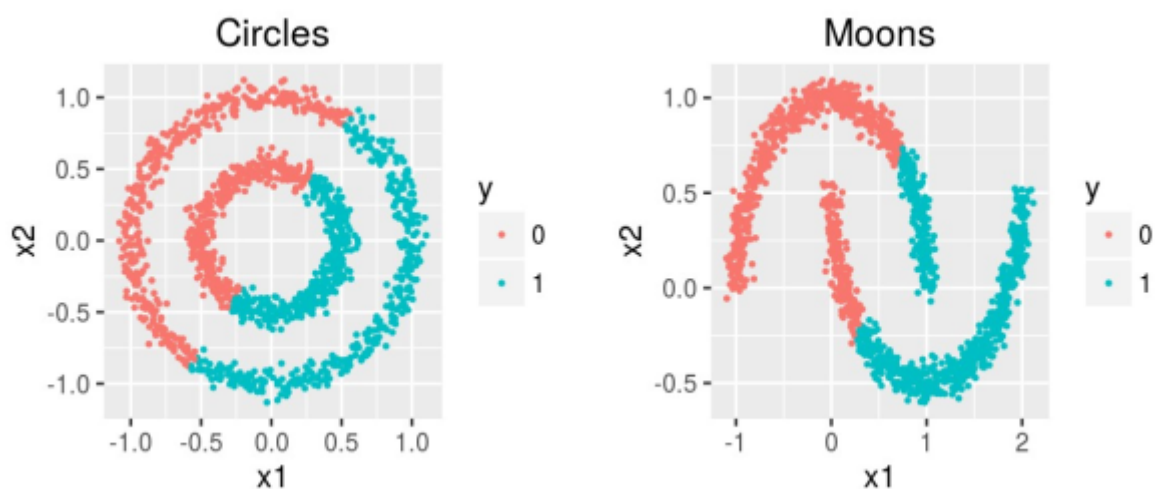
Essas variações e otimizações demonstram que, embora o K -médias original seja conceitualmente simples, seu desempenho prático pode ser substancialmente melhorado por meio de técnicas computacionais e heurísticas bem escolhidas.

2.2 Limitações do algoritmo k-médias

Apesar de ser amplamente utilizado na prática, o algoritmo K -médias apresenta algumas limitações importantes que devem ser consideradas na sua aplicação:

1. **Sensibilidade à escolha randômica dos centros iniciais:** A inicialização aleatória dos centróides pode levar a soluções locais de baixa qualidade. Frequentemente, utiliza-se algum tipo de supervisão ou conhecimento prévio na escolha dos centros iniciais, por exemplo, selecionando uma amostra representativa de cada classe. Essa abordagem, contudo, requer o conhecimento prévio dos rótulos (*labels*) das amostras, o que pode não estar disponível em cenários não supervisionados.

2. **Escolha incorreta do número de agrupamentos (K):** A determinação adequada do valor de K é uma tarefa não trivial e pode influenciar diretamente na qualidade dos agrupamentos obtidos. Diversas heurísticas e métodos, como o *método do cotovelo* (*Elbow Method*) ou o *silhouette score*, são comumente empregados para auxiliar nessa escolha.
3. **Formato dos agrupamentos:** O algoritmo K-médias tende a encontrar apenas agrupamentos compactos e aproximadamente esféricos, devido ao uso da distância Euclidiana como medida de similaridade. Quando os dados possuem estruturas de agrupamentos com formatos não esféricos, densidades diferentes ou tamanhos desbalanceados, o K-médias pode falhar em identificá-los corretamente.



2.3 O algoritmo k-means++

O algoritmo k -means é um dos métodos mais utilizados em aprendizado de máquina e mineração de dados para a tarefa de agrupamento (*clustering*). Sua popularidade decorre da simplicidade conceitual, da facilidade de implementação e da rapidez com que converge em muitos casos práticos. Entretanto, apesar dessas vantagens, o método tradicional apresenta uma limitação fundamental: a **forte dependência da escolha inicial dos centróides**. Como os centros são usualmente escolhidos de forma aleatória, o resultado final pode variar significativamente entre execuções, levando a soluções subótimas, especialmente quando os dados possuem agrupamentos de tamanhos ou densidades diferentes.

Essa sensibilidade à inicialização motivou a busca por estratégias mais robustas de escolha dos centróides iniciais. O algoritmo k -means++, proposto por Arthur e Vassilvitskii (2007), surgiu como uma resposta simples e eficaz a esse problema. A ideia central consiste em introduzir um processo probabilístico guiado pela estrutura dos dados, em vez de uma seleção totalmente aleatória. Em particular, o k -means++ atribui a cada ponto uma probabilidade de ser escolhido proporcional ao quadrado da sua distância em relação ao centróide mais próximo já selecionado — o chamado **método de ponderação D^2** . Essa estratégia aumenta a chance de escolher centros bem espaçados, evitando

concentrações iniciais inadequadas e promovendo uma cobertura mais representativa do espaço dos dados.

Do ponto de vista teórico, o k -means++ fornece uma melhoria substancial sobre o k -means clássico: ele é $O(\log k)$ -competitivo em relação à solução ótima, o que significa que, em expectativa, o erro obtido por sua inicialização está dentro de um fator logarítmico do mínimo possível. Essa garantia de desempenho era inexistente para o método tradicional. Além disso, o algoritmo mantém o mesmo nível de simplicidade e eficiência computacional, o que o torna particularmente atrativo para aplicações em larga escala.

Na prática, os experimentos de Arthur e Vassilvitskii mostraram que o k -means++ não apenas melhora a **qualidade dos agrupamentos finais**, mas também **acelera a convergência** do processo iterativo. Isso ocorre porque, ao partir de uma configuração inicial mais promissora, o número de iterações necessárias para a estabilização tende a ser significativamente menor. Em diversas bases de dados reais e sintéticas, o k -means++ produziu resultados mais precisos e consistentes, evidenciando a importância de uma fase de inicialização cuidadosa como parte integrante do processo de *clustering*.

Motivação e formulação do problema

O algoritmo k -means++ foi proposto com o objetivo de mitigar uma limitação fundamental do k -means clássico: a dependência da inicialização aleatória dos centróides. Essa etapa inicial é crítica, pois determina a qualidade da solução final e o tempo de convergência. Inicializações ruins podem levar a mínimos locais de baixa qualidade, resultando em particionamentos distorcidos e em grande variabilidade entre diferentes execuções do algoritmo.

O problema central abordado por Arthur e Vassilvitskii é o de escolher um conjunto inicial de centróides que forneça uma boa aproximação ao agrupamento ótimo. Seja $X = \{\vec{x}_1, \vec{x}_2, \dots, \vec{x}_n\} \subset \mathbb{R}^d$ o conjunto de amostras, e $C = \{\vec{c}_1, \vec{c}_2, \dots, \vec{c}_K\}$ o conjunto de centróides. O objetivo do k -means é minimizar a função custo:

$$\Phi(C) = \sum_{\vec{x} \in X} \min_{\vec{c} \in C} \|\vec{x} - \vec{c}\|^2. \quad (10)$$

No algoritmo tradicional, os centróides iniciais são escolhidos aleatoriamente e, em seguida, o processo iterativo de atualização é aplicado até a convergência. Já no k -means++, a inicialização é probabilística e guiada pela estrutura dos dados, de modo a minimizar a expectativa do custo inicial $\Phi(C)$ antes mesmo da primeira iteração.

Estratégia de inicialização ponderada (D^2 -seeding)

A principal contribuição teórica do k -means++ é a estratégia de inicialização baseada em *ponderação quadrática por distância* (D^2 -weighting). O procedimento é descrito da seguinte forma:

1. Escolhe-se o primeiro centróide $\vec{c}_1$ aleatoriamente entre os pontos de X .

2. Para cada ponto $\vec{x} \in X$, calcula-se $D(\vec{x})$, a distância entre $\vec{x}$ e o centróide mais próximo já escolhido:

$$D(\vec{x}) = \min_{\vec{c} \in C} \|\vec{x} - \vec{c}\|.$$

3. O próximo centróide é escolhido aleatoriamente entre os pontos de X com probabilidade proporcional a $D(\vec{x})^2$:

$$P(\vec{x}) = \frac{D(\vec{x})^2}{\sum_{\vec{y} \in X} D(\vec{y})^2}.$$

4. O processo é repetido até que K centróides sejam selecionados.

Essa escolha probabilística tende a favorecer pontos distantes dos centróides já escolhidos, promovendo uma boa cobertura inicial do espaço de dados e reduzindo significativamente o risco de convergência a soluções locais ruins.

Análise de desempenho e garantia teórica

Arthur e Vassilvitskii demonstraram que o conjunto inicial de centróides obtido pelo k -means++ garante, em expectativa, um valor da função custo (10) no máximo $O(\log K)$ vezes pior do que o custo ótimo global Φ_{OPT} :

$$\mathbb{E}[\Phi(C)] \leq 8(\log K + 2) \Phi_{\text{OPT}}. \quad (11)$$

Esse resultado teórico é notável, pois fornece uma **garantia de aproximação** para um problema que, em sua forma geral, é *NP-hard*. Em contraste, o k -means tradicional não possui nenhuma garantia de desempenho — a qualidade da solução depende inteiramente da inicialização e da estrutura dos dados.

Além disso, os autores mostraram que essa inicialização melhora não apenas a qualidade da solução final, mas também a velocidade de convergência. Como o algoritmo inicia com centróides mais representativos, o número médio de iterações necessárias para atingir a convergência é reduzido, mantendo a complexidade total assintoticamente equivalente ao k -means original.

Implicações práticas

Na prática, o k -means++ apresenta resultados consistentemente melhores do que a versão clássica, tanto em conjuntos de dados sintéticos quanto reais. A melhoria na escolha dos centróides iniciais reduz a variabilidade entre execuções e tende a produzir agrupamentos mais estáveis e interpretáveis. Além disso, o custo adicional introduzido pela fase de inicialização é marginal — linear no número de amostras N — e, portanto, não compromete a eficiência do método.

Assim, o k -means++ representa uma evolução conceitualmente simples, porém teoricamente fundamentada e empiricamente eficaz, consolidando-se como o padrão moderno de inicialização para o algoritmo k -means.

2.3.1 Pseudocódigo

O pseudocódigo a seguir descreve o processo completo de inicialização do k -means++ seguido pelas iterações padrão do algoritmo k -means. A fase de inicialização (D^2 -seeding) garante uma escolha mais informada dos centróides iniciais, melhorando a qualidade da solução final (agrupamentos) e acelerando a convergência.

Algoritmo 2 Algoritmo k -means++

Require: Conjunto de dados $X = \{\vec{x}_1, \vec{x}_2, \dots, \vec{x}_N\}$, número de clusters K

Ensure: Conjunto de centróides finais $C = \{\vec{c}_1, \vec{c}_2, \dots, \vec{c}_K\}$

- 1: Escolher aleatoriamente o primeiro centróide $\vec{c}_1$ entre os pontos de X
- 2: **for** $i = 2$ **até** K **do**
- 3: **for** cada ponto $\vec{x} \in X$ **do**
- 4: Calcular $D(\vec{x}) = \min_{\vec{c} \in C} \|\vec{x} - \vec{c}\|$
- 5: **end for**
- 6: Escolher $\vec{c}_i$ de X com probabilidade

$$P(\vec{x}) = \frac{D(\vec{x})^2}{\sum_{\vec{y} \in X} D(\vec{y})^2}$$

- 7: Adicionar $\vec{c}_i$ ao conjunto de centróides C
- 8: **end for**

Iterações do k -means padrão:

- 9: **repeat**
- 10: **for** cada ponto $\vec{x} \in X$ **do**
- 11: Atribuir $\vec{x}$ ao cluster s_i cujo centróide $\vec{c}_i$ é o mais próximo
- 12: **end for**
- 13: **for** cada cluster s_i **do**
- 14: Atualizar o centróide:

$$\vec{c}_i = \frac{1}{|s_i|} \sum_{\vec{x} \in s_i} \vec{x}$$

- 15: **end for**
 - 16: **until** os centróides não se alterarem significativamente ou atingir número máximo de iterações
 - 17: **Retornar** conjunto final de centróides C
-

2.4 O algoritmo *Fuzzy C-Means*

O algoritmo *Fuzzy C-Means* (FCM) representa uma extensão natural do método K -médias, introduzindo o conceito de pertencimento parcial de uma amostra a diferentes agrupamentos. Enquanto o K -médias atribui cada amostra exclusivamente a um único cluster, o FCM permite que cada ponto de dados pertença simultaneamente a vários grupos, com diferentes graus de associação. Essa característica é especialmente útil em contextos onde as fronteiras entre os agrupamentos não são bem definidas ou quando há sobreposição entre as classes.

A motivação fundamental do FCM decorre do fato de que, em muitos conjuntos de dados reais, as

divisões entre os agrupamentos são difusas e graduais, e não estritamente delimitadas. Em problemas de reconhecimento de padrões, visão computacional, análise de imagens e bioinformática, é comum encontrar amostras que compartilham características de mais de um grupo. O modelo de agrupamento “suave” proposto pelo FCM busca representar essa incerteza de forma explícita, oferecendo uma interpretação mais realista da estrutura subjacente dos dados.

Matematicamente, o FCM minimiza uma função objetivo semelhante à do K -médias, mas ponderada por graus de pertinência *fuzzy*. Cada amostra $\vec{x}_i$ possui um conjunto de valores $u_{ij} \in [0, 1]$, que expressam o quanto $\vec{x}_i$ pertence ao cluster j . Um parâmetro de controle $m > 1$, denominado **expoente de fuzzificação**, regula o grau de sobreposição entre os agrupamentos: quanto maior o valor de m , mais difusa é a partição. Assim, o FCM não apenas identifica agrupamentos, mas também quantifica o nível de incerteza associado à atribuição de cada ponto.

Na prática, o algoritmo FCM é particularmente eficaz em cenários nos quais os dados apresentam ruído, ambiguidade ou transições graduais entre regiões de interesse, como na segmentação de imagens médicas, na classificação de texturas e na análise de padrões complexos. Combinando flexibilidade interpretativa e fundamentação matemática rigorosa, o FCM constitui uma das abordagens mais importantes entre os métodos de agrupamento baseados em teoria *fuzzy*.

Formulação matemática do *Fuzzy C-Means*

Seja $X = \{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_N\} \subset \mathbb{R}^d$ o conjunto de amostras e C o número de clusters desejado. No *Fuzzy C-Means* introduzimos a matriz de pertinência

$$U = [u_{ij}] \in [0, 1]^{N \times C},$$

onde u_{ij} representa o grau de pertinência da amostra $\mathbf{x}_i$ ao cluster j . As pertinências satisfazem a restrição de soma unitária por amostra:

$$\sum_{j=1}^C u_{ij} = 1 \quad \text{para todo } i = 1, \dots, N. \quad (12)$$

O *Fuzzy C-Means* minimiza a seguinte função objetivo (também chamada de energia ou potencial):

$$J_m(U, \{\mathbf{c}_j\}) = \sum_{i=1}^N \sum_{j=1}^C u_{ij}^m \|\mathbf{x}_i - \mathbf{c}_j\|^2, \quad (13)$$

onde $m > 1$ é o *expoente de fuzzificação* (ou parâmetro de *fuzziness*) que controla o grau de suavidade da partição. Tipicamente usa-se $m \in [1.5, 2.5]$ e muitas aplicações adotam $m = 2$.

As atualizações que levam a um mínimo local de J_m são obtidas alternando entre a otimização em relação a U (fixos os centróides $\mathbf{c}_j$) e a otimização em relação aos centróides (fixa U). A seguir estão as fórmulas clássicas de atualização.

Atualização dos centróides

Fixando U , a derivada de J_m em relação a $\mathbf{c}_j$ igualada a zero fornece a expressão do centróide ótimo:

$$\mathbf{c}_j = \frac{\sum_{i=1}^N u_{ij}^m \mathbf{x}_i}{\sum_{i=1}^N u_{ij}^m}, \quad j = 1, \dots, C. \quad (14)$$

Ou seja, cada centróide é a média ponderada das amostras, com pesos u_{ij}^m .

Atualização das pertinências

Fixando os centróides $\{\mathbf{c}_j\}$ e impondo a restrição (12) via multiplicadores de Lagrange, obtém-se a fórmula de atualização das pertinências:

$$u_{ij} = \frac{1}{\sum_{k=1}^C \left(\frac{\|\mathbf{x}_i - \mathbf{c}_k\|}{\|\mathbf{x}_i - \mathbf{c}_j\|} \right)^{\frac{2}{m-1}}} \quad \text{para } i = 1, \dots, N, \quad j = 1, \dots, C, \quad (15)$$

com a convenção de que se $\|\mathbf{x}_i - \mathbf{c}_k\| = 0$ para algum k (isto é, $\mathbf{x}_i$ coincide com um centróide), então define-se $u_{ik} = 1$ e $u_{ij} = 0$ para $j \neq k$.

Demonstração da fórmula de u_{ij} via multiplicadores de Lagrange

Fixe os centróides $\{\mathbf{c}_j\}_{j=1}^C$ e considere a minimização da função objetivo (para um índice i fixo, referente à amostra $\mathbf{x}_i$)

$$J_i(u_{i1}, \dots, u_{iC}) = \sum_{j=1}^C u_{ij}^m d_{ij}^2, \quad d_{ij} := \|\mathbf{x}_i - \mathbf{c}_j\|,$$

sujeita à restrição de normalização

$$\sum_{j=1}^C u_{ij} = 1.$$

Forme o Lagrangeano com multiplicador λ_i :

$$\mathcal{L}(u_{i1}, \dots, u_{iC}, \lambda_i) = \sum_{j=1}^C u_{ij}^m d_{ij}^2 + \lambda_i \left(\sum_{j=1}^C u_{ij} - 1 \right).$$

Condicionando um extremo, diferenciamos $\mathcal{L}$ em relação a cada u_{ij} e igualamos a zero:

$$\frac{\partial \mathcal{L}}{\partial u_{ij}} = m u_{ij}^{m-1} d_{ij}^2 + \lambda_i = 0.$$

Isolando u_{ij}^{m-1} obtemos

$$u_{ij}^{m-1} = -\frac{\lambda_i}{m} \cdot \frac{1}{d_{ij}^2}.$$

Note que o lado direito é positivo (logo $\lambda_i < 0$), e portanto podemos escrever

$$u_{ij} = \left(\frac{K_i}{d_{ij}^2} \right)^{\frac{1}{m-1}},$$

onde $K_i := \left(-\frac{\lambda_i}{m} \right) > 0$ é uma constante que depende apenas de i . Para determinar K_i aplica-se a restrição de soma:

$$\sum_{j=1}^C u_{ij} = \sum_{j=1}^C \left(\frac{K_i}{d_{ij}^2} \right)^{\frac{1}{m-1}} = 1 \quad \Rightarrow \quad K_i^{\frac{1}{m-1}} = \frac{1}{\sum_{k=1}^C d_{ik}^{-\frac{2}{m-1}}}.$$

Substituindo de volta, obtemos a expressão normalizada:

$$u_{ij} = \frac{d_{ij}^{-\frac{2}{m-1}}}{\sum_{k=1}^C d_{ik}^{-\frac{2}{m-1}}} = \frac{1}{\sum_{k=1}^C \left(\frac{d_{ij}}{d_{ik}} \right)^{\frac{2}{m-1}}}.$$

Essa é a fórmula clássica utilizada para atualizar as pertinências no FCM.

Observação sobre casos degenerados: se algum $d_{ij} = 0$ (isto é, $\mathbf{x}_i = \mathbf{c}_j$), então a derivação acima envolve divisão por zero. Define-se, por convenção, $u_{ij} = 1$ e $u_{ik} = 0$ para $k \neq j$ — isto é, a amostra pertence plenamente ao cluster cujo centróide coincide com ela.

Algoritmo iterativo

O procedimento prático consiste em:

1. Inicializar $U^{(0)}$ (ou os centróides $\{\mathbf{c}_j^{(0)}\}$) aleatoriamente, respeitando $\sum_j u_{ij} = 1$.
2. Repetir até convergência:
 - (a) Atualizar os centróides por (14).
 - (b) Atualizar as pertinências por (15).
3. Parar quando $\|U^{(t)} - U^{(t-1)}\|_\infty < \varepsilon$ ou quando a variação relativa de J_m for pequena.

O procedimento iterativo do algoritmo *Fuzzy C-Means* alterna entre a atualização dos centróides e dos graus de pertinência das amostras, até que uma condição de parada seja satisfeita. O pseudocódigo a seguir resume o método:

Algoritmo 3 Algoritmo *Fuzzy C-Means (FCM)*

Require: Conjunto de dados $X = \{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_N\}$; número de clusters C ; expoente de fuzzificação $m > 1$; tolerância ε

Ensure: Matrizes de pertinência $U = [u_{ij}]$ e centróides $\{\mathbf{c}_j\}$

1: Inicializar aleatoriamente a matriz de pertinência $U^{(0)}$ tal que $\sum_{j=1}^C u_{ij} = 1, \forall i$

2: **repeat**

3: Atualizar os centróides:

$$\mathbf{c}_j = \frac{\sum_{i=1}^N (u_{ij})^m \mathbf{x}_i}{\sum_{i=1}^N (u_{ij})^m}, \quad j = 1, \dots, C$$

4: Atualizar as pertinências:

$$u_{ij} = \frac{1}{\sum_{k=1}^C \left(\frac{\|\mathbf{x}_i - \mathbf{c}_j\|}{\|\mathbf{x}_i - \mathbf{c}_k\|} \right)^{\frac{2}{m-1}}}, \quad i = 1, \dots, N, j = 1, \dots, C$$

5: Calcular a variação $\Delta U = \|U^{(t)} - U^{(t-1)}\|$

6: **until** $\Delta U < \varepsilon$ ou número máximo de iterações atingido

7: **Retornar** os centróides finais $\{\mathbf{c}_j\}$ e a matriz U

Propriedades e observações práticas

- **Convergência:** o algoritmo alternado monotonamente diminui J_m e converge para um ponto estacionário (geralmente um mínimo local), analogamente ao K-médias. Não há garantia de alcançar o ótimo global.
- **Papel do parâmetro m :** quando $m \rightarrow 1^+$, as pertinências tendem a 0/1 e o FCM aproxima o particionamento duro (K-médias). Valores maiores de m tornam a partição mais difusa.
- **Complexidade:** cada iteração exige o cálculo de distâncias entre cada amostra e cada centróide, logo o custo por iteração é $O(NCd)$. Se T for o número de iterações, a complexidade total é $O(TNCd)$.
- **Inicialização:** como em outros métodos de otimização local, o resultado depende da inicialização; recomenda-se executar várias vezes com inicializações diferentes ou usar técnicas de inicialização mais cuidadosas.
- **Robustez a ruído:** o FCM pode ser sensível a outliers, pois as amostras ruidosas influenciam os centróides por meio dos pesos u_{ij}^m . Extensões como *possibilistic c-means* ou variantes robustas foram propostas para mitigar isso.
- **Interpretação:** as pertinências u_{ij} fornecem uma medida direta de incerteza na associação das amostras aos clusters e são úteis em tarefas que exigem decisões suaves (ex.: segmentação de imagem com transições).

Entre extensões e variantes relevantes destacam-se:

- **Possibilistic C-Means (PCM):** remove a restrição de soma unitária das pertinências, tratando inconsistências e outliers.
- **Fuzzy c-means kernelizado:** aplica FCM em espaços de características implícitos via núcleos (kernel).
- **Gustafson–Kessel:** usa matrizes de covariância por cluster para modelar formas elípticas.

2.5 Considerações finais

O algoritmo k-médias e suas variantes constituem uma das bases conceituais mais sólidas e influentes no campo do agrupamento de dados. Sua simplicidade, eficiência e fácil interpretação tornam-no uma ferramenta de referência tanto para aplicações práticas quanto para estudos teóricos sobre aprendizado não supervisionado. A partir de um modelo geométrico e intuitivo de representação por protótipos, o *K*-médias demonstra como é possível obter uma partição eficiente de grandes conjuntos de dados utilizando princípios de minimização de erro quadrático médio.

No entanto, como discutido ao longo deste capítulo, a versão clássica do algoritmo apresenta algumas limitações importantes, como a sensibilidade à inicialização aleatória, a necessidade de especificar previamente o número de clusters e a incapacidade de lidar adequadamente com agrupamentos de formatos não esféricos ou com variâncias distintas. Essas restrições motivaram o surgimento de diversas extensões e aperfeiçoamentos.

Entre essas variantes, o k-means++ destaca-se por introduzir uma estratégia de inicialização probabilística (*D²-seeding*) que melhora significativamente a qualidade das soluções e acelera a convergência, além de oferecer garantias teóricas de aproximação em relação ao ótimo global. Já o *Fuzzy C-Means* amplia o paradigma de particionamento rígido, permitindo que cada amostra pertença a múltiplos clusters com diferentes graus de pertinência, tornando-se especialmente útil em cenários com fronteiras difusas ou sobreposição entre grupos.

Essas variantes demonstram que, embora o k-médias seja um método simples, ele constitui o ponto de partida para uma ampla família de algoritmos baseados em protótipos, capazes de capturar diferentes aspectos da estrutura dos dados. Mais do que um algoritmo específico, o *K*-médias representa uma *ideia central* na ciência de dados: a busca por representações compactas e interpretáveis que revelem a organização intrínseca dos conjuntos de dados.

Nos capítulos seguintes, exploraremos abordagens alternativas ao agrupamento por protótipos, incluindo métodos baseados em densidade, hierarquia e grafos, que ampliam ainda mais as possibilidades de análise e oferecem soluções mais flexíveis para conjuntos de dados complexos e de alta dimensionalidade.

Tabela 2: Comparação entre os algoritmos K -médias, K -means++ e *Fuzzy C-Means*.

Característica	K -médias	K -means++	<i>Fuzzy C-Means</i>
Tipo de partição	Hard (cada ponto pertence a um único cluster).	Hard (igual ao K -médias).	Suave (<i>fuzzy</i>); cada ponto possui graus de pertinência em vários clusters.
Critério de otimização	Minimiza a soma das distâncias quadráticas dentro dos clusters: $\sum_{i=1}^K \sum_{\vec{x} \in s_i} \ \vec{x} - \vec{c}_i\ ^2.$	Mesmo critério do K -médias, porém com inicialização melhorada.	Minimiza $J_m = \sum_{i=1}^N \sum_{j=1}^K u_{ij}^m \ \vec{x}_i - \vec{c}_j\ ^2$, com u_{ij} representando os graus de pertinência.
Inicialização	Aleatória (pode levar a mínimos locais ruins).	Probabilística guiada pela distância (D^2 -seeding); produz centróides mais bem distribuídos.	Pode ser aleatória ou baseada em resultados de outros métodos (ex.: K -means).
Garantia teórica	Nenhuma — solução depende fortemente da inicialização.	Garantia de aproximação $O(\log K)$ ao ótimo esperado.	Convergência monotônica para mínimo local de J_m .
Controle de sobreposição	Não permite sobreposição entre clusters.	Idem.	Controlado pelo parâmetro de fuzzificação m ; quanto maior m , mais difusa é a partição.
Sensibilidade a outliers	Alta — centróides deslocam-se facilmente por amostras extremas.	Menor que o K -médias, mas ainda sensível.	Alta — amostras ruidosas afetam as médias ponderadas.
Complexidade computacional	$O(TNKD)$, linear em relação a N .	Mesmo custo assintótico do K -médias, com pequeno acréscimo inicial ($O(NK)$).	$O(TNKD)$, semelhante ao K -médias, porém com custo constante maior devido às atualizações de U .
Vantagens principais	Simplicidade, rapidez e interpretabilidade.	Inicialização robusta e convergência mais estável.	Representa incerteza e fronteiras suaves entre clusters.
Limitações principais	Depende fortemente da inicialização e do formato esférico dos clusters.	Exige cálculo adicional na inicialização; ainda pressupõe clusters esféricos.	Sensível a ruído e escolha do parâmetro m ; possível convergência lenta.

3 Agrupamento hierárquico

O **clustering hierárquico** é uma abordagem clássica e amplamente utilizada para a análise de agrupamentos de dados, que busca organizar objetos em uma estrutura hierárquica de grupos ou clusters. Diferentemente de métodos como o *k-means*, que exigem a definição prévia do número de clusters, o clustering hierárquico constrói uma representação em múltiplos níveis de granularidade, permitindo a visualização da relação de proximidade entre os dados de forma mais intuitiva.

A motivação principal para o uso do clustering hierárquico reside em sua capacidade de revelar a **estrutura inerente aos dados**. Em muitas aplicações, os dados apresentam relações complexas ou aninhadas que não podem ser capturadas adequadamente por métodos de partição plana. Por exemplo, em biologia, é comum que espécies compartilhem ancestrais comuns, formando hierarquias naturais; em marketing, consumidores podem ser agrupados em subsegmentos dentro de segmentos maiores. O clustering hierárquico oferece, portanto, uma maneira de explorar essas relações em diferentes níveis de detalhe.

Além disso, o método é altamente **flexível**, pois não exige conhecimento prévio sobre o número de clusters nem restrições de forma geométrica dos grupos. Ele se baseia apenas em uma *medida de similaridade ou dissimilaridade* entre os objetos, o que permite que seja aplicado a diferentes tipos de dados e domínios. Outra justificativa relevante para o seu uso é a **interpretabilidade visual** que o método proporciona por meio de dendrogramas — diagramas que ilustram a fusão ou divisão dos clusters em diferentes níveis, facilitando a análise exploratória de dados.

Por fim, o clustering hierárquico se mostra particularmente útil em contextos em que a **exploração hierárquica das relações** entre dados é desejável, fornecendo insights que vão além de uma simples segmentação. Essa característica torna o método não apenas uma ferramenta de agrupamento, mas também um instrumento de descoberta de padrões e relações ocultas em conjuntos de dados complexos.

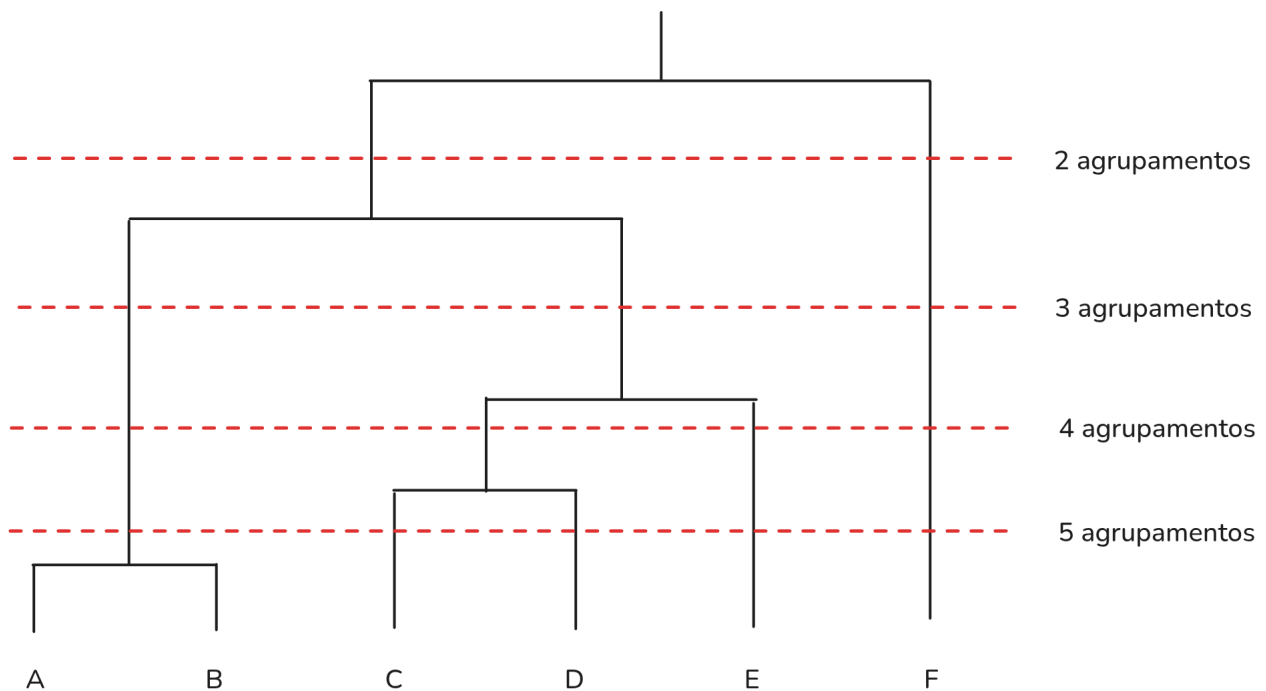
3.1 Abordagens do agrupamento hierárquico

O agrupamento hierárquico é uma das abordagens mais tradicionais e interpretáveis em análise de agrupamentos. Diferentemente dos métodos baseados em protótipos, como o *K-médias*, o agrupamento hierárquico não requer a definição prévia do número de clusters. Em vez disso, ele constrói uma estrutura hierárquica de partições que pode ser representada graficamente por um *dendrograma*, no qual cada nível de corte corresponde a uma possível divisão dos dados em grupos.

Existem duas estratégias principais para a construção dessa hierarquia: a **abordagem aglomerativa** (ou *bottom-up*) e a **abordagem divisiva** (ou *top-down*). Ambas produzem uma sequência de agrupamentos aninhados, mas seguem direções opostas no processo de formação dos clusters.

3.2 Abordagem aglomerativa (Bottom-Up)

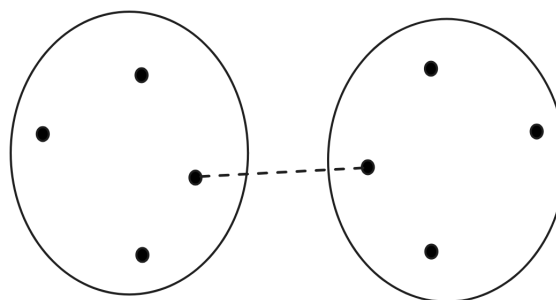
A abordagem **aglomerativa** é a mais amplamente utilizada e conceitualmente simples. O processo tem início considerando cada amostra como um cluster independente. A cada iteração, os dois clusters mais semelhantes são fundidos em um novo agrupamento, reduzindo o número total de clusters em uma unidade. Esse procedimento é repetido até que todos os elementos estejam reunidos em um único grupo, formando uma árvore hierárquica de fusões sucessivas.



A medida de similaridade entre clusters pode ser definida de diversas formas, originando diferentes variantes do método aglomerativo. As formas mais comuns de definir a distância entre dois agrupamentos A e B são:

- **Single Linkage (vizinhança simples):**

$$d(A, B) = \min_{\mathbf{x}_i \in A, \mathbf{x}_j \in B} \|\mathbf{x}_i - \mathbf{x}_j\|.$$

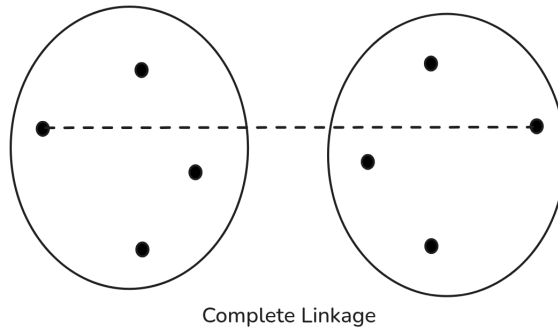


Single Linkage

Essa definição privilegia conexões pontuais entre grupos, favorecendo a formação de cadeias (*chaining effect*).

- **Complete Linkage (vizinhança completa):**

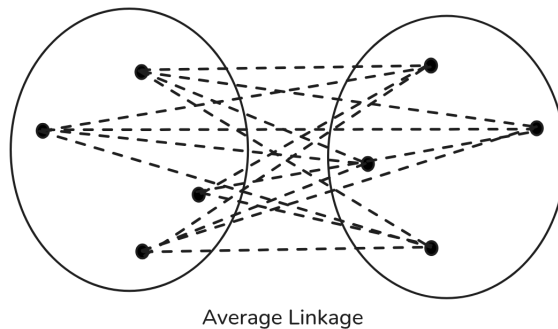
$$d(A, B) = \max_{\mathbf{x}_i \in A, \mathbf{x}_j \in B} \|\mathbf{x}_i - \mathbf{x}_j\|.$$



Gera agrupamentos mais compactos e homogêneos, pois considera a maior distância entre elementos de diferentes clusters.

- **Average Linkage (vizinhança média):**

$$d(A, B) = \frac{1}{|A||B|} \sum_{\mathbf{x}_i \in A} \sum_{\mathbf{x}_j \in B} \|\mathbf{x}_i - \mathbf{x}_j\|.$$



Compromisso entre as abordagens anteriores, levando em conta todas as distâncias entre pares de elementos.

Em cada passo, o par de clusters com menor distância $d(A, B)$ é fundido, e o processo continua até que todos os objetos pertençam a um único agrupamento. O resultado pode ser visualizado em um *dendrograma*, no qual a altura de cada fusão representa a distância entre os agrupamentos unidos.

Algoritmo 4 Agrupamento Hierárquico Aglomerativo (HAC)

Require: Conjunto de dados $X = \{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_N\}$, métrica de distância $d(\cdot, \cdot)$, critério de ligação (*linkage*) definido por uma função de atualização de distância.

Ensure: Dendrograma representando a hierarquia de fusões.

- 1: Inicialize N clusters unitários: $\mathcal{C} = \{\{\mathbf{x}_1\}, \{\mathbf{x}_2\}, \dots, \{\mathbf{x}_N\}\}$
- 2: Calcule a matriz de distâncias entre todos os pares de clusters:

$$D(C_i, C_j) = d(\mathbf{x}_p, \mathbf{x}_q), \quad \mathbf{x}_p \in C_i, \mathbf{x}_q \in C_j$$

segundo o critério de ligação escolhido (e.g., *single*, *complete*, *average*).

- 3: **for** $t = 1$ até $N - 1$ **do**

- 4: Encontre o par de clusters mais próximos:

$$(C_p, C_q) = \arg \min_{C_i, C_j \in \mathcal{C}, i \neq j} D(C_i, C_j)$$

- 5: Una os clusters:

$$C_r = C_p \cup C_q$$

- 6: Atualize o conjunto de clusters:

$$\mathcal{C} \leftarrow (\mathcal{C} \setminus \{C_p, C_q\}) \cup \{C_r\}$$

- 7: Atualize as distâncias entre C_r e os demais clusters $C_k \in \mathcal{C}$ de acordo com o critério de ligação:

$$D(C_r, C_k) = \text{linkage}(C_p, C_q, C_k)$$

- 8: Registre a fusão $(C_p, C_q, D(C_p, C_q))$ no dendrograma.

- 9: **end for**

- 10: **return** Dendrograma com $N - 1$ fusões hierárquicas.
-

Na implementação ingênua, o custo é $O(n^3)$ devido à atualização repetida das distâncias. Com estruturas de dados apropriadas (ex.: heaps ou árvores de vizinhança), o custo pode ser reduzido para $O(n^2 \log n)$. O dendrograma pode ser cortado a qualquer altura h para gerar uma partição em k clusters — bastando interromper o processo após $n - k$ fusões.

3.2.1 Critério de Ward para funções de ligação

Entre os diversos critérios de ligação utilizados no agrupamento hierárquico, o **método de Ward** se destaca por adotar uma abordagem estatística e geométrica distinta das demais. Enquanto métodos como *single linkage* ou *complete linkage* baseiam-se em distâncias entre pares de amostras, o critério de Ward busca minimizar o aumento da *variância intra-cluster* a cada etapa de fusão. Por essa razão, ele é frequentemente interpretado como uma versão hierárquica do algoritmo k-médias.

A ideia central do método de Ward é que, a cada fusão entre dois clusters, deve-se escolher o par cuja união cause o menor incremento possível na soma dos erros quadráticos dentro dos grupos (também conhecida como *within-cluster sum of squares* — WCSS).

Sejam dois agrupamentos A e B , contendo respectivamente n_A e n_B amostras, com centróides

μ_A e μ_B . A função de dispersão intra-cluster associada a um agrupamento C é definida como:

$$W(C) = \sum_{\mathbf{x}_i \in C} \|\mathbf{x}_i - \mu_C\|^2.$$

O objetivo é fundir os clusters A e B de modo a minimizar o aumento total da dispersão:

$$\Delta W = W(A \cup B) - (W(A) + W(B)).$$

Pode-se mostrar que o incremento na variância total devido à fusão de A e B pode ser expresso em função das médias e tamanhos dos grupos como:

$$\Delta W = \frac{n_A n_B}{n_A + n_B} \|\mu_A - \mu_B\|^2.$$

Essa expressão indica que o custo da fusão depende da distância entre os centróides e do tamanho relativo dos grupos. Assim, o par de clusters escolhido a cada iteração é aquele que minimiza ΔW :

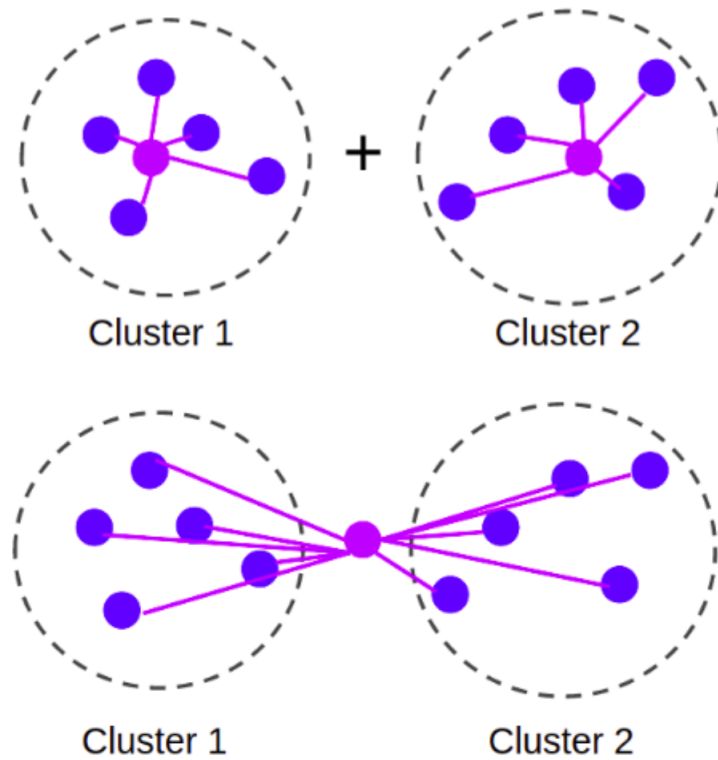
$$(A, B) = \arg \min_{A \neq B} \frac{n_A n_B}{n_A + n_B} \|\mu_A - \mu_B\|^2.$$

O método de Ward pode ser interpretado como uma extensão hierárquica do algoritmo K -médias, pois ambos compartilham o mesmo princípio de minimizar a variância interna dos clusters. A cada fusão, o algoritmo escolhe os agrupamentos cuja combinação resulta na menor perda de homogeneidade, mantendo os clusters o mais compactos e esféricos possível.

Geometricamente, o critério penaliza fusões entre grupos distantes e de tamanhos muito diferentes, favorecendo formações mais balanceadas e coerentes. Estatisticamente, ele pode ser visto como uma aproximação da minimização da variância residual sob um modelo de análise de variância (ANOVA). O critério de Ward apresenta algumas propriedades relevantes:

- Produz clusters mais compactos e homogêneos do que outros critérios de ligação.
- É sensível à escala das variáveis — sendo recomendável padronizar os dados antes da aplicação.
- Favorece partições com clusters de tamanhos semelhantes, o que pode ser uma limitação em dados com densidades desiguais.
- Possui complexidade computacional $O(n^2)$, similar aos demais métodos aglomerativos.

Na prática, o método de Ward é amplamente adotado em aplicações de mineração de dados, bioinformática e psicometria, especialmente quando se deseja uma estrutura hierárquica de agrupamentos que mantenha a coerência de variância interna ao longo de toda a hierarquia.



Unir agrupamentos de modo a minimizar o espalhamento intra-cluster

O critério de Ward avalia a “distância” entre dois agrupamentos como o aumento na soma dos erros quadráticos intra-cluster (WCSS) resultante da fusão. Matematicamente, para clusters A e B :

$$D(A, B) = \arg \min_{A \neq B} \frac{n_A n_B}{n_A + n_B} \|\boldsymbol{\mu}_A - \boldsymbol{\mu}_B\|^2.$$

Isso equivale ao incremento de variância interna causado pela junção dos dois grupos. Assim, o algoritmo busca, em cada iteração, fundir o par de clusters cuja fusão minimize esse aumento de variância, resultando em agrupamentos mais compactos e homogêneos — análogo ao comportamento do k-médias, mas em formato hierárquico.

Algoritmo 5 Agrupamento Hierárquico Aglomerativo com Critério de Ward

Require: Conjunto de dados $X = \{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_N\}$, número de amostras N , métrica de distância $d(\cdot, \cdot)$ (geralmente Euclidiana).

Ensure: Dendrograma representando a hierarquia de fusões segundo o critério de Ward.

1: Inicialize N clusters unitários:

$$\mathcal{C} \leftarrow \{\{\mathbf{x}_1\}, \{\mathbf{x}_2\}, \dots, \{\mathbf{x}_N\}\}$$

2: Para cada par (i, j) de clusters, calcule a distância inicial:

$$D(i, j) = \frac{n_i n_j}{n_i + n_j} \|\boldsymbol{\mu}_i - \boldsymbol{\mu}_j\|^2$$

onde n_i e n_j são as cardinalidades dos clusters C_i e C_j , e $\boldsymbol{\mu}_i$ e $\boldsymbol{\mu}_j$ seus centróides.

3: **for** $t = 1$ até $N - 1$ **do**

4: Encontre o par de clusters mais próximos segundo o critério de Ward:

$$(p, q) = \arg \min_{i \neq j} D(i, j)$$

5: Una os clusters:

$$C_r = C_p \cup C_q$$

6: Atualize o conjunto de clusters:

$$\mathcal{C} \leftarrow (\mathcal{C} \setminus \{C_p, C_q\}) \cup \{C_r\}$$

7: Calcule o novo centróide e tamanho:

$$\boldsymbol{\mu}_r = \frac{n_p \boldsymbol{\mu}_p + n_q \boldsymbol{\mu}_q}{n_p + n_q}, \quad n_r = n_p + n_q$$

8: Atualize as distâncias entre o novo cluster C_r e todos os outros $C_k \in \mathcal{C}$:

$$D(r, k) = \frac{n_r n_k}{n_r + n_k} \|\boldsymbol{\mu}_r - \boldsymbol{\mu}_k\|^2$$

9: Remova as distâncias envolvendo C_p e C_q da matriz D .

10: Registre a fusão $(C_p, C_q, D(p, q))$ no dendrograma.

11: **end for**

12: **return** Dendrograma com $N - 1$ fusões hierárquicas segundo o critério de variância mínima.

Tabela 3: Comparação entre os principais critérios de ligação (linkage) no agrupamento hierárquico.

Critério de Ligação	Características Principais	Comportamento Típico
Single Linkage	Baseia-se na menor distância entre elementos dos clusters. Tende a unir grupos por pontos intermediários (efeito <i>chaining</i>).	Forma clusters alongados e sensíveis a ruído e outliers.
Complete Linkage	Considera a maior distância entre elementos dos clusters. Favorece agrupamentos compactos e de baixa variância interna.	Produz clusters de formato esférico e tamanhos semelhantes.
Average Linkage	Compromisso entre os critérios <i>single</i> e <i>complete</i> . Considera todas as distâncias entre pares de pontos.	Equilibra compactação e conectividade; menos sensível a outliers.
Centroid Linkage	Mede a distância entre os centróides dos clusters. Pode violar a monotonicidade (possibilidade de inversão no dendrograma).	Rápido e intuitivo, mas menos estável em dados não esféricos.
Ward's Linkage	Minimiza o aumento da variância intra-cluster. Garante clusters compactos e homogêneos.	Gera partições similares ao K -médias; sensível à escala das variáveis.

A escolha do critério de ligação influencia diretamente a forma e a estrutura dos agrupamentos resultantes. Enquanto os critérios baseados em distâncias pontuais (*single* e *complete*) enfatizam conectividade local, métodos como *average* e *Ward* privilegiam a homogeneidade global. Na prática, o critério de Ward é frequentemente preferido quando se busca uma partição compacta e estatisticamente consistente.

3.3 Abordagem divisiva (Top-Down)

Na abordagem **divisiva**, o processo é inverso. Inicia-se com um único cluster contendo todas as amostras, e, a cada iteração, o grupo mais heterogêneo é dividido em dois ou mais subgrupos, de acordo com algum critério de dissimilaridade. O procedimento continua de forma recursiva até que cada amostra forme um cluster individual ou até que um critério de parada seja atingido.

Embora conceitualmente mais próxima do raciocínio humano de “dividir para conquistar”, a abordagem divisiva é menos comum na prática devido ao seu maior custo computacional. Enquanto o método aglomerativo possui complexidade $O(n^2)$ (ou $O(n^3)$ em implementações ingênuas), as versões divisivas geralmente exigem a execução de múltiplos agrupamentos internos, tornando-se significativamente mais custosas para grandes conjuntos de dados.

Entre os métodos divisivos mais conhecidos está o **DIANA (Divisive Analysis Clustering)**, proposto por Kaufman e Rousseeuw, que utiliza uma heurística baseada em medidas de dissimilaridade média para decidir quais elementos devem ser separados a cada passo.

Algoritmo 6 Agrupamento Hierárquico Divisivo (Top-Down)

Require: Conjunto de dados $X = \{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_N\}$, métrica de distância $d(\cdot, \cdot)$, critério de divisão (*split*), número desejado de clusters K (ou limiar de heterogeneidade).

Ensure: Dendrograma representando a hierarquia de divisões.

1: Inicialize o conjunto de clusters com todos os dados:

$$\mathcal{C} \leftarrow \{X\}$$

2: Calcule a heterogeneidade de X , por exemplo, pela soma das distâncias quadradas aos centróides:

$$H(C) = \sum_{\mathbf{x}_i \in C} \|\mathbf{x}_i - \mathbf{c}_C\|^2$$

onde $\mathbf{c}_C$ é o centróide de C .

3: **while** $|\mathcal{C}| < K$ e $\exists C \in \mathcal{C}$ com $H(C)$ acima do limiar **do**

4: Escolha o cluster mais heterogêneo:

$$C_{\max} = \arg \max_{C \in \mathcal{C}} H(C)$$

5: Divida $C_{\max}$ em dois subgrupos $\{C_1, C_2\}$ usando um algoritmo de partição interna (ex.: K -médias com $K = 2$ ou corte espectral).

6: Atualize o conjunto de clusters:

$$\mathcal{C} \leftarrow (\mathcal{C} \setminus \{C_{\max}\}) \cup \{C_1, C_2\}$$

7: Registre a divisão ($C_{\max} \rightarrow C_1, C_2$) no dendrograma.

8: Recalcule as heterogeneidades $H(C_1)$ e $H(C_2)$.

9: **end while**

10: **return** Dendrograma com divisões hierárquicas sucessivas.

O método divisivo requer definir como os clusters serão divididos. As abordagens mais comuns incluem:

- Aplicar o algoritmo K -médias com $K = 2$ dentro de cada cluster candidato;
- Utilizar métodos baseados em grafos (ex.: corte mínimo ou máxima modularidade);
- Analisar a variância interna e separar amostras mais distantes do centróide.

Sobre a condição de parada, o processo termina quando:

- O número desejado de clusters K é alcançado; ou
- A heterogeneidade máxima $H(C_{\max})$ fica abaixo de um limiar pré-definido; ou
- Todos os clusters atingem um tamanho mínimo.

O custo computacional depende do método de divisão escolhido. Se k-médias for usado com $K = 2$, o custo aproximado é $O(TNKD)$ por divisão, levando a um custo total da ordem de $O(n^2)$ a $O(n^3)$, dependendo da profundidade da hierarquia. Assim como na abordagem aglomerativa, o resultado é um dendrograma, porém construído no sentido inverso — de uma raiz única até folhas individuais, representando subdivisões sucessivas.

3.4 Comparação entre as abordagens

A principal diferença entre as duas abordagens está na direção do processo:

- Na **abordagem aglomerativa**, a hierarquia é construída por fusões sucessivas (*bottom-up*), partindo de clusters elementares até um agrupamento global.
- Na **abordagem divisiva**, a hierarquia é construída por divisões sucessivas (*top-down*), partindo de um único grupo até múltiplos subgrupos.

Enquanto o método aglomerativo é mais eficiente e amplamente utilizado em aplicações práticas, o método divisivo oferece maior flexibilidade teórica e pode capturar estruturas de agrupamento mais complexas em determinados contextos. Em ambos os casos, o resultado final é um modelo hierárquico que fornece múltiplos níveis de granularidade, permitindo ao analista escolher o número de clusters de acordo com a aplicação.

3.5 Índice Calinski-Harabasz

O **Índice Calinski-Harabasz** (também conhecido como *Calinski–Harabasz Index* ou simplesmente **CH Index**) é amplamente utilizado para determinar o número ótimo de clusters em um particionamento. Esse critério combina as variâncias *intra-cluster* e *inter-cluster* para avaliar a qualidade da segmentação obtida.

Formalmente, o índice é definido como:

$$CH(K) = \frac{B(K)/(K - 1)}{W(K)/(n - K)},$$

onde:

- $W(K)$ representa a variação **intra-cluster** (ou *within-cluster variation*), isto é, a soma das distâncias quadráticas entre os pontos e seus respectivos centróides;
- $B(K)$ representa a variação **inter-cluster** (ou *between-cluster variation*), ou seja, a soma das distâncias quadráticas entre os centróides dos clusters e o centróide global dos dados;
- n é o número total de amostras, e K o número de clusters.

O objetivo do índice é encontrar o número de clusters K^* que equilibra duas propriedades desejáveis:

- **Baixa variância intra-cluster:** os pontos dentro de um mesmo cluster devem ser próximos entre si, minimizando $W(K)$;
- **Alta variância inter-cluster:** os centróides dos clusters devem estar o mais afastados possível entre si, maximizando $B(K)$.

Em termos geométricos, $W(K)$ mede o quão “compactos” são os clusters, enquanto $B(K)$ mede o quão “separados” eles estão no espaço de atributos. Assim, o número ótimo de clusters K^* é aquele que maximiza o valor do índice:

$$K^* = \arg \max_K CH(K).$$

Tipicamente, o gráfico de $CH(K)$ exibe um comportamento semelhante a uma curva unimodal, com um máximo bem definido próximo ao número natural de agrupamentos no conjunto de dados.

Apesar de sua simplicidade e boa eficiência computacional, o índice de Calinski e Harabasz apresenta algumas limitações:

- É mais eficaz quando os clusters possuem formato aproximadamente esférico e variância homogênea;
- Pode superestimar K em conjuntos com alta densidade ou sobreposição de grupos;
- É sensível à presença de ruído e outliers.

Ainda assim, devido à sua interpretação intuitiva e à facilidade de cálculo, o CH Index é amplamente adotado como critério de parada ou validação de agrupamentos em métodos como K -médias, *Gaussian Mixture Models* e abordagens hierárquicas.

A figura 3 ilustra o comportamento típico do índice $CH(K)$ em função do número de clusters K . Observa-se que, inicialmente, o índice aumenta à medida que a segmentação melhora (redução de $W(K)$ e aumento de $B(K)$), até atingir um ponto de máximo — indicando o número de clusters mais apropriado. A partir desse ponto, a subdivisão excessiva dos grupos causa aumento artificial da variância intra-cluster, e o índice tende a decrescer.

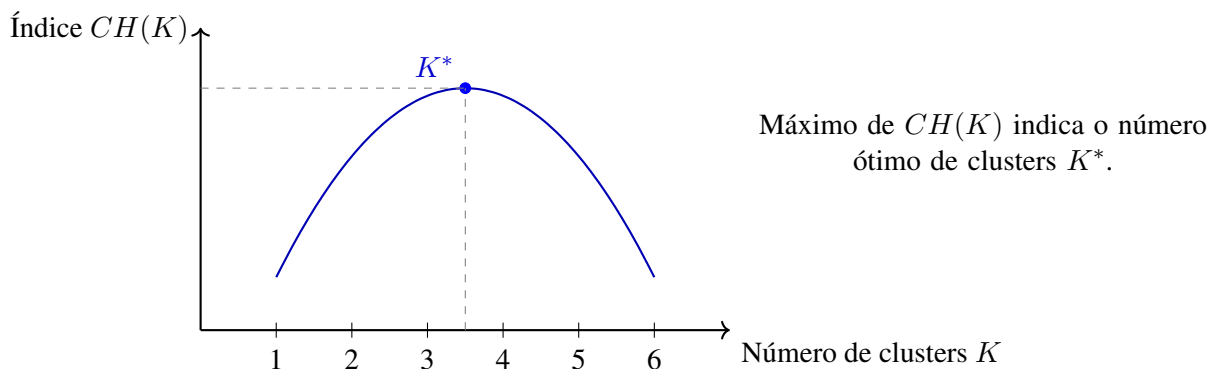


Figura 3: Comportamento típico do Índice Calinski–Harabasz em função do número de clusters K . O máximo da curva indica o número ótimo K^* .

3.6 O método do cotovelo

Uma das principais dificuldades ao utilizar o algoritmo k-médias consiste na escolha do número de clusters, k , a ser utilizado. Como o algoritmo requer esse parâmetro de forma explícita, torna-se essencial adotar algum critério para estimar um valor adequado de k . Entre os métodos mais conhecidos e intuitivos para essa finalidade está o **método do cotovelo** (*Elbow Method*).

A ideia central do método do cotovelo é analisar a variação da **inércia intra-cluster** ou **soma dos erros quadráticos internos** (*Within-Cluster Sum of Squares*, WCSS) à medida que o número de clusters K aumenta. Esse valor é definido como:

$$WCSS(K) = \sum_{k=1}^K \sum_{\vec{x}_i \in S_k} \|\vec{x}_i - \vec{\mu}_k\|^2 \quad (16)$$

onde $\vec{x}_i$ representa uma amostra pertencente ao cluster S_k e $\vec{\mu}_k$ é o centroide do respectivo cluster. Essa medida indica o grau de compactação dos agrupamentos: quanto menor o WCSS, mais próximos estão os pontos de seus centroides.

Com o aumento do número de clusters K , o valor de WCSS tende a diminuir, pois mais grupos implicam em partições menores e, portanto, em menor variabilidade interna. No entanto, essa redução apresenta **retornos decrescentes**: após certo ponto, o ganho em qualidade (redução da variância interna) não compensa o aumento da complexidade do modelo.

O método do cotovelo consiste, então, em plotar a curva de $WCSS(K)$ em função de K e observar o ponto onde ocorre uma mudança perceptível de inclinação — o chamado **cotovelo da curva**. Esse ponto é interpretado como o número ótimo de clusters, pois representa o equilíbrio entre simplicidade e representatividade do modelo. Formalmente, o número ótimo de clusters é aquele que minimiza a soma de variâncias internas sem introduzir subdivisões artificiais.

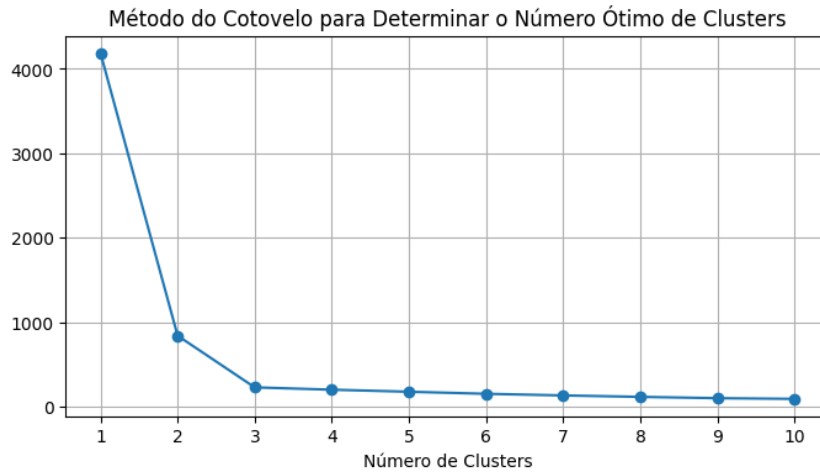


Figura 4: Exemplo ilustrativo do método do cotovelo. O ponto de inflexão da curva indica o número ótimo de clusters.

Apesar de sua simplicidade e interpretação intuitiva, o método do cotovelo apresenta algumas limitações. Em muitos casos, a curva $WCSS(K)$ não apresenta um ponto de inflexão claro, dificultando a escolha do número ótimo de clusters.

tando a identificação do número ótimo de agrupamentos. Além disso, o método baseia-se em uma métrica puramente geométrica, sem considerar eventuais distribuições estatísticas ou características semânticas dos dados.

Mesmo assim, o método do cotovelo permanece como uma das abordagens mais amplamente utilizadas para análise exploratória inicial, sendo frequentemente combinado com outros critérios, como os índices de Calinski–Harabasz, Davies–Bouldin, AIC ou BIC, para uma decisão mais robusta sobre o número ideal de clusters.

3.7 Considerações finais

O agrupamento hierárquico constitui uma das abordagens mais clássicas e interpretáveis no contexto do aprendizado não supervisionado. Sua principal virtude reside na capacidade de revelar a estrutura hierárquica subjacente dos dados, permitindo ao analista visualizar relações de similaridade em múltiplos níveis de granularidade por meio do *dendrograma*. Diferentemente de métodos como o *K*-médiãs, o agrupamento hierárquico não requer a definição prévia do número de clusters, o que o torna especialmente útil em análises exploratórias e na compreensão da organização interna dos conjuntos de dados.

Ao longo deste capítulo, foram discutidas as duas principais estratégias para a construção da hierarquia: a **abordagem aglomerativa**, que parte de clusters individuais e realiza fusões sucessivas (*bottom-up*), e a **abordagem divisiva**, que inicia com um único agrupamento e o divide recursivamente em subgrupos (*top-down*). Embora a versão aglomerativa seja mais comum e eficiente, ambas as estratégias fornecem representações complementares da estrutura dos dados.

Um aspecto crucial do agrupamento hierárquico é a definição do **critério de ligação** (*linkage*), que determina como a similaridade entre clusters é medida a cada etapa. Critérios como *single linkage*, *complete linkage* e *average linkage* refletem diferentes interpretações de proximidade entre grupos, enquanto o **método de Ward** introduz uma formulação estatística que busca minimizar o aumento da variância intra-cluster, produzindo partições mais compactas e homogêneas. A escolha adequada do critério de ligação impacta diretamente na forma final do dendrograma e, consequentemente, na interpretação dos agrupamentos.

Também foram abordadas medidas quantitativas para a **avaliação da qualidade de agrupamentos**, como o Índice Calinski–Harabasz, que combina a variância interna e externa dos clusters para indicar o número ótimo de grupos. Essas métricas complementam a análise visual, fornecendo critérios objetivos para a seleção de partições significativas.

Em termos práticos, o agrupamento hierárquico é amplamente utilizado em bioinformática, análise de textos, psicometria e mineração de dados, onde a estrutura hierárquica é relevante para interpretar relações de similaridade entre instâncias. No entanto, sua complexidade computacional, que cresce quadraticamente com o número de amostras, pode limitar sua aplicação em bases de dados muito extensas. Além disso, a sensibilidade à escolha da métrica de distância e do critério de ligação requer uma análise cuidadosa dos resultados.

Apesar dessas limitações, o agrupamento hierárquico permanece uma ferramenta essencial na análise exploratória de dados. Mais do que um algoritmo, ele oferece uma forma de compreender as relações estruturais entre observações e de revelar padrões ocultos em diferentes escalas. Nos capítulos seguintes, exploraremos métodos alternativos de agrupamento, incluindo abordagens baseadas em densidade, grafos e espectrais, que buscam superar algumas das restrições do modelo hierárquico tradicional, mantendo sua capacidade de revelar a geometria intrínseca dos dados.

4 Modelos de mistura de Gaussianas

Os **Modelos de Mistura de Gaussianas** (*Gaussian Mixture Models* — GMM) representam uma das abordagens mais elegantes e amplamente utilizadas para o problema de agrupamento de dados sob uma perspectiva probabilística. Enquanto métodos como o K -médias ou o agrupamento hierárquico dependem fortemente de medidas geométricas de distância, os modelos de mistura formulam o problema de agrupamento como uma tarefa de inferência estatística, na qual cada amostra é considerada proveniente de uma das várias distribuições probabilísticas subjacentes que compõem a mistura.

A ideia fundamental é supor que os dados observados foram gerados a partir de uma combinação (ou mistura) de K distribuições Gaussianas, cada uma representando um cluster latente na população. Assim, em vez de atribuir uma amostra a um único grupo de maneira rígida, como ocorre em algoritmos de partição clássicos, o GMM associa a cada ponto um conjunto de **probabilidades de pertencimento**, refletindo a incerteza inerente ao processo de agrupamento. Dessa forma, o GMM fornece uma descrição mais flexível e realista dos dados, especialmente em contextos onde os clusters se sobrepõem ou apresentam diferentes formas e orientações.

O objetivo do processo de aprendizado consiste em estimar os parâmetros que melhor expliquem os dados observados. Essa estimação é tipicamente realizada por meio do **Algoritmo da Expectação-Maximização** (*Expectation-Maximization* — EM), um procedimento iterativo que alterna entre duas etapas:

- **Etapa E (Expectação):** calcula a probabilidade de cada amostra pertencer a cada componente da mistura;
- **Etapa M (Maximização):** atualiza os parâmetros das distribuições Gaussianas de forma a maximizar a verossimilhança total dos dados.

A principal vantagem dos modelos de mistura de Gaussianas está em sua capacidade de representar clusters de formas elípticas e com diferentes orientações, superando a limitação do K -médias, que assume fronteiras esféricas. Além disso, o GMM fornece uma base estatística rigorosa para o agrupamento suave (*soft clustering*), permitindo que cada amostra possua uma distribuição de probabilidades sobre os clusters, em vez de uma atribuição exclusiva.

Por outro lado, os GMMs também apresentam desafios: são sensíveis à inicialização, podem convergir para máximos locais da função de verossimilhança e requerem cuidado na escolha do número de componentes K , que pode ser determinado por critérios de informação, como o *Akaike Information Criterion* (AIC) ou o *Bayesian Information Criterion* (BIC).

Em síntese, os modelos de mistura de Gaussianas estabelecem uma ponte entre os métodos de agrupamento geométricos e os probabilísticos, oferecendo um arcabouço matematicamente sólido, interpretável e extensível a diversas aplicações, incluindo reconhecimento de padrões, visão computacional, análise de imagens e modelagem de dados complexos.

4.1 Formulação matemática

Seja um conjunto de dados

$$X = \{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n\}, \quad \mathbf{x}_i \in \mathbb{R}^d,$$

assumimos que as observações são amostras *i.i.d.* (independentes e identicamente distribuídas) de uma densidade desconhecida $p(\mathbf{x})$. Supomos ainda que essa densidade possa ser representada como uma **mistura finita de K componentes Gaussianas**, ou seja,

$$p(\mathbf{x}; \boldsymbol{\theta}) = \sum_{k=1}^K \pi_k p(\mathbf{x} \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k) = \pi_1 p(\mathbf{x} \mid \boldsymbol{\mu}_1, \boldsymbol{\Sigma}_1) + \pi_2 p(\mathbf{x} \mid \boldsymbol{\mu}_2, \boldsymbol{\Sigma}_2) + \dots + \pi_K p(\mathbf{x} \mid \boldsymbol{\mu}_K, \boldsymbol{\Sigma}_K),$$

onde:

$$\sum_{k=1}^K \pi_k = 1, \quad \pi_k \geq 0, \quad \text{e} \quad \boldsymbol{\theta} = (\pi_1, \dots, \pi_K, \boldsymbol{\mu}_1, \dots, \boldsymbol{\mu}_K, \boldsymbol{\Sigma}_1, \dots, \boldsymbol{\Sigma}_K)$$

representa o conjunto de todos os parâmetros do modelo.

As componentes $p(\mathbf{x} \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)$ são densidades Gaussianas multivariadas, definidas por:

$$p(\mathbf{x} \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k) = \frac{1}{(2\pi)^{d/2} |\boldsymbol{\Sigma}_k|^{1/2}} \exp\left(-\frac{1}{2}(\mathbf{x} - \boldsymbol{\mu}_k)^T \boldsymbol{\Sigma}_k^{-1} (\mathbf{x} - \boldsymbol{\mu}_k)\right).$$

Os pesos π_k representam as **probabilidades a priori** de cada componente, isto é, a probabilidade de que uma amostra $\mathbf{x}$ escolhida aleatoriamente do conjunto de dados tenha sido gerada pela k -ésima componente da mistura. Assim, a distribuição de mistura pode ser interpretada como um processo em duas etapas:

1. Escolhe-se aleatoriamente uma componente k com probabilidade π_k ;
2. Gera-se uma amostra $\mathbf{x}$ de uma distribuição Gaussiana $\mathcal{N}(\boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)$.

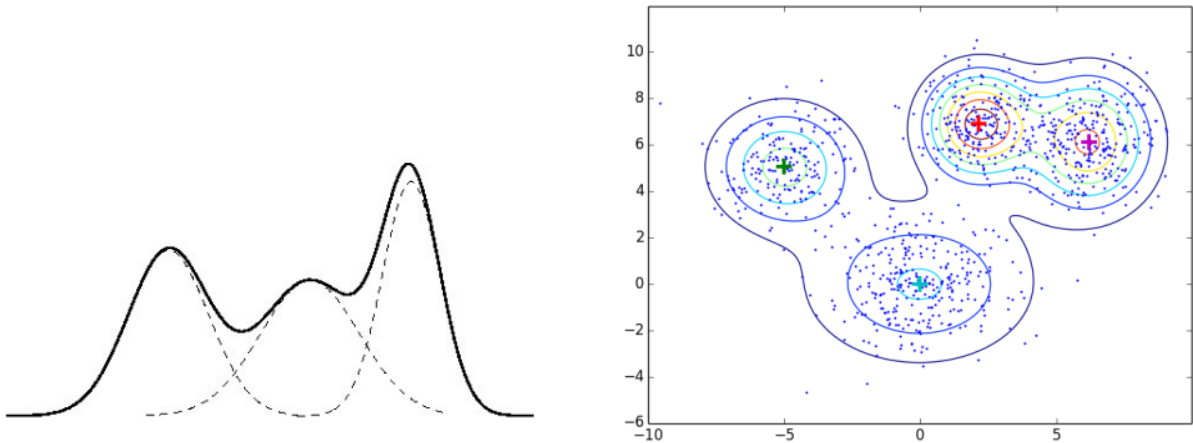


Figura 5: Exemplos ilustrativos de misturas de Gaussianas nos casos unidimensional (esquerda) e bidimensional (direita). Cada componente Gaussiana é ponderada por um coeficiente π_k .

4.2 Comparação entre k-médias e GMM

Embora o algoritmo k-médias e os Modelos de Mistura de Gaussianas (*Gaussian Mixture Models* — GMM) sejam frequentemente utilizados para a tarefa de agrupamento de dados, suas formulações matemáticas e interpretações probabilísticas são substancialmente diferentes. Ambos os métodos partem da mesma intuição — representar um conjunto de dados por meio de um conjunto de grupos caracterizados por seus centros —, porém o GMM oferece uma generalização probabilística mais rica e flexível do K-médias clássico.

No algoritmo k-médias, o único parâmetro estimado para cada cluster k é o seu **centróide** μ_k , que representa a média dos pontos pertencentes ao grupo. Cada amostra é atribuída de maneira determinística ao cluster mais próximo em termos de distância Euclidiana, o que resulta em uma partição rígida (*hard clustering*). Assim, o K-médias busca minimizar a soma das distâncias quadráticas intra-cluster:

$$J = \sum_{k=1}^K \sum_{\mathbf{x}_i \in C_k} \|\mathbf{x}_i - \mu_k\|^2.$$

Por outro lado, o modelo de mistura de Gaussianas adota uma **abordagem probabilística**. Cada cluster é modelado como uma distribuição Gaussiana multivariada $\mathcal{N}(\mu_k, \Sigma_k)$. Portanto, enquanto o k-médias considera apenas a posição dos centróides, o GMM leva em conta tanto a forma quanto a dispersão de cada grupo, oferecendo uma representação mais expressiva e adequada para dados com variâncias e orientações distintas.

Em termos conceituais, o k-médias pode ser visto como um caso particular do modelo de mistura de Gaussianas em que:

- todas as matrizes de covariância são esféricas e idênticas ($\Sigma_k = \sigma^2 \mathbf{I}$);
- todas as probabilidades a priori são iguais ($\pi_k = 1/K$);
- as atribuições de cluster são determinísticas (em vez de probabilísticas).

Consequentemente, o GMM estende o k-médias ao introduzir flexibilidade na forma dos clusters e permitir uma modelagem probabilística de pertencimento (*soft clustering*). Essa generalização torna o GMM particularmente adequado para dados com sobreposição entre grupos, diferentes escalas de variância ou correlação entre atributos, cenários nos quais o k-médias tende a falhar. Por outro lado, a estimação das matrizes de covariância dos agrupamentos pode se tornar instável se a dimensionalidade dos dados é muito elevada, prejudicando o desempenho do algoritmo GMM.

4.3 Algoritmo EM: Expectation-Maximization

O **Algoritmo Expectation-Maximization (EM)** é um procedimento iterativo utilizado para encontrar os **estimadores de máxima verossimilhança** (EMV's) dos parâmetros de modelos estatísticos em situações nas quais as equações analíticas de maximização são intratáveis.

Esses casos ocorrem tipicamente quando o modelo estatístico envolve **variáveis latentes**, além dos parâmetros desconhecidos e dos dados observados. Em outras palavras, há variáveis não observadas que influenciam o processo de geração dos dados. Tais variáveis podem representar, por exemplo, o componente da mistura que deu origem a cada amostra em um modelo de mistura Gaussiana. Assim, o EM é amplamente aplicado em modelos de mistura, onde a presença de dados faltantes ou variáveis ocultas impede a maximização direta da função de verossimilhança.

Para encontrar o estimador de máxima verossimilhança, seria necessário calcular as derivadas da função log-verossimilhança em relação a todos os parâmetros e variáveis latentes, resolvendo simultaneamente o sistema resultante. No entanto, para modelos que envolvem variáveis latentes — como o *Gaussian Mixture Model* (GMM) — esse processo é, em geral, **impossível de resolver analiticamente**. O algoritmo EM contorna essa dificuldade ao reformular o problema em duas etapas iterativas, que alternam entre a estimativa das variáveis latentes e a atualização dos parâmetros do modelo.

Considere um modelo estatístico que gera um conjunto de observações X , associado a um conjunto de variáveis latentes Z e a um vetor de parâmetros θ . A **função de verossimilhança completa** é dada por:

$$L(\theta; X, Z) = p(X, Z \mid \theta).$$

O estimador de máxima verossimilhança de θ é obtido pela maximização da **verossimilhança marginal** dos dados observados:

$$L(\theta; X) = p(X \mid \theta) = \int p(X, Z \mid \theta) dZ.$$

Entretanto, essa integral é geralmente **intratável**, pois calcular a marginalização sobre Z é computacionalmente inviável na maioria dos casos.

O algoritmo EM resolve esse problema de forma iterativa, aplicando os dois passos seguintes:

1. **Etapas E (Expectation):** Calcular o valor esperado da log-verossimilhança completa, com respeito à distribuição condicional de Z dado X , utilizando a estimativa atual dos parâmetros $\theta^{(t)}$:

$$Q(\theta \mid \theta^{(t)}) = \mathbb{E}_{Z \mid X, \theta^{(t)}} [\log L(\theta; X, Z)].$$

2. **Etapas M (Maximization):** Atualizar os parâmetros do modelo de forma a maximizar a função esperada Q :

$$\theta^{(t+1)} = \arg \max_{\theta} Q(\theta \mid \theta^{(t)}).$$

Essas duas etapas são repetidas alternadamente até que a função de verossimilhança marginal $p(X \mid \theta)$ convirja para um máximo local.

Nos modelos de mistura, como o GMM, as variáveis latentes Z indicam o grupo ou componente de mistura ao qual cada amostra $\mathbf{x}_i \in X$ pertence. O EM alterna entre:

- **Estimar** a probabilidade de cada amostra pertencer a cada componente (Etapa E);
- **Atualizar** os parâmetros das distribuições Gaussianas e os pesos da mistura (Etapa M).

Observações Importantes

1. As observações X podem ser tanto discretas quanto contínuas (por exemplo, Gaussianas multivariadas);
2. As variáveis latentes Z são discretas, assumindo um número fixo de valores, de modo que se associa uma variável latente por observação;
3. Os parâmetros θ são contínuos e podem incluir parâmetros globais (comuns a todas as amostras) e parâmetros específicos de cada valor da variável latente.

4.4 Modelo de mistura de Gaussianas com duas componentes

Seja $X = \{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n\}$ uma amostra de n observações independentes provenientes de uma mistura de duas distribuições Gaussianas multivariadas de dimensão d , e seja $Z = \{z_1, z_2, \dots, z_n\}$ o conjunto de variáveis latentes que determinam de qual componente cada observação se origina. Assim:

$$\mathbf{X}_i \mid (Z_i = 1) \sim \mathcal{N}(\boldsymbol{\mu}_1, \boldsymbol{\Sigma}_1), \quad \mathbf{X}_i \mid (Z_i = 2) \sim \mathcal{N}(\boldsymbol{\mu}_2, \boldsymbol{\Sigma}_2),$$

com

$$P(Z_i = 1) = \pi_1, \quad P(Z_i = 2) = \pi_2 = 1 - \pi_1.$$

A densidade condicional de $\mathbf{x}_i$ dado $Z_i = j$ é:

$$f(\mathbf{x}_i \mid Z_i = j, \boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j) = \frac{1}{(2\pi)^{d/2} |\boldsymbol{\Sigma}_j|^{1/2}} \exp \left[-\frac{1}{2} (\mathbf{x}_i - \boldsymbol{\mu}_j)^T \boldsymbol{\Sigma}_j^{-1} (\mathbf{x}_i - \boldsymbol{\mu}_j) \right]. \quad (*)$$

O vetor de parâmetros desconhecidos é dado por:

$$\boldsymbol{\theta} = (\pi_1, \pi_2, \boldsymbol{\mu}_1, \boldsymbol{\mu}_2, \boldsymbol{\Sigma}_1, \boldsymbol{\Sigma}_2).$$

A **função de verossimilhança incompleta** é expressa como:

$$L(\boldsymbol{\theta}; X) = \prod_{i=1}^n \sum_{j=1}^2 \pi_j f(\mathbf{x}_i \mid Z_i = j, \boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j).$$

Já a **função de verossimilhança completa** pode ser escrita como:

$$L(\boldsymbol{\theta}; X, Z) = p(X, Z \mid \boldsymbol{\theta}) = \prod_{i=1}^n \prod_{j=1}^2 [\pi_j f(\mathbf{x}_i \mid Z_i = j, \boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j)]^{I(Z_i=j)},$$

onde $I(Z_i = j)$ é uma variável indicadora que vale 1 se $Z_i = j$ e 0 caso contrário.

Aplicando logaritmo e exponencial, obtém-se:

$$L(\boldsymbol{\theta}; X, Z) = \exp \left\{ \sum_{i=1}^n \sum_{j=1}^2 I(Z_i = j) [\log \pi_j + \log f(\mathbf{x}_i | Z_i = j, \boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j)] \right\}.$$

Substituindo $f(\cdot)$ pela expressão em (*):

$$L(\boldsymbol{\theta}; X, Z) = \exp \left\{ \sum_{i=1}^n \sum_{j=1}^2 I(Z_i = j) \left[\log \pi_j - \frac{1}{2} \log |\boldsymbol{\Sigma}_j| - \frac{1}{2} (\mathbf{x}_i - \boldsymbol{\mu}_j)^T \boldsymbol{\Sigma}_j^{-1} (\mathbf{x}_i - \boldsymbol{\mu}_j) - \frac{d}{2} \log 2\pi \right] \right\}.$$

Passo E (Expectation)

Dada a estimativa atual dos parâmetros $\boldsymbol{\theta}^{(t)}$, a distribuição condicional de Z_i dado X é determinada pelo teorema de Bayes:

$$T_{j,i}^{(t)} = P(Z_i = j | \mathbf{x}_i, \boldsymbol{\theta}^{(t)}) = \frac{f(\mathbf{x}_i | Z_i = j, \boldsymbol{\mu}_j^{(t)}, \boldsymbol{\Sigma}_j^{(t)}) \pi_j^{(t)}}{\sum_{l=1}^2 f(\mathbf{x}_i | Z_i = l, \boldsymbol{\mu}_l^{(t)}, \boldsymbol{\Sigma}_l^{(t)}) \pi_l^{(t)}}.$$

Essas quantidades $T_{j,i}^{(t)}$ correspondem às **probabilidades de pertencimento** (ou *membership probabilities*), e representam os “scores” de cada amostra em relação a cada componente da mistura.

A função $Q(\boldsymbol{\theta} | \boldsymbol{\theta}^{(t)})$ a ser maximizada é:

$$\begin{aligned} Q(\boldsymbol{\theta} | \boldsymbol{\theta}^{(t)}) &= \mathbb{E}_{Z|X, \boldsymbol{\theta}^{(t)}} [\log L(\boldsymbol{\theta}; X, Z)] = \\ &= \sum_{i=1}^n \sum_{j=1}^2 T_{j,i}^{(t)} \left[\log \pi_j - \frac{1}{2} \log |\boldsymbol{\Sigma}_j| - \frac{1}{2} (\mathbf{x}_i - \boldsymbol{\mu}_j)^T \boldsymbol{\Sigma}_j^{-1} (\mathbf{x}_i - \boldsymbol{\mu}_j) - \frac{d}{2} \log 2\pi \right]. \end{aligned} \quad (17)$$

Passo M (Maximization)

Nesta etapa, atualizamos os parâmetros de modo a maximizar $Q(\boldsymbol{\theta} | \boldsymbol{\theta}^{(t)})$.

Como $\pi_2 = 1 - \pi_1$, podemos simplificar:

$$Q(\boldsymbol{\theta} | \boldsymbol{\theta}^{(t)}) = \sum_{i=1}^n \left[T_{1,i}^{(t)} \log \pi_1 + T_{2,i}^{(t)} \log(1 - \pi_1) \right].$$

Derivando em relação a π_1 e igualando a zero:

$$\frac{\partial Q}{\partial \pi_1} = \sum_{i=1}^n \left[\frac{T_{1,i}^{(t)}}{\pi_1} - \frac{T_{2,i}^{(t)}}{1 - \pi_1} \right] = 0.$$

Daí obtemos:

$$\pi_1^{(t+1)} = \frac{1}{n} \sum_{i=1}^n T_{1,i}^{(t)}, \quad \pi_2^{(t+1)} = 1 - \pi_1^{(t+1)}.$$

Maximizando Q em relação a $\boldsymbol{\mu}_j$:

$$\frac{\partial Q}{\partial \boldsymbol{\mu}_j} = \sum_{i=1}^n T_{j,i}^{(t)} \boldsymbol{\Sigma}_j^{-1} (\mathbf{x}_i - \boldsymbol{\mu}_j) = 0,$$

o que leva a:

$$\boldsymbol{\mu}_j^{(t+1)} = \frac{\sum_{i=1}^n T_{j,i}^{(t)} \mathbf{x}_i}{\sum_{i=1}^n T_{j,i}^{(t)}}.$$

De modo análogo, tem-se:

$$\boldsymbol{\Sigma}_j^{(t+1)} = \frac{\sum_{i=1}^n T_{j,i}^{(t)} (\mathbf{x}_i - \boldsymbol{\mu}_j^{(t+1)}) (\mathbf{x}_i - \boldsymbol{\mu}_j^{(t+1)})^T}{\sum_{i=1}^n T_{j,i}^{(t)}}.$$

Para o cálculo da matriz de covariâncias $\boldsymbol{\Sigma}_j$, é necessário relembrar algumas propriedades fundamentais da diferenciação matricial, em especial aquelas relacionadas à derivada de formas quadráticas. A primeira observação importante é que o operador **traço** ($\text{tr}[\cdot]$) é invariante sob permutações cíclicas de produtos de matrizes, ou seja:

$$\text{tr}[ABC] = \text{tr}[BCA] = \text{tr}[CAB],$$

onde A , B e C são matrizes arbitrárias de dimensões compatíveis.

Esse resultado é extremamente útil para o cálculo de derivadas de formas quadráticas. Em particular, deseja-se obter a derivada da expressão $\mathbf{x}^T A \mathbf{x}$ em relação à matriz A . O segredo consiste em reescrever essa expressão utilizando a propriedade do traço:

$$\mathbf{x}^T A \mathbf{x} = \text{tr}[\mathbf{x}^T A \mathbf{x}] = \text{tr}[\mathbf{x} \mathbf{x}^T A],$$

onde a primeira igualdade decorre do fato de que o traço de um escalar é igual ao próprio escalar.

Do cálculo matricial multivariado, sabemos que:

$$\frac{\partial \text{tr}[BA]}{\partial A} = B^T,$$

de modo que podemos aplicar esse resultado à forma quadrática:

$$\frac{\partial (\mathbf{x}^T A \mathbf{x})}{\partial A} = \frac{\partial \text{tr}[\mathbf{x} \mathbf{x}^T A]}{\partial A} = (\mathbf{x} \mathbf{x}^T)^T = \mathbf{x} \mathbf{x}^T.$$

Outro resultado útil do cálculo matricial é a derivada do logaritmo do determinante de uma matriz

A:

$$\frac{\partial \log |A|}{\partial A} = (A^{-1})^T.$$

De posse dessas propriedades, podemos calcular o estimador de máxima verossimilhança para a matriz de covariância Σ_j a partir da expressão da log-verossimilhança. Reescrevendo a função Q obtida na etapa anterior, temos:

$$Q(\boldsymbol{\theta} \mid \boldsymbol{\theta}^{(t)}) = \frac{1}{2} \sum_{i=1}^n T_{j,i}^{(t)} [\log |\Sigma_j^{-1}| - (\mathbf{x}_i - \boldsymbol{\mu}_j)^T \Sigma_j^{-1} (\mathbf{x}_i - \boldsymbol{\mu}_j)],$$

onde utilizamos o fato de que $|\Sigma_j^{-1}| = |\Sigma_j|^{-1}$.

Derivando Q em relação à matriz Σ_j^{-1} , temos:

$$\frac{\partial Q(\boldsymbol{\theta} \mid \boldsymbol{\theta}^{(t)})}{\partial \Sigma_j^{-1}} = \frac{1}{2} \sum_{i=1}^n T_{j,i}^{(t)} [\Sigma_j - (\mathbf{x}_i - \boldsymbol{\mu}_j)(\mathbf{x}_i - \boldsymbol{\mu}_j)^T] = 0.$$

Sabendo que a matriz de covariância é simétrica ($\Sigma_j^T = \Sigma_j$), podemos isolar Σ_j , obtendo:

$$\Sigma_j = \frac{\sum_{i=1}^n T_{j,i}^{(t)} (\mathbf{x}_i - \boldsymbol{\mu}_j)(\mathbf{x}_i - \boldsymbol{\mu}_j)^T}{\sum_{i=1}^n T_{j,i}^{(t)}}.$$

Essa é precisamente a forma do **estimador de máxima verossimilhança** da matriz de covariância da j -ésima componente de uma mistura Gaussiana, ponderada pelas probabilidades de pertencimento obtidas na Etapa E do algoritmo EM.

4.4.1 Algoritmo EM+GMM para agrupamento de dados

O algoritmo *Expectation-Maximization* aplicado aos *Modelos de Mistura de Gaussianas* (**EM+GMM**) pode ser resumido nos seguintes passos fundamentais:

1. Inicialização dos parâmetros:

$$\{\boldsymbol{\mu}_k, \Sigma_k, \pi_k\}_{k=1}^K.$$

2. Iteração dos passos do EM até a convergência:

- **E-step (Expectation):** Associar a cada amostra $\mathbf{x}_n$ um *score* $\gamma(z_{nk})$, representando a probabilidade de pertencimento da amostra ao cluster k :

$$T_{j,i}^{(t)} = P(Z_i = j \mid \mathbf{x}_i, \boldsymbol{\theta}^{(t)}).$$

- **M-step (Maximization):** Dados os *scores* $T_{j,i}^{(t)}$, atualizar os parâmetros $\boldsymbol{\mu}_k$, Σ_k e π_k para cada cluster k .

3. **Avaliação da log-verossimilhança:** Calcular a função log-verossimilhança a cada iteração. Se o valor estabilizar (não houver melhora significativa), o algoritmo é considerado convergente.

Sobre a inicialização dos parâmetros

Uma forma prática de inicializar os parâmetros consiste em utilizar o algoritmo k-médias:

$$\boldsymbol{\mu}_k^{(0)} = \text{centróide do cluster } k, \quad \boldsymbol{\Sigma}_k^{(0)} = \text{cov}(\text{cluster } k) \quad \text{ou simplesmente} \quad \boldsymbol{\Sigma}_k^{(0)} = I,$$

$$\pi_k^{(0)} = \frac{N_k}{N},$$

onde N_k é o número de amostras atribuídas ao cluster k e N é o número total de amostras.

E-step: Calcular os **scores de pertencimento** (probabilidades a posteriori):

$$T_{j,i}^{(t)} = \frac{\pi_j^{(t)} p(\mathbf{x}_i | \boldsymbol{\mu}_j^{(t)}, \boldsymbol{\Sigma}_j^{(t)})}{\sum_{k=1}^K \pi_k^{(t)} p(\mathbf{x}_i | \boldsymbol{\mu}_k^{(t)}, \boldsymbol{\Sigma}_k^{(t)})}.$$

Note que esses valores formam uma matriz $K \times N$, em que cada linha corresponde a um cluster e cada coluna a uma amostra.

M-step: Atualizar os parâmetros utilizando os *scores* $T_{j,i}^{(t)}$:

$$N_j = \sum_{i=1}^N T_{j,i}^{(t)},$$

$$\pi_j^{(t+1)} = \frac{N_j}{N},$$

$$\boldsymbol{\mu}_j^{(t+1)} = \frac{\sum_{i=1}^N T_{j,i}^{(t)} \mathbf{x}_i}{\sum_{i=1}^N T_{j,i}^{(t)}},$$

$$\boldsymbol{\Sigma}_j^{(t+1)} = \frac{\sum_{i=1}^N T_{j,i}^{(t)} (\mathbf{x}_i - \boldsymbol{\mu}_j^{(t+1)}) (\mathbf{x}_i - \boldsymbol{\mu}_j^{(t+1)})^T}{\sum_{i=1}^N T_{j,i}^{(t)}}.$$

Critério de parada

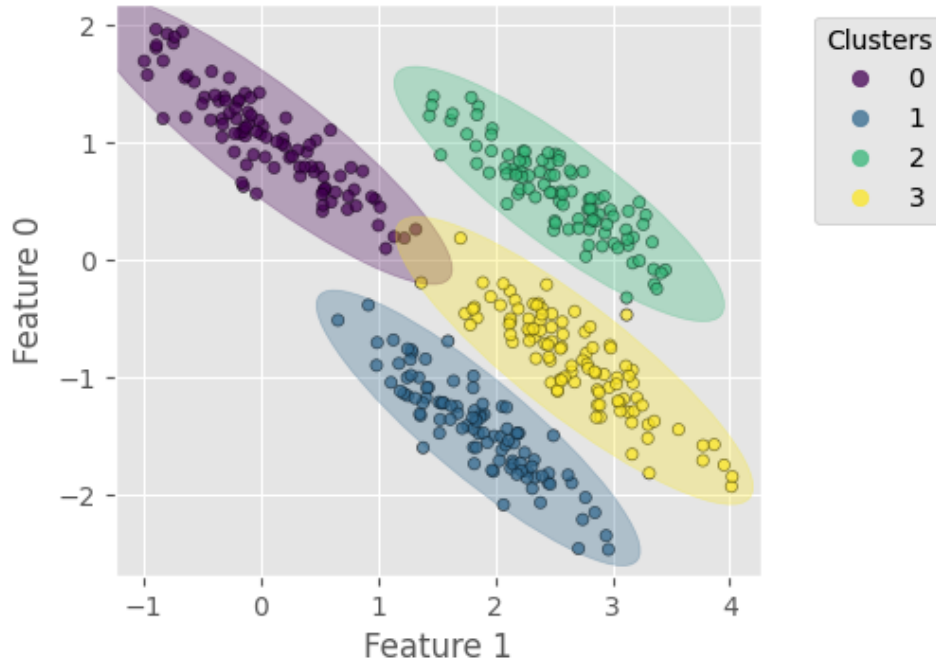
A log-verossimilhança mede o grau de confiança de que o conjunto de dados $X = \{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_N\}$ foi gerado pelos parâmetros estimados:

$$\log p(X | \boldsymbol{\theta}) = \sum_{n=1}^N \log \left[\sum_{k=1}^K \pi_k p(\mathbf{x}_n | \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k) \right].$$

Essa quantidade deve ser computada ao final de cada iteração. Quando o valor da log-verossimilhança

se estabiliza, considera-se que o algoritmo atingiu um ponto de máxima verossimilhança e, portanto, convergiu.

Uma vantagem do GMM em relação ao k-médias é justamente a sua capacidade de detectar agrupamentos não circulares, como indica a figura a seguir.



4.5 Seleção do número agrupamentos: critérios AIC e BIC

Um dos desafios fundamentais no uso de *Modelos de Mistura de Gaussianas* (GMM) é a escolha do número adequado de componentes K . Enquanto o algoritmo EM é capaz de estimar eficientemente os parâmetros do modelo para um K fixo, ele não fornece informações diretas sobre o número ótimo de componentes que melhor descreve os dados. A seleção incorreta de K pode levar tanto a modelos *subajustados* (com poucos clusters, que não capturam a estrutura dos dados) quanto a modelos *superajustados* (com muitos clusters, que se ajustam excessivamente ao ruído).

Para resolver esse problema, dois critérios estatísticos amplamente utilizados são o **Critério de Informação de Akaike** (AIC) e o **Critério de Informação Bayesiano** (BIC). Ambos se baseiam na maximização da verossimilhança penalizada pela complexidade do modelo, buscando um equilíbrio entre ajuste e simplicidade.

4.5.1 Critério de Informação de Akaike (AIC)

O *Akaike Information Criterion* (AIC) foi proposto por Hirotugu Akaike em 1974 e mede a qualidade relativa de um modelo estatístico, penalizando o excesso de parâmetros. Sua formulação é dada por:

$$\text{AIC} = 2p - 2 \log \hat{L},$$

onde:

- p é o número total de parâmetros estimados do modelo (no caso do GMM, inclui médias, covariâncias e pesos de mistura);
- $\hat{L} = p(X | \hat{\theta})$ é o valor máximo da função de verossimilhança para os parâmetros estimados.

O AIC tende a favorecer modelos que se ajustam bem aos dados, porém pode penalizar menos severamente modelos complexos, sendo, portanto, mais propenso a escolher modelos ligeiramente superparametrizados.

4.5.2 Critério de Informação Bayesiano (BIC)

O *Bayesian Information Criterion* (BIC), proposto por Gideon Schwarz em 1978, adota uma penalização mais forte em relação ao número de parâmetros, baseando-se em princípios bayesianos. Sua formulação é expressa como:

$$\text{BIC} = p \log(n) - 2 \log \hat{L},$$

onde:

- n é o número de observações (amostras);
- p é novamente o número de parâmetros estimados;
- $\hat{L}$ é o valor máximo da função de verossimilhança.

Por penalizar de forma mais intensa a complexidade do modelo (via termo $\log(n)$), o BIC tende a favorecer modelos mais simples e é considerado mais conservador que o AIC, especialmente para grandes conjuntos de dados.

Aplicação ao Modelo GMM

No contexto de modelos de mistura de Gaussianas, ambos os critérios são aplicados da seguinte forma:

1. Ajusta-se um modelo GMM para cada valor candidato de K ;
2. Calcula-se o valor da log-verossimilhança $\log \hat{L}$ para o modelo ajustado;
3. Computam-se os valores de AIC e BIC utilizando as fórmulas apresentadas;
4. Escolhe-se o valor de K que minimiza o critério (AIC ou BIC).

No caso de um GMM com K componentes e dados em $\mathbb{R}^d$, o número de parâmetros p é dado por:

$$p = (K - 1) + Kd + \frac{Kd(d + 1)}{2},$$

correspondendo, respectivamente, às proporções de mistura, às médias e às matrizes de covariância de cada componente.

De modo geral:

- O **AIC** é mais adequado quando o objetivo é *previsão* e quando há interesse em capturar estruturas sutis nos dados, mesmo que isso implique em um modelo mais complexo.
- O **BIC**, por outro lado, é mais indicado quando o objetivo é *inferência* e quando se busca identificar o número real de componentes subjacentes, pois penaliza fortemente a complexidade.

A prática comum consiste em comparar os valores de AIC e BIC para diferentes valores de K , e selecionar aquele que proporciona o menor valor do critério. Em muitos casos, ambos os critérios apontam para valores semelhantes de K , mas quando divergem, o BIC tende a sugerir um número menor de componentes.

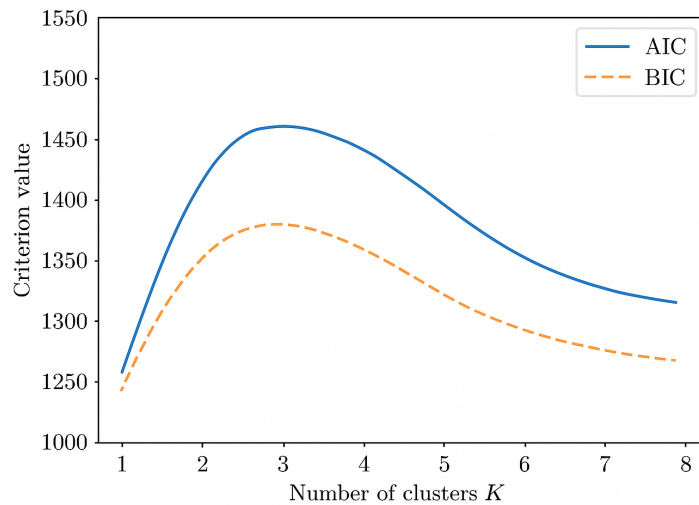


Figura 6: Exemplo ilustrativo dos valores de AIC e BIC em função do número de componentes K . O ponto de mínimo indica o número ótimo de clusters segundo cada critério.

Em síntese, os critérios AIC e BIC constituem ferramentas fundamentais para a seleção do número ótimo de agrupamentos em modelos GMM, permitindo um compromisso adequado entre qualidade de ajuste e simplicidade do modelo.

4.6 Considerações finais

Os *Modelos de Mistura de Gaussianas* (*Gaussian Mixture Models* – GMM) representam uma das abordagens mais poderosas e flexíveis para o problema de agrupamento de dados. Diferentemente de

métodos puramente geométricos, como o K -médias, os GMMs modelam explicitamente a distribuição estatística dos dados, permitindo capturar estruturas mais complexas, como clusters elípticos, de diferentes tamanhos e orientações.

A principal característica do GMM é sua natureza probabilística: cada amostra é associada a uma probabilidade de pertencer a cada componente da mistura, o que possibilita uma interpretação mais rica e contínua do agrupamento — em contraste com a atribuição rígida de métodos determinísticos. Essa flexibilidade é particularmente vantajosa em cenários onde os limites entre os grupos são difusos ou onde se deseja modelar incertezas inerentes às observações.

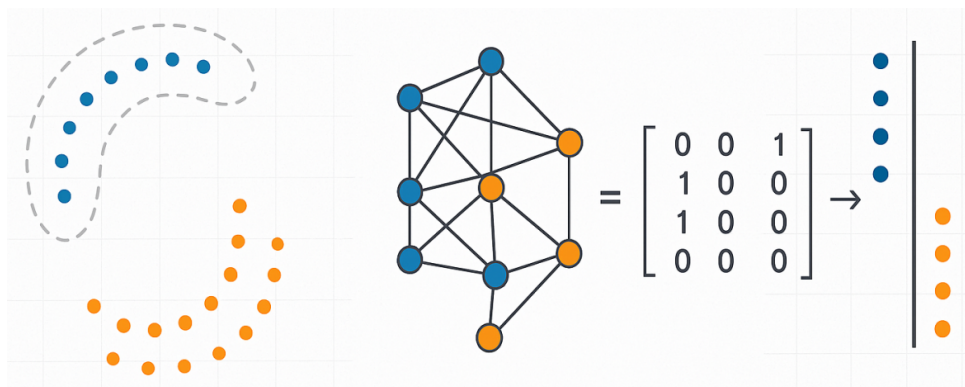
O algoritmo de *Expectation-Maximization* (EM) é o núcleo do processo de estimação dos parâmetros do modelo. Ele fornece uma maneira eficiente de maximizar a verossimilhança mesmo na presença de variáveis latentes, alternando entre etapas de estimação das probabilidades de pertinência (*E-step*) e atualização dos parâmetros (*M-step*). Apesar de seu poder, o EM é sensível à inicialização dos parâmetros, podendo convergir para máximos locais, o que motiva o uso de técnicas como o k -means++ ou múltiplas inicializações para garantir maior robustez.

Outro aspecto importante na aplicação de GMMs é a escolha adequada do número de componentes K . Como discutido, critérios de informação como o *Akaike Information Criterion* (AIC) e o *Bayesian Information Criterion* (BIC) fornecem uma forma sistemática de balancear a qualidade do ajuste com a complexidade do modelo, evitando o sobreajuste.

Em síntese, os GMMs oferecem uma estrutura estatística elegante e versátil para o agrupamento de dados, unindo a intuição geométrica dos métodos baseados em distâncias com o rigor probabilístico dos modelos de densidade. Sua aplicação é ampla, abrangendo desde reconhecimento de padrões e visão computacional até biologia computacional e mineração de dados. Compreender o funcionamento e as limitações dos GMMs é, portanto, essencial para qualquer profissional ou pesquisador que deseje empregar técnicas modernas e robustas de aprendizado não supervisionado.

5 Agrupamento espectral (Spectral clustering)

O *Agrupamento Espectral (Spectral Clustering)* é uma técnica de agrupamento de dados baseada em grafos que utiliza propriedades espectrais (autovalores e autovetores) de matrizes associadas a similaridades entre amostras. Diferentemente de métodos tradicionais como o k-médias, que operam diretamente no espaço dos dados originais e assumem clusters aproximadamente esféricos, o agrupamento espectral permite detectar estruturas não lineares e agrupamentos de formas arbitrárias, aproveitando a conectividade dos dados em um espaço de similaridades.



A motivação central do método está no fato de que muitas estruturas de agrupamento são melhor representadas não pela distância Euclidiana direta, mas pela relação de vizinhança entre as amostras. Para capturar essas relações, o conjunto de dados é modelado como um **grafo ponderado não direcionado**, no qual:

- Cada vértice representa uma amostra do conjunto de dados;
- As arestas conectam pares de vértices com pesos w_{ij} que indicam o grau de similaridade entre $\vec{x}_i$ e $\vec{x}_j$.

O objetivo do agrupamento espectral é, portanto, particionar esse grafo em subconjuntos de vértices (clusters) de modo que as conexões dentro de cada grupo sejam fortes, e as conexões entre grupos sejam fracas. Para isso, o método utiliza a **matriz Laplaciana do grafo**, que captura a estrutura de conectividade global e local entre as amostras. A análise espectral (isto é, a decomposição em autovalores e autovetores) do Laplaciano fornece uma representação embutida dos dados em um espaço de menor dimensionalidade, no qual as relações de vizinhança são preservadas de forma mais apropriada. A matriz Laplaciana de um grafo possui propriedades notáveis — em especial, seus autovetores associados aos menores autovalores codificam as partições naturais do grafo.

O procedimento típico do *Spectral Clustering* envolve três etapas principais:

1. Construir a matriz de similaridade W a partir dos dados, utilizando uma métrica apropriada (por exemplo, a distância Euclidiana ou um kernel Gaussiano);
2. Calcular os autovetores associados aos menores autovalores do Laplaciano (ou Laplaciano normalizado);

3. Aplicar o algoritmo K -médias sobre as representações obtidas no espaço dos autovetores.

Esse processo transforma o problema de agrupamento original — potencialmente não linear e de difícil separação — em um problema linear no espaço espectral, onde as partições podem ser identificadas com maior facilidade.

O agrupamento espectral é amplamente utilizado em aplicações que envolvem dados complexos e de alta dimensionalidade, como:

- segmentação de imagens e vídeos;
- análise de redes e comunidades;
- agrupamento de séries temporais e dados de forma livre;
- aprendizado de representações e redução de dimensionalidade.

Sua principal vantagem reside na capacidade de detectar agrupamentos não convexos e de capturar relações topológicas globais entre os dados, tornando-o um método fundamental em modernas abordagens de aprendizado não supervisionado.

5.1 A Matriz Laplaciana

Definição 1 (Matriz de Adjacências). A matriz de adjacências de um grafo ponderado $G = (V, E, w)$ é uma matriz simétrica $W = [w_{ij}]$ tal que:

$$w_{ij} = \begin{cases} w(i, j), & \text{se } (i, j) \in E, \\ 0, & \text{caso contrário.} \end{cases}$$

Definição 2 (Grau de um Vértice). O grau de um vértice v_i , denotado por $d(v_i)$, é a soma dos pesos das arestas incidentes a ele, ou seja:

$$d(v_i) = \sum_{j=1}^n w_{ij}.$$

Definição 3 (Matriz Laplaciana). A matriz Laplaciana de um grafo $G = (V, E)$ é definida como:

$$L = D - A,$$

onde:

$$D = \text{diag}(d(v_1), d(v_2), \dots, d(v_n))$$

é a matriz diagonal dos graus e $A = W$ é a matriz de adjacências.

Toda a informação estrutural contida em A também está em L , mas o espectro de L (seus autovalores e autovetores) é mais bem comportado, o que torna a matriz Laplaciana preferível para análise espectral.

Propriedades da Matriz Laplaciana

Teorema. A matriz Laplaciana L de um grafo ponderado satisfaz as seguintes propriedades:

1. Para todo vetor $\vec{f} \in \mathbb{R}^n$:

$$\vec{f}^T L \vec{f} = \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n w_{ij} (f_i - f_j)^2.$$

2. L é simétrica e positiva semidefinida.

3. O menor autovalor de L é zero e está associado ao autovetor constante $\vec{1}$.

4. L possui n autovalores reais e não negativos:

$$0 = \lambda_1 \leq \lambda_2 \leq \dots \leq \lambda_n.$$

Pela definição de $L = D - W$, temos:

$$\vec{f}^T L \vec{f} = \vec{f}^T D \vec{f} - \vec{f}^T W \vec{f} = \sum_i f_i^2 d_{ii} - \sum_i \sum_j f_i w_{ij} f_j.$$

Multiplicando e dividindo por dois:

$$\vec{f}^T L \vec{f} = \frac{1}{2} \left(\sum_i f_i^2 d_{ii} - 2 \sum_i \sum_j f_i w_{ij} f_j + \sum_j f_j^2 d_{jj} \right).$$

Como $d_{ii} = \sum_j w_{ij}$, podemos reescrever:

$$\vec{f}^T L \vec{f} = \frac{1}{2} \sum_i \sum_j w_{ij} f_i^2 - \sum_i \sum_j f_i w_{ij} f_j + \frac{1}{2} \sum_i \sum_j w_{ij} f_j^2.$$

Agrupando os termos:

$$\vec{f}^T L \vec{f} = \frac{1}{2} \sum_i \sum_j w_{ij} (f_i^2 - 2f_i f_j + f_j^2) = \frac{1}{2} \sum_i \sum_j w_{ij} (f_i - f_j)^2.$$

A demonstração da segunda propriedade é realizada a seguir.

(a) *Simetria:* Como as matrizes D e W são simétricas, sua diferença $L = D - W$ também é simétrica.

(b) *Positiva semidefinida:* Como $w_{ij} (f_i - f_j)^2 \geq 0$ para todo i, j , segue diretamente que $\vec{f}^T L \vec{f} \geq 0$, o que implica que L é positiva semidefinida.

Para a terceira propriedade, seja $\vec{1} = [1, 1, \dots, 1]^T$. Então:

$$L\vec{1} = (D - W)\vec{1} = D\vec{1} - W\vec{1}.$$

Mas,

$$(D\vec{1})_i = d_i = \sum_j w_{ij} \quad \text{e} \quad (W\vec{1})_i = \sum_j w_{ij}.$$

Logo:

$$L\vec{1} = 0,$$

o que mostra que $\lambda_1 = 0$ e o autovetor associado é $\vec{1}$.

A quarta propriedade decorre diretamente das anteriores: como L é uma matriz simétrica e positiva semidefinida, todos os seus autovalores são reais e não negativos, e o menor deles é zero. Essas propriedades fazem do Laplaciano uma ferramenta fundamental para o *Agrupamento Espectral*, pois sua decomposição em autovetores fornece informações valiosas sobre a estrutura de conectividade e as partições naturais do grafo.

5.2 Laplacianas normalizadas

Em alguns casos, é comum utilizar versões **normalizadas** da matriz Laplaciana. A principal motivação para isso é obter propriedades espectrais mais estáveis, como a existência de um limite superior conhecido para o maior autovalor. Pode-se demonstrar que, nas matrizes Laplacianas normalizadas, os autovalores satisfazem:

$$0 \leq \lambda_i \leq 2.$$

Existem duas formas principais de normalização utilizadas na literatura:

Laplaciana normalizada não simétrica

A primeira forma, chamada de *Laplaciana normalizada não simétrica*, é definida como:

$$L_1 = D^{-1}L,$$

onde D é a matriz diagonal dos graus. Essa forma é particularmente útil em algoritmos que utilizam operações baseadas em transições estocásticas ou normalizações por linha, pois $D^{-1}W$ pode ser interpretada como uma matriz de probabilidades de transição em um grafo direcionado.

Laplaciana normalizada simétrica

A segunda forma, conhecida como *Laplaciana normalizada simétrica*, é definida como:

$$L_2 = D^{-1/2}LD^{-1/2}.$$

Essa versão é simétrica e positiva semidefinida, o que garante propriedades espectrais mais bem comportadas e permite a aplicação direta de técnicas de decomposição espectral ortogonal. Além disso, ela preserva a relação de similaridade entre os autovetores de L e de L_2 , sendo amplamente utilizada

em algoritmos de *Spectral Clustering* modernos.

Em resumo, ambas as versões normalizadas têm como objetivo ajustar a influência dos vértices com diferentes graus de conectividade, evitando que vértices com muitos vizinhos dominem a estrutura espectral do grafo.

5.3 Imersão Laplaciana 1-D (na reta)

Objetivo: encontrar um mapeamento suave $\vec{f}$ dos vértices de G para a reta real de modo a preservar proximidades — ou seja, o que é próximo no grafo deve permanecer próximo na reta. Deseja-se minimizar o seguinte critério:

$$J(\vec{f}) = \sum_{i=1}^n \sum_{j=1}^n w_{ij} (f_i - f_j)^2$$

pois se v_i e v_j são vizinhos, espera-se que $f_i \approx f_j$, isto é, que suas coordenadas na reta sejam próximas.

O problema consiste em determinar $\vec{f} \in \mathbb{R}^n$. Sabemos que o critério $J(\vec{f})$ pode ser expresso na forma matricial:

$$J(\vec{f}) = \vec{f}^T L \vec{f}$$

de modo que o problema de otimização torna-se:

$$\arg \min_{\vec{f} \in \mathbb{R}^n} \vec{f}^T L \vec{f}$$

Porém, para termos uma solução válida, é preciso considerar algumas restrições. A primeira delas é garantir que a norma da solução $\vec{f} \in \mathbb{R}^n$ seja unitária, ou seja,

$$\vec{f}^T D \vec{f} = 1$$

O que importa é a direção do vetor $\vec{f}$ e não o seu comprimento.

A segunda restrição visa eliminar a solução trivial, que é aquele vetor $\vec{f} \in \mathbb{R}^n$ que mapeia todos os vértices do grafo para o mesmo ponto na reta, fazendo com que $\vec{f}$ seja constante, ou seja, proporcional ao vetor $\vec{1} = [1, 1, \dots, 1]^T$. Sendo assim, o problema fica:

$$\begin{aligned} & \arg \min_{\vec{f} \in \mathbb{R}^n} \vec{f}^T L \vec{f} \\ & \text{sujeito a } \vec{f}^T D \vec{f} = 1 \quad \text{e} \quad \vec{f}^T D \vec{1} = 0 \end{aligned}$$

Utilizando multiplicadores de Lagrange, podemos escrever a função Lagrangiana a ser minimizada como:

$$J(\vec{f}, \lambda, \mu) = \vec{f}^T L \vec{f} - \lambda(\vec{f}^T D \vec{f} - 1)$$

Para encontrar o mínimo dessa função, derivamos e igualamos a zero:

$$\frac{\partial J(\vec{f}, \lambda)}{\partial \vec{f}} = L\vec{f} - \lambda D\vec{f} = 0$$

Assim, obtemos uma *equação de autovalor e autovetor*:

$$L\vec{f} = \lambda D\vec{f}$$

ou, equivalentemente,

$$D^{-1}L\vec{f} = \lambda\vec{f}$$

A solução corresponde ao autovetor associado ao menor autovalor não nulo da matriz Laplaciana normalizada (de modo a evitar o mapeamento constante):

$$\vec{f} = \vec{v}_1$$

O menor valor obtido para $J(\vec{f})$ é conhecido como a **conectividade algébrica** do grafo G . Por essa razão, quanto menor for λ_1 , melhor o mapeamento obtido.

Algoritmo 7 Imersão Laplaciana na Reta

Require: Conjunto de amostras $\{\vec{y}_i \in \mathbb{R}^d\}_{i=1}^n$

Ensure: Vetor $\vec{f}$ com as coordenadas de imersão 1-D na reta real

1: A partir do conjunto de amostras, construir um grafo ponderado $G = (V, E, W)$, onde:

- Cada vértice $v_i \in V$ representa uma amostra $\vec{y}_i$;
- O peso w_{ij} entre vértices v_i e v_j é definido por uma medida de similaridade, por exemplo:

$$w_{ij} = \exp\left(-\frac{\|\vec{y}_i - \vec{y}_j\|^2}{2\sigma^2}\right)$$

para v_i e v_j vizinhos no grafo.

A matriz de adjacência resultante é $A = [w_{ij}]$.

2: Calcular o grau de cada vértice:

$$d_i = \sum_{j=1}^n w_{ij}$$

e formar a matriz diagonal dos graus $D = \text{diag}(d_1, d_2, \dots, d_n)$. Em seguida, obter a matriz Laplaciana não normalizada:

$$L = D - A.$$

3: Normalizar a Laplaciana, se desejado, utilizando:

$$L_{\text{norm}} = D^{-1}L \quad (\text{forma não simétrica})$$

ou

$$L_{\text{sym}} = D^{-1/2}LD^{-1/2} \quad (\text{forma simétrica}).$$

4: Efetuar a decomposição espectral da Laplaciana escolhida:

$$L\vec{v}_i = \lambda_i\vec{v}_i, \quad i = 1, \dots, n.$$

5: Ordenar os autovalores $\lambda_1 \leq \lambda_2 \leq \dots \leq \lambda_n$ e selecionar o autovetor associado ao **menor autovalor não nulo**:

$$\vec{f} = \vec{v}_{n-1}.$$

Este vetor $\vec{f}$ fornece a imersão 1-D dos vértices de G na reta real, preservando a estrutura de vizinhança do grafo.

6: Retornar o vetor de imersão $\vec{f}$.

5.4 Imersão Laplaciana Multidimensional

No problema generalizado de imersão Laplaciana, o objetivo é mapear cada vetor de entrada no espaço original $\mathbb{R}^m$ para um vetor de saída $\vec{y} \in \mathbb{R}^d$, com $d < m$. Dessa forma, em vez de projetar os dados sobre uma única coordenada (como no caso unidimensional), devemos estimar d coordenadas no espaço de saída, representando a estrutura intrínseca dos dados em uma dimensão reduzida.

Denotamos as coordenadas dos pontos no espaço de saída por uma matriz $Y = [\vec{y}_1, \vec{y}_2, \dots, \vec{y}_d]$

em que cada linha $\vec{\phi}_i$ de Y representa as coordenadas da i -ésima amostra no espaço embutido (*embedding*):

$$Y = \begin{bmatrix} | & | & \dots & | \\ \vec{v}_1 & \vec{v}_2 & \dots & \vec{v}_d \\ | & | & \dots & | \end{bmatrix}_{n \times d}$$

Sendo assim, podemos generalizar o critério $J(\vec{f})$ definido anteriormente para a forma matricial:

$$J(Y) = \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n w_{ij} \|\vec{\phi}_i - \vec{\phi}_j\|^2.$$

Minimizar $J(Y)$ força os vértices próximos no grafo a permanecerem próximos no espaço d -dimensional. Expandindo o critério:

$$J(Y) = \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n w_{ij} (\vec{\phi}_i - \vec{\phi}_j)^T (\vec{\phi}_i - \vec{\phi}_j).$$

Desenvolvendo os termos:

$$J(Y) = \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n w_{ij} \left(\vec{\phi}_i \vec{\phi}_i^T - \vec{\phi}_i \vec{\phi}_j^T - \vec{\phi}_j \vec{\phi}_i^T + \vec{\phi}_j \vec{\phi}_j^T \right).$$

Usando a definição de grau $d_i = \sum_j w_{ij}$, podemos simplificar a expressão:

$$J(Y) = \sum_{i=1}^n d_i \vec{\phi}_i \vec{\phi}_i^T - \sum_{i=1}^n \sum_{j=1}^n w_{ij} \vec{\phi}_i \vec{\phi}_j^T.$$

Na notação matricial, o primeiro termo é o traço da matriz DYY^T e o segundo é o traço de WYY^T . Assim:

$$J(Y) = \text{Tr}(DYY^T) - \text{Tr}(WYY^T).$$

Como o operador traço é invariante a permutações cíclicas, temos:

$$J(Y) = \text{Tr}(Y^T DY) - \text{Tr}(Y^T WY),$$

ou equivalentemente,

$$J(Y) = \text{Tr}(Y^T (D - W)Y) = \text{Tr}(Y^T LY),$$

onde $L = D - W$ é a matriz Laplaciana do grafo.

O problema de imersão multidimensional pode ser formulado como:

$$\begin{aligned} \arg \min_Y \quad & \text{Tr}(Y^T LY) \\ \text{sujeito a} \quad & Y^T DY = I, \end{aligned}$$

onde a restrição $Y^T DY = I$ evita soluções triviais (como o mapeamento constante) e assegura que as novas coordenadas sejam ortogonais sob a métrica induzida por D .

Definimos a função Lagrangiana como:

$$J(Y, \Lambda) = \text{Tr}(Y^T LY) - \text{Tr}[\Lambda(Y^T DY - I)],$$

onde Λ é uma matriz diagonal dos multiplicadores de Lagrange.

Derivando em relação a Y e igualando a zero:

$$\frac{\partial J(Y, \Lambda)}{\partial Y} = 2LY - 2DY\Lambda = 0,$$

o que nos leva ao **problema generalizado de autovalores e autovetores**:

$$LY = DY\Lambda \quad \Rightarrow \quad D^{-1}LY = Y\Lambda.$$

A solução é obtida selecionando as d colunas de Y como sendo os autovetores associados aos d menores autovalores não nulos da matriz Laplaciana normalizada. Assim:

$$Y = \begin{bmatrix} | & | & \dots & | \\ \vec{v}_1 & \vec{v}_2 & \dots & \vec{v}_d \\ | & | & \dots & | \end{bmatrix}_{n \times d}$$

onde cada autovetor $\vec{v}_k$ define uma coordenada independente do novo espaço embutido. A matriz Y representa, portanto, o conjunto de novas coordenadas que preservam a estrutura de vizinhança do grafo original em um espaço de menor dimensionalidade.

5.5 Relação entre cortes em grafos e agrupamento espectral

Uma das interpretações mais elegantes e intuitivas do *agrupamento espectral* está enraizada na teoria dos grafos. Ao representar um conjunto de dados como um grafo ponderado $G = (V, E, W)$, em que os vértices $V = \{v_1, v_2, \dots, v_n\}$ correspondem às amostras e as arestas E são ponderadas pelos pesos w_{ij} que refletem a similaridade entre os vértices v_i e v_j , o problema de agrupamento de dados pode ser visto como o problema de **particionar o grafo** em subconjuntos de vértices fortemente conectados internamente e fracamente conectados entre si.

Algoritmo 8 Algoritmo de Agrupamento Espectral (Spectral Clustering)

Require: Conjunto de dados $\mathcal{X} = \{\vec{x}_1, \vec{x}_2, \dots, \vec{x}_n\}$, número de clusters K

Ensure: Partição dos dados em K grupos

1: **function** SPECTRALCLUSTERING($\mathcal{X}, K$)

2: **1. Construção do grafo de similaridade:**

3: Construir o grafo $G = (V, E)$, onde cada vértice v_i representa um ponto $\vec{x}_i$.

4: Calcular a matriz de afinidades $W = [w_{ij}]$, onde:

$$w_{ij} = \exp\left(-\frac{\|\vec{x}_i - \vec{x}_j\|^2}{2\sigma^2}\right)$$

se $\vec{x}_j$ estiver entre os k vizinhos mais próximos de $\vec{x}_i$, e $w_{ij} = 0$ caso contrário.

5: **2. Cálculo das matrizes estruturais:**

6: Calcular a matriz de graus $D = \text{diag}(d_1, d_2, \dots, d_n)$, com $d_i = \sum_j w_{ij}$.

7: Calcular a matriz Laplaciana normalizada:

$$L_{\text{sym}} = D^{-1/2}(D - W)D^{-1/2}$$

ou, alternativamente, a Laplaciana não simétrica $L_{\text{rw}} = D^{-1}(D - W)$.

8: **3. Decomposição espectral:**

9: Calcular os d menores autovalores não nulos de L_{sym} e os autovetores correspondentes.

10: Formar a matriz $Y \in \mathbb{R}^{n \times d}$, em que cada coluna é um autovetor.

11: **4. Normalização (opcional):**

12: Normalizar cada linha de Y para ter norma unitária:

$$U_{ij} \leftarrow \frac{Y_{ij}}{\sqrt{\sum_j Y_{ij}^2}}$$

13: **5. Agrupamento final:**

14: Aplicar o algoritmo **K-médias** sobre as linhas da matriz Y para obter os K clusters finais.

15: **return** Grupos $\{C_1, C_2, \dots, C_K\}$

16: **end function**

Cortes em grafos

Formalmente, dado um grafo ponderado G , o *corte* entre dois subconjuntos disjuntos A e B de vértices é definido como:

$$\text{cut}(A, B) = \sum_{i \in A, j \in B} w_{ij}.$$

O problema do **corte mínimo** (*minimum cut*) consiste em encontrar uma partição de V em dois subconjuntos A e B que minimizem $\text{cut}(A, B)$. Intuitivamente, queremos remover o menor número possível de arestas (ou de peso total mínimo) que desconectem o grafo em componentes distintas.

No entanto, o corte mínimo puro tende a gerar soluções degeneradas — por exemplo, separando apenas um único vértice do restante do grafo. Para evitar isso, foram propostas variantes normalizadas, como o *Normalized Cut* ($Ncut$) e o *Ratio Cut* ($RCut$), que penalizam cortes desbalanceados. Por exemplo:

$$\text{Rcut}(A, B) = \frac{\text{cut}(A, B)}{|A|} + \frac{\text{cut}(A, B)}{|B|},$$

onde $|A|$ denota a cardinalidade do conjunto A (número de vértices em A).

Relação com a Laplaciana do grafo

Surpreendentemente, esses critérios de corte podem ser expressos em termos da **matriz Laplaciana** $L = D - W$, em que D é a matriz diagonal dos graus. Pode-se mostrar que o problema de minimizar o *Ratio Cut* é equivalente a resolver o seguinte problema de otimização quadrática:

$$\min_{\vec{f} \in \mathbb{R}^n} \vec{f}^T L \vec{f}$$

sujeito a restrições de ortogonalidade e normalização. O vetor $\vec{f}$ aqui representa uma função de particionamento suave sobre os vértices, de modo que valores próximos de $\vec{f}_i$ e $\vec{f}_j$ indicam que v_i e v_j pertencem ao mesmo grupo.

O objetivo consiste em resolver o seguinte problema de minimização:

$$\min_{A \cup B = V} \text{RCut}(A, B),$$

em que A e B são subconjuntos disjuntos tais que $A \cup B = V$ e $A \cap B = \emptyset$.

Para associar o critério RCut à matriz Laplaciana L , definimos um vetor $\vec{f} \in \mathbb{R}^n$ como:

$$f_i = \begin{cases} \sqrt{\frac{|B|}{|A|}}, & v_i \in A, \\ -\sqrt{\frac{|A|}{|B|}}, & v_i \in B. \end{cases}$$

Sabemos que minimizar

$$\vec{f}^T L \vec{f} = \frac{1}{2} \sum_{(i,j) \in E} w_{ij} (f_i - f_j)^2$$

é o objetivo da imersão laplaciana. Note que há dois tipos de arestas (i, j) para as quais $f_i \neq f_j$: aquelas que conectam um vértice de A a um vértice de B , e vice-versa. Assim, substituindo os valores de f_i e f_j , temos:

$$\vec{f}^T L \vec{f} = \frac{1}{2} \sum_{i \in A, j \in B} w_{ij} \left(\sqrt{\frac{|B|}{|A|}} + \sqrt{\frac{|A|}{|B|}} \right)^2.$$

Expandindo o termo quadrático:

$$\left(\sqrt{\frac{|B|}{|A|}} + \sqrt{\frac{|A|}{|B|}} \right)^2 = \frac{|B|}{|A|} + \frac{|A|}{|B|} + 2.$$

Logo:

$$\vec{f}^T L \vec{f} = \frac{1}{2} \left(\frac{|B|}{|A|} + \frac{|A|}{|B|} + 2 \right) \sum_{i \in A, j \in B} w_{ij}.$$

Como o somatório $\sum_{i \in A, j \in B} w_{ij}$ é o peso total do corte entre A e B , temos:

$$\text{cut}(A, B) = \sum_{i \in A, j \in B} w_{ij},$$

e portanto:

$$\vec{f}^T L \vec{f} = \frac{1}{2} \left(\frac{|B|}{|A|} + \frac{|A|}{|B|} + 2 \right) \text{cut}(A, B).$$

Simplificando, obtemos:

$$\vec{f}^T L \vec{f} = \left(\frac{|A| + |B|}{|A|} + \frac{|A| + |B|}{|B|} \right) \text{cut}(A, B).$$

Como $|A| + |B| = |V|$, podemos escrever:

$$\vec{f}^T L \vec{f} = |V| \left(\frac{\text{cut}(A, B)}{|A|} + \frac{\text{cut}(A, B)}{|B|} \right),$$

ou seja:

$$\vec{f}^T L \vec{f} = |V| \text{RCut}(A, B).$$

Propriedades Importantes

1. O vetor $\vec{f}$ definido acima é ortogonal ao vetor constante $\vec{1}$:

$$\langle \vec{f}, \vec{1} \rangle = \sum_{i=1}^n f_i = |A| \sqrt{\frac{|B|}{|A|}} - |B| \sqrt{\frac{|A|}{|B|}} = \sqrt{|A||B|} - \sqrt{|A||B|} = 0.$$

2. A norma de $\vec{f}$ é:

$$\|\vec{f}\|^2 = \sum_{i=1}^n f_i^2 = |A| \frac{|B|}{|A|} + |B| \frac{|A|}{|B|} = |A| + |B| = |V| = n.$$

Assim, minimizar o RCut é equivalente a resolver o seguinte problema:

$$\min_{\vec{f}} \vec{f}^T L \vec{f}$$

sujeito a:

$$\langle \vec{f}, \vec{1} \rangle = 0 \quad \text{e} \quad \|\vec{f}\| = \sqrt{n}.$$

Em que:

$$f_i = \begin{cases} \sqrt{\frac{|B|}{|A|}}, & v_i \in A, \\ -\sqrt{\frac{|A|}{|B|}}, & v_i \in B. \end{cases}$$

Note que este problema é **NP-Hard**, pois o vetor solução $\vec{f}$ possui n componentes, cada uma podendo assumir dois valores distintos (um para cada conjunto A e B), resultando em 2^n possíveis configurações. Assim, uma busca exaustiva por todas as partições seria computacionalmente inviável, já que o número de soluções cresce exponencialmente — situação análoga a percorrer todos os vértices de um hipercubo em $\mathbb{R}^n$.

Relaxação espectral e complexidade computacional

O problema de encontrar a partição discreta ótima ou o corte mínimo (isto é, a melhor atribuição de vértices a clusters distintos) é **NP-Hard**, uma vez que requer examinar todas as possíveis divisões do conjunto de vértices. Entretanto, ao relaxar a restrição discreta sobre $\vec{f}$ e permitir que ela assuma valores contínuos, o problema torna-se um problema de **autovalores generalizado**:

$$L\vec{f} = \lambda D\vec{f},$$

cujas soluções podem ser obtidas de forma eficiente através de métodos polinomiais de decomposição espectral.

Assim, o **agrupamento espectral** surge como uma *relaxação contínua* do problema de corte mínimo, oferecendo uma aproximação elegante e computacionalmente tratável para um problema originalmente intratável. Os autovetores associados aos menores autovalores não nulos da matriz Laplaciana (ou Laplaciana normalizada) fornecem uma representação embutida (*embedding*) dos vértices em um espaço euclidiano de baixa dimensão, onde agrupamentos podem ser facilmente obtidos, por exemplo, por meio do algoritmo *k*-médias.

Portanto, podemos afirmar que o *Spectral Clustering* transforma um problema de otimização combinatória (NP-Hard) em um problema contínuo de álgebra linear (polinomial), em que a estrutura de conectividade do grafo é revelada pelo espectro da matriz Laplaciana. Essa é uma das razões pelas quais os métodos espectrais são amplamente utilizados em análise de redes, reconhecimento de padrões e aprendizado de representações geométricas.

5.6 Considerações finais

O agrupamento espectral representa uma das abordagens mais elegantes e matematicamente fundamentadas para a identificação de estruturas em dados complexos. Ao contrário de métodos tradicionais, como o *k*-médias, que dependem de suposições implícitas sobre a forma e a separabilidade dos agrupamentos no espaço original, o agrupamento espectral atua sobre uma representação baseada em grafos, explorando as propriedades espectrais da matriz Laplaciana.

Essa formulação permite capturar relações de vizinhança e conectividade que não são necessariamente lineares, tornando o método particularmente eficaz para detectar agrupamentos de formas arbitrárias e em contextos onde a geometria do conjunto de dados é intrinsecamente não euclidiana.

Do ponto de vista teórico, o agrupamento espectral estabelece uma ponte notável entre problemas combinatórios (como o *Minimum Cut* e o *Normalized Cut*) e problemas contínuos resolvidos por meio da decomposição espectral. A relaxação do problema discreto original — que é **NP-Hard** — conduz a uma formulação contínua em termos de autovalores e autovetores da Laplaciana, permitindo uma solução polinomial via álgebra linear. Dessa forma, o agrupamento espectral fornece uma aproximação eficiente e matematicamente justificável para a otimização de critérios de corte em grafos.

Além de sua fundamentação teórica, o método também apresenta grande versatilidade prática: ele pode ser combinado com diferentes estratégias de construção de grafos (como o ε -vizinhança ou *k*-vizinhos mais próximos), e pode empregar distintas variantes da Laplaciana (comum, simétrica ou normalizada), adaptando-se a diferentes tipos de dados e escalas.

Em síntese, o agrupamento espectral deve ser entendido não apenas como um algoritmo de agrupamento, mas como um **framework unificador** que combina conceitos de teoria dos grafos, geometria diferencial e álgebra linear para revelar a estrutura latente dos dados. Ao final, a decomposição espectral fornece uma imersão dos dados em um espaço onde relações de similaridade locais se traduzem em separações globais bem definidas, resultando em agrupamentos coesos e interpretáveis.

6 Agrupamento de dados com em árvores geradoras mínimas

O método de agrupamento baseado em Árvores Geradoras Mínimas (MST — Minimum Spanning Trees) é uma abordagem que explora propriedades topológicas dos dados para identificar estruturas de agrupamento sem impor suposições fortes sobre a forma ou a distribuição dos grupos. Diferentemente de métodos como o k-médias, que pressupõem clusters aproximadamente esféricos, ou de modelos probabilísticos como o GMM, que assumem uma estrutura paramétrica subjacente, o método MST é essencialmente não paramétrico e baseado em grafos, permitindo lidar com clusters de formatos arbitrários.

A ideia central consiste em representar o conjunto de dados como um grafo ponderado, em que cada vértice corresponde a uma amostra e as arestas conectam pares de pontos com pesos dados por uma medida de dissimilaridade (geralmente a distância Euclidiana). A Árvore Geradora Mínima é, então, o subgrafo conexo que conecta todos os vértices com o menor custo total possível, ou seja, minimizando a soma dos pesos das arestas.

Uma vez construída a MST, o processo de agrupamento pode ser realizado removendo-se as arestas mais longas — que, em geral, conectam regiões de baixa densidade —, resultando em um conjunto de subárvores correspondentes a diferentes grupos de dados. Essa estratégia reflete uma intuição geométrica: clusters densos correspondem a regiões fortemente conectadas, enquanto as transições entre clusters aparecem como arestas longas que unem regiões distintas do espaço de amostras.

Além de sua simplicidade conceitual, o método MST oferece uma interpretação visual e hierárquica do agrupamento, sendo particularmente útil em cenários onde a noção de densidade ou conectividade é mais natural do que a de centroide. Aplicações desse tipo de agrupamento são encontradas em análise de imagens, bioinformática, e reconhecimento de padrões em dados de alta dimensionalidade.

Nos tópicos seguintes, exploraremos a fundamentação teórica desse método, suas variantes e algoritmos clássicos (como os de Prim e Kruskal) para a construção da árvore geradora mínima, bem como estratégias de corte e critérios para determinar o número ótimo de clusters.

6.1 O problema da árvore geradora mínima (MST)

O problema da **Árvore Geradora Mínima** (*Minimum Spanning Tree* — MST) consiste em, dentre todas as possíveis árvores geradoras de um grafo G , obter aquela de menor peso total.

Definição 1. Dado um grafo ponderado $G = (V, E, w)$, onde $w : E \rightarrow \mathbb{R}^+$ representa o peso associado a cada aresta $e \in E$, o objetivo é encontrar a árvore geradora T que minimiza o peso total:

$$w(T) = \sum_{e \in T} w(e)$$

isto é, a soma dos pesos das arestas que compõem a árvore.

Exemplo motivador: suponha que se deseje conectar diversos bairros de uma cidade utilizando fibra

óptica. Qual é a forma de interligação que minimiza o custo total de instalação, garantindo que todos os bairros permaneçam conectados?

Estratégia gulosa: A abordagem clássica para resolver o problema da MST é baseada em um método *guloso* (*greedy*), em que a cada passo escolhe-se a aresta de menor custo que seja *segura*. Uma aresta é considerada *segura* se, ao ser adicionada à árvore parcial T , esta continuar sendo uma árvore (isto é, sem formar ciclos).

A seguir, apresentamos uma versão genérica do algoritmo para construção da árvore geradora mínima:

Algoritmo 9 MST_Generica(G, w)

```

1:  $T \leftarrow \emptyset$ 
2: while  $T$  não for uma árvore geradora do
3:   Encontre uma aresta segura  $(u, v)$ 
4:    $T \leftarrow T \cup \{(u, v)\}$ 
5: end while return  $T$ 

```

Questão importante: Como determinar se uma aresta é *segura*?

Nos próximos tópicos veremos dois algoritmos clássicos que resolvem essa questão de maneiras distintas — o **Algoritmo de Kruskal** e o **Algoritmo de Prim**. Cada um deles adota uma estratégia específica para definir e selecionar as arestas seguras, embora ambos garantam a obtenção de uma árvore geradora mínima ao final do processo.

6.2 O Algoritmo de Kruskal

Objetivo: A cada passo, escolher a aresta (u, v) de menor custo que não forme um ciclo no conjunto atual $T + (u, v)$.

Ideia: Iniciar o algoritmo considerando cada vértice de G como a raiz de uma árvore independente. A cada iteração, adiciona-se a aresta de menor custo que conecta duas árvores distintas, até que todas estejam unidas em uma única árvore geradora mínima.

Para isso, utilizam-se três primitivas fundamentais, responsáveis por representar e manipular conjuntos disjuntos de vértices (estrutura conhecida como *Union-Find* ou *Disjoint Set Union* — *DSU*):

(1) Make_Set(v): cria uma árvore contendo um único vértice v como raiz:

Algoritmo 10 Make_Set(v)

```

1:  $v.p \leftarrow v$                                      # o pai do vértice  $v$  é ele mesmo (raiz)
2:  $v.rank \leftarrow 0$                                 # altura inicial da árvore (nível)

```

(2) Find_Set(v): Retorna a raiz da árvore à qual o vértice v pertence. Utiliza o método de *path compression* para acelerar buscas subsequentes.

Algoritmo 11 Find_Set(v)

```
1: if  $v \neq v.p$  then                                     # se  $v$  não é raiz
2:    $v.p \leftarrow \text{Find\_Set}(v.p)$                      # compressão do caminho
3: end if
4: return  $v.p$                                              # retorna a raiz da árvore de  $v$ 
```

(3) **Union(u, v)**: Faz a fusão das raízes das árvores contendo u e v , criando uma única árvore.

Algoritmo 12 Union(u, v)

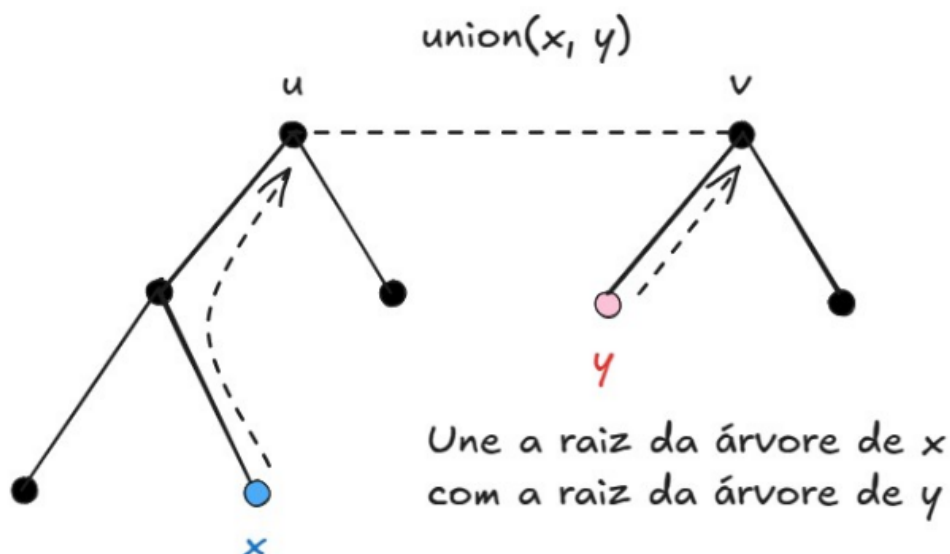
```
1: Link(Find_Set( $u$ ), Find_Set( $v$ ))
```

(4) **Função auxiliar**: Link(u, v) — realiza a fusão com base na altura (rank) das árvores.

Algoritmo 13 Link(u, v)

```
1: if  $u.rank > v.rank$  then
2:    $v.p \leftarrow u$ 
3: else
4:    $u.p \leftarrow v$ 
5:   if  $u.rank == v.rank$  then
6:      $v.rank \leftarrow v.rank + 1$ 
7:   end if
8: end if
```

Comentário: Quando realizamos Union(x, y), primeiro encontramos as raízes das árvores de x e y , que chamaremos de u e v , respectivamente. A função Link realiza a fusão dessas duas árvores, tornando a árvore de maior altura a nova raiz. Se ambas tiverem a mesma altura, escolhe-se uma como raiz e incrementa-se o seu rank em uma unidade. Esse procedimento evita o crescimento excessivo da profundidade da árvore, garantindo eficiência próxima a $O(\alpha(n))$, onde α é a função inversa de Ackermann, que cresce muito devagar (bem mais devagar que a função logarítmica).



Algoritmo de Kruskal

O pseudocódigo completo do algoritmo é apresentado a seguir:

Algoritmo 14 MST_Kruskal(G, w)

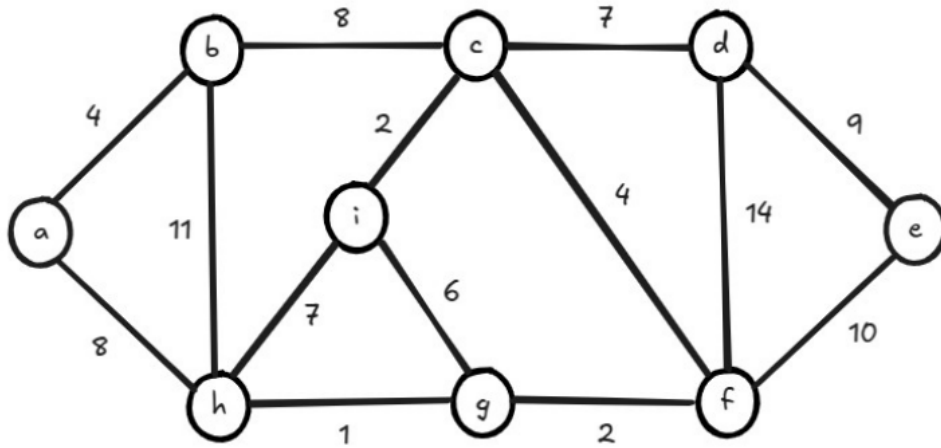
```
1:  $T \leftarrow \emptyset$ 
2: for cada  $v \in V$  do
3:   Make_Set( $v$ )
4: end for
5: Crie uma lista com todas as arestas  $e \in E$ 
6: Ordene a lista em ordem crescente de pesos  $w(e)$ 
7: for cada aresta  $e = (u, v)$  na lista ordenada do
8:   if Find_Set( $u$ )  $\neq$  Find_Set( $v$ ) then
9:      $T \leftarrow T \cup \{(u, v)\}$ 
10:    Union( $u, v$ )
11:   end if
12: end for return  $T$ 
```

Discussão: O algoritmo de Kruskal segue uma *estratégia gulosa*, pois em cada iteração seleciona a aresta de menor peso possível que não forma ciclos na árvore parcial. Ao final, o conjunto T obtido contém exatamente $|V| - 1$ arestas, correspondendo à **árvore geradora mínima (MST)** do grafo G .

A seguir, realizaremos uma simulação passo a passo (*trace*) da execução do algoritmo de Kruskal. Para isso, utilizaremos a seguinte notação:

- E^- : conjunto das arestas de peso mínimo **não seguras**, ou seja, aquelas que formam ciclos (Find_Set(u) = Find_Set(v));
- E^+ : conjunto das arestas de peso mínimo **seguras**, isto é, aquelas que não formam ciclos (Find_Set(u) $\neq$ Find_Set(v));
- e_k : aresta escolhida no passo k .

Exemplo: Suponha que os vértices de um grafo representem bairros de uma cidade e as arestas, os custos de instalação de fibra óptica entre pares de bairros. Deseja-se determinar o custo mínimo de interligar todos os bairros — isto é, encontrar a **árvore geradora mínima (MST)** do grafo G , o que pode ser feito pelo algoritmo de Kruskal.



A lista das arestas ordenadas de acordo com seus pesos é dada por:

$$E_w = [1, 2, 2, 4, 4, 6, 7, 7, 8, 8, 9, 10, 11, 14]$$

$$E = \{(g, h), (c, i), (f, g), (a, b), (c, f), (g, i), (c, d), (a, h), (b, c), (d, e), (e, f), (b, h), (d, f)\}$$

Tabela 1 — Execução passo a passo do algoritmo de Kruskal

Passo k	E^- (arestas não seguras)	E^+ (arestas seguras)	Aresta escolhida e_k
1	—	$\{(g, h)\}$	(g, h)
2	$\{(c, i), (f, g)\}$	—	(c, i)
3	$\{(g, f)\}$	—	(g, f)
4	$\{(a, b), (c, f)\}$	—	(a, b)
5	$\{(c, f)\}$	—	(c, f)
6	$\{(g, i)\}$	—	—
7	$\{(h, i)\}$	$\{(c, d)\}$	(c, d)
8	$\{(a, h), (b, c)\}$	—	(a, h)
9	$\{(b, c)\}$	—	—
10	$\{(d, e)\}$	—	(d, e)

Tabela 4: Simulação do algoritmo de Kruskal passo a passo.

Ao final da execução, o conjunto de arestas selecionadas forma a **Árvore Geradora Mínima (MST)** do grafo. Essa árvore conecta todos os vértices com o menor custo total possível, sem formar ciclos.

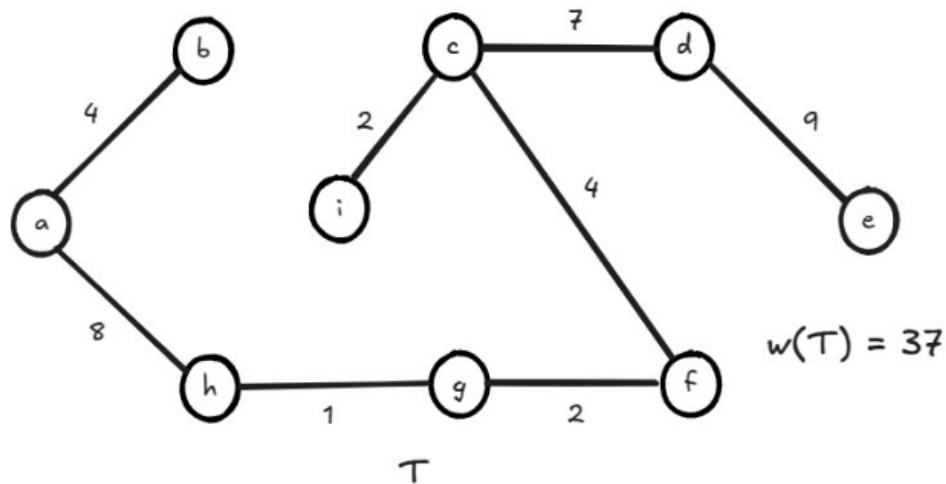


Figura 7: Árvore Geradora Mínima (MST) resultante da execução do algoritmo de Kruskal.

Nesta seção, demonstraremos a **otimalidade do algoritmo de Kruskal**, isto é, que o algoritmo sempre retorna uma *árvore geradora mínima* (MST) do grafo G .

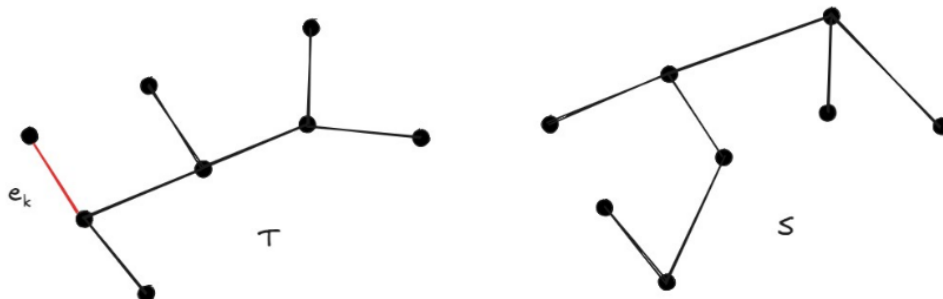
Teorema 3. *Toda árvore T gerada pelo algoritmo de Kruskal é uma árvore geradora mínima (MST) de G .*

Esse resultado garante que o algoritmo é **correto** e **ótimo**, pois sempre retorna uma solução de custo mínimo.

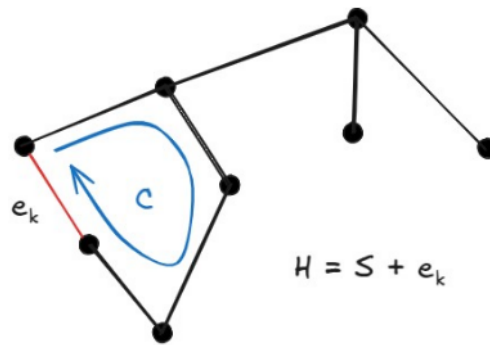
Prova por contradição.

Hipótese: Seja T a árvore retornada pelo algoritmo de Kruskal.

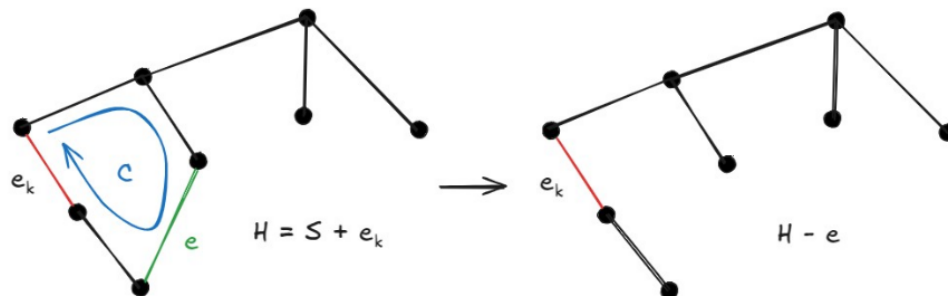
1. Suponha, por contradição, que exista uma árvore $S \neq T$ tal que $w(S) < w(T)$, onde $w(\cdot)$ representa o peso total da árvore.
2. Seja $e_k \in T$ a primeira aresta adicionada em T que não pertence a S . Como S e T são diferentes, tal aresta necessariamente existe.



3. Considere o subgrafo $H = S + e_k$. Como S é uma árvore, a adição de e_k cria um **ciclo** C em H (pois uma árvore com n vértices e n arestas contém exatamente um ciclo).



4. Dentro desse ciclo C , existe pelo menos uma aresta $e \in S$ tal que $e \notin T$ (caso contrário, o ciclo C já existiria em T , o que é impossível, pois T é uma árvore). O subgrafo $H - e$ é conexo e contém exatamente $n - 1$ arestas, logo $H - e$ é uma árvore geradora de G .



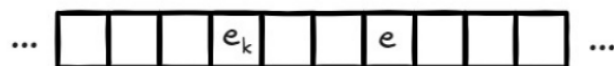
5. Como o algoritmo de Kruskal processa as arestas em ordem crescente de peso, temos que:

$$w(e_k) \leq w(e)$$

Assim, o peso total de $H - e$ é menor ou igual ao de S :

$$w(H - e) \leq w(S)$$

Lista de arestas (após ordenação)



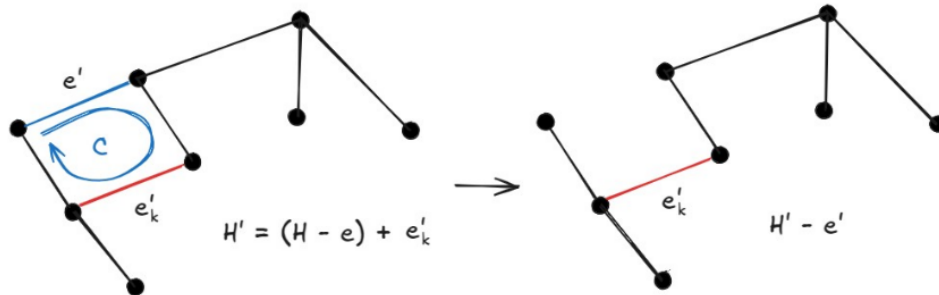
$$e_k \text{ precede } e \rightarrow w(e_k) \leq w(e)$$

6. Repetindo o processo, é possível substituir sucessivamente as arestas de S por arestas de T , gerando uma sequência de árvores que se aproximam de T :

$$S \rightarrow (H - e) \rightarrow (H' - e') \rightarrow (H'' - e'') \rightarrow \dots \rightarrow T$$

com

$$w(S) \geq w(H - e) \geq w(H' - e') \geq \dots \geq w(T)$$



Dessa forma, chegamos a uma contradição com a hipótese inicial de que $w(S) < w(T)$. Portanto, não existe árvore com peso menor que T , o que mostra que T é de peso mínimo.

T é uma árvore geradora mínima (MST) de G .

□

6.2.1 Análise da complexidade

Nesta seção, analisaremos a complexidade de tempo do algoritmo de Kruskal com base em suas operações fundamentais.

Primeiramente, observemos o custo das operações elementares do algoritmo:

- **Make_Set (v)**: Possui custo constante $O(1)$, pois apenas inicializa o vértice v como uma árvore isolada. Como essa operação é executada n vezes (uma para cada vértice), o custo total é:

$$O(n)$$

- **Ordenação das arestas**: A ordenação das m arestas pelo peso pode ser realizada utilizando algoritmos eficientes de ordenação como *MergeSort* ou *QuickSort*, ambos com complexidade:

$$O(m \log m)$$

- **Find_Set (v)**: Retorna a raiz da árvore à qual v pertence. Essa operação depende da altura h da árvore. No caso de uma árvore balanceada, temos $h = \log n$, de modo que:

$$O(\log n)$$

(se a árvore não for binária, apenas a base do logaritmo se altera).

- **Union(u, v)**: Essa operação realiza a fusão de duas árvores distintas, e internamente executa duas chamadas a `Find_Set`, uma para cada vértice. Assim, seu custo também é:

$$O(\log n)$$

Note que a operação `Find_Set(v)` é chamada duas vezes para cada aresta (u, v) (uma vez para cada extremidade). Logo, o custo total associado a essas chamadas é:

$$2m \cdot O(\log n) = O(m \log n)$$

A operação `Union(u, v)` é executada apenas quando uma aresta é adicionada à árvore, o que ocorre $n - 1$ vezes. Assim, seu custo é:

$$O(n \log n)$$

Somando todos os custos, temos:

$$C = O(n) + O(m \log m) + O(m \log n) + O(n \log n)$$

O termo dominante é o de maior ordem assintótica. Sabendo que $m < n^2$, temos:

$$\log m < \log n^2 = 2 \log n = O(\log n)$$

Portanto, o termo $O(m \log m)$ é assintoticamente equivalente a $O(m \log n)$.

$$\boxed{C = O(m \log n)}$$

Assim, concluímos que a complexidade de tempo do **algoritmo de Kruskal** é $O(m \log n)$, o que o torna um dos algoritmos mais eficientes para encontrar árvores geradoras mínimas em grafos esparsos.

6.3 O Algoritmo de Prim

O **algoritmo de Prim** também utiliza uma abordagem *gulosa* (*greedy*) para a construção da Árvore Geradora Mínima (MST). A ideia central é iniciar a partir de um vértice raiz r e, a cada passo, adicionar a aresta de menor custo que conecta o conjunto de vértices já visitados ao conjunto dos vértices ainda não visitados. Esse processo continua até que todos os vértices estejam conectados, formando uma árvore de custo mínimo.

Definição das variáveis

- $\lambda(v)$ ou $v.key$: menor custo de entrada para o vértice v até o momento;
- $\pi(v)$: predecessor do vértice v na árvore T ;

- Q : fila de prioridades (*priority queue*), onde a maior prioridade é atribuída ao vértice com o menor valor de $\lambda(v)$.

Veremos a seguir que o algoritmo de Prim é muito similar ao algoritmo de Dijkstra, diferenciando-se apenas pelo critério de atualização dos pesos.

Algoritmo 15 MST_Prim(G, w, r)

```

1: for each  $v \in V$  do
2:    $\lambda(v) \leftarrow \infty$ 
3:    $\pi(v) \leftarrow \text{nil}$ 
4: end for
5:  $\lambda(r) \leftarrow 0$ 
6:  $Q \leftarrow \emptyset$                                      # fila de prioridades
7: for each  $v \in V$  do
8:   Insert( $Q, v$ )
9: end for
10: while  $Q \neq \emptyset$  do
11:    $u \leftarrow \text{ExtractMin}(Q)$ 
12:    $S \leftarrow S \cup \{u\}$ 
13:   for each  $v \in N(u)$  do
14:     if  $v \in Q$  and  $w(u, v) < \lambda(v)$  then
15:        $\lambda(v) \leftarrow w(u, v)$                          # encontrou melhor entrada
16:        $\pi(v) \leftarrow u$ 
17:       Decrease_Key( $Q, v, w(u, v)$ )
18:     end if
19:   end for
20: end while
21: return  $T = \{(\pi(v), v) \mid v \in V \setminus \{r\}\}$ 

```

Comentário sobre a condição interna

Note que o comando `if` dentro do laço `for` pode ser escrito de forma equivalente como:

Forma equivalente da condição interna

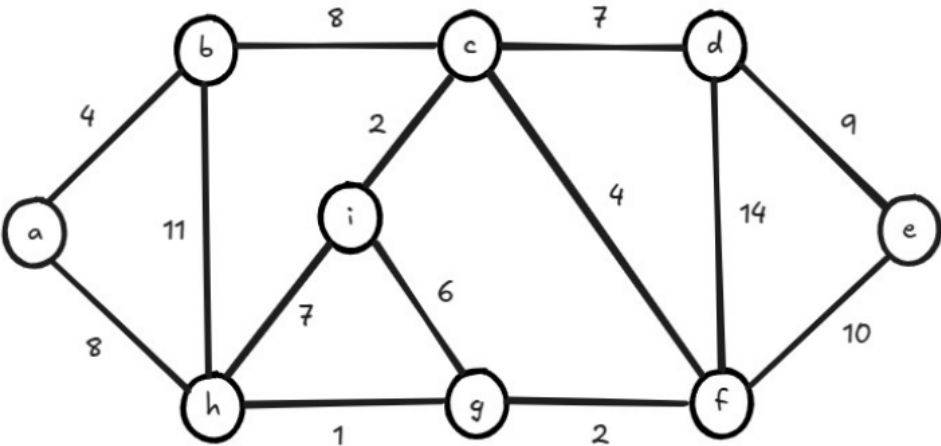
```

1: if  $v \in Q$  then
2:    $\lambda(v) \leftarrow \min\{\lambda(v), w(u, v)\}$ 
3:   if  $\lambda(v)$  foi atualizada then
4:      $\pi(v) \leftarrow u$ 
5:     Decrease_Key( $Q, v, w(u, v)$ )
6:   end if
7: end if

```

Esse procedimento garante que, em cada iteração, o algoritmo sempre seleciona a aresta de menor custo que expande a árvore parcial, assegurando a construção incremental da **árvore geradora mínima**.

A seguir veremos um trace da execução completa do algoritmo de Prim em um simples exemplo.



Durante a execução do algoritmo de Prim, a **fila de prioridades** Q armazena os vértices ainda não visitados, ordenados de acordo com o menor valor atual de $\lambda(v)$. A tabela a seguir mostra a evolução dessa fila ao longo das iterações do algoritmo.

Evolução da Fila de Prioridades

Tabela 5: Evolução da fila de prioridades $\lambda(v)$ no algoritmo de Prim

Fila Q	a	b	c	d	e	f	g	h	i
$\lambda(v)^{(0)}$	0	∞	∞	∞	∞	∞	∞	∞	∞
$\lambda(v)^{(1)}$		4	∞	∞	∞	∞	∞	8	∞
$\lambda(v)^{(2)}$			8	∞	∞	∞	∞	8	∞
$\lambda(v)^{(3)}$				7	∞	4	∞	8	2
$\lambda(v)^{(4)}$				7	∞	4	6	7	
$\lambda(v)^{(5)}$				7	10		2	7	
$\lambda(v)^{(6)}$				7	10			1	
$\lambda(v)^{(7)}$				7	9				
$\lambda(v)^{(8)}$					9				

Observações

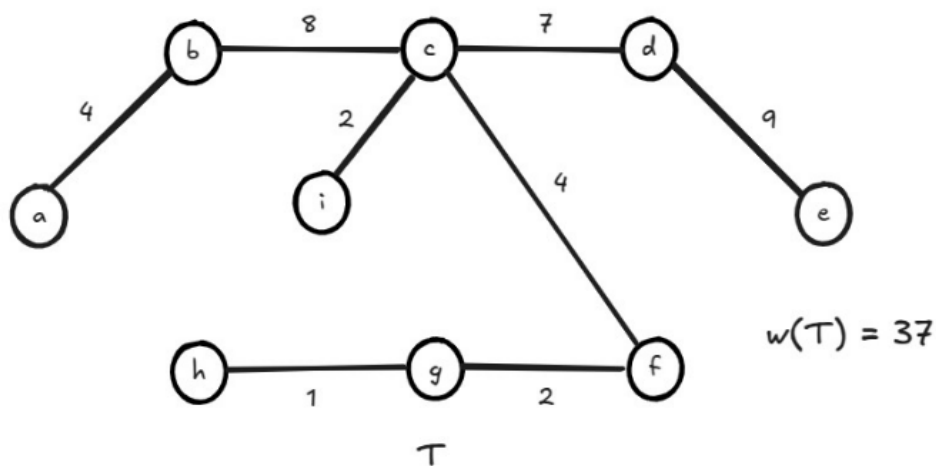
- A cada iteração, o vértice u extraído de Q é aquele com o menor valor de $\lambda(v)$.
- O vetor $\pi(v)$ registra a árvore parcial construída ao longo do processo.
- Quando todos os vértices são removidos de Q , a estrutura de predecessores π define a **árvore geradora mínima**.

Ordem de Acesso aos Vértices

A seguir, apresentamos a ordem de extração dos vértices da fila de prioridades e a atualização dos valores de $\lambda(v)$ e $\pi(v)$ a cada iteração:

Iteração	u	$V' = \{v \in N(u) \wedge v \in Q\}$	Atualizações
1	a	$\{b, h\}$	$\lambda(b) = \min\{\infty, w(a, b)\} = 4 \Rightarrow \pi(b) = a$ $\lambda(h) = \min\{\infty, w(a, h)\} = 8 \Rightarrow \pi(h) = a$
2	b	$\{c, h\}$	$\lambda(c) = \min\{\infty, w(b, c)\} = 8 \Rightarrow \pi(c) = b$ $\lambda(h) = \min\{8, w(b, h)\} = 8$
3	c	$\{d, f, i\}$	$\lambda(d) = \min\{\infty, w(c, d)\} = 7 \Rightarrow \pi(d) = c$ $\lambda(f) = \min\{\infty, w(c, f)\} = 4 \Rightarrow \pi(f) = c$ $\lambda(i) = \min\{\infty, w(c, i)\} = 2 \Rightarrow \pi(i) = c$
4	i	$\{h, g\}$	$\lambda(h) = \min\{8, w(i, h)\} = 7 \Rightarrow \pi(h) = i$ $\lambda(g) = \min\{\infty, w(i, g)\} = 6 \Rightarrow \pi(g) = i$
5	f	$\{d, e, g\}$	$\lambda(d) = \min\{7, w(f, d)\} = 7$ $\lambda(e) = \min\{\infty, w(f, e)\} = 10 \Rightarrow \pi(e) = f$ $\lambda(g) = \min\{6, w(f, g)\} = 2 \Rightarrow \pi(g) = f$
6	g	$\{h\}$	$\lambda(h) = \min\{7, w(g, h)\} = 1 \Rightarrow \pi(h) = g$
7	h	$\emptyset$	—
8	d	$\{e\}$	$\lambda(e) = \min\{10, w(d, e)\} = 9 \Rightarrow \pi(e) = d$
9	e	$\emptyset$	—

Árvore geradora mínima



Note que, apesar da árvore geradora mínima (MST) obtida pelo algoritmo de Prim ser diferente

daquela obtida pelo algoritmo de Kruskal, ambas possuem o mesmo peso mínimo,

$$w(T) = 37.$$

Em geral, a condição para que a MST seja única é que todos os pesos das arestas do grafo $G = (V, E, w)$ sejam distintos, ou seja, não devemos ter arestas com pesos iguais no grafo de entrada.

Mapa de predecessores

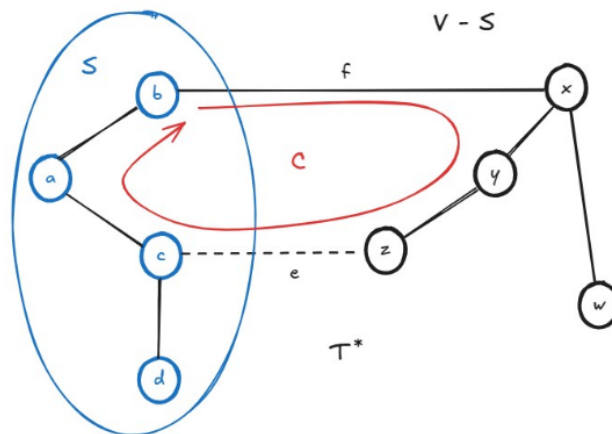
A Tabela 6 apresenta o mapa de predecessores $\pi(v)$ e os valores $\lambda(v)$ associados a cada vértice ao final da execução do algoritmo de Prim.

Tabela 6: Mapa de predecessores e valores de $\lambda(v)$ ao final da execução do algoritmo de Prim

Vértice v	a	b	c	d	e	f	g	h	i
$\pi(v)$ (predecessor)	–	a	b	c	d	c	f	g	c
$\lambda(v)$ (peso mínimo)	0	4	8	7	9	4	2	1	2

A seguir veremos um resultado fundamental para provar a otimalidade do algoritmo de Prim.

Teorema 4 (Propriedade do corte). *Seja $G = (V, E, w)$ um grafo ponderado, S um subconjunto qualquer de V , e $e \in E$ a aresta de menor custo com exatamente uma extremidade em S . Então, a árvore geradora mínima (MST) de G contém a aresta e .*



Prova por contradição.

1. Seja T^* uma MST de G .
2. **Hipótese:** suponha que $e \notin T^*$ (ou seja, e é a aresta de menor peso com exatamente uma extremidade em S , mas não pertence a T^*).

3. Ao adicionar e em T^* , cria-se um único ciclo C .
4. Para que $T^* - e$ fosse ainda uma MST, deve existir alguma outra aresta f com apenas uma extremidade em S (pois, caso contrário, T^* seria desconexo).
5. Então, $T = T^* + e - f$ também é uma árvore geradora. Como $w(e) < w(f)$, segue que T tem peso menor do que T^* .
6. Portanto, T^* não pode ser uma MST de G (contradição), o que invalida a hipótese inicial de que $e \notin T^*$.

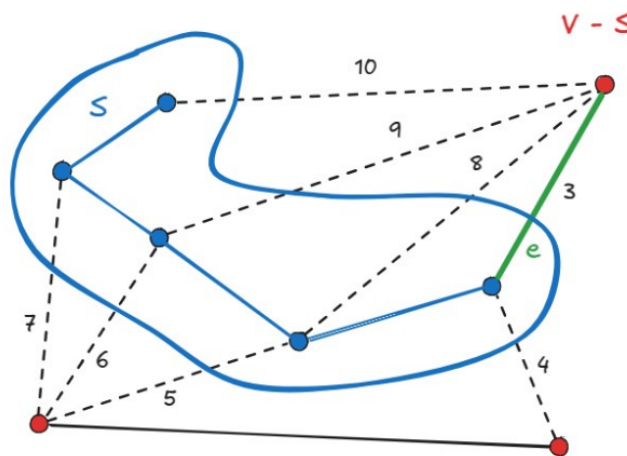
Assim, concluímos que a aresta e pertence a toda MST de G . □

Teorema 5 (Otimalidade do algoritmo de Prim). *A árvore T obtida pelo algoritmo de Prim é uma MST de G .*

Demonstração. 1. Seja S o subconjunto de vértices de G já incluídos na árvore T durante a execução do algoritmo.

2. O algoritmo de Prim, a cada iteração, adiciona em T a aresta de menor custo com exatamente uma extremidade em S .
3. Portanto, pela *Propriedade do Corte*, toda aresta e adicionada por Prim pertence a uma MST de G .

Logo, o algoritmo de Prim é ótimo, pois constrói uma árvore geradora mínima de G . □



6.3.1 Análise da Complexidade

Basicamente, há duas formas de analisar a complexidade do algoritmo de Dijkstra, dependendo das estruturas de dados utilizadas.

Caso 1: Fila Q como array simples

Considere a fila de prioridades Q implementada como um *array* simples de n elementos.

- a) Inicialização dos $\lambda(v)$: $O(n)$
- b) Inserção dos vértices na fila Q : $O(n)$ (ou $O(1)$ por vértice)
- c) O `while` principal é executado n vezes (uma para cada $v \in Q$)
- d) Extração do mínimo: $u = \text{ExtractMin}(Q)$ é $O(n)$ (encontrar o menor elemento no array)
- e) Atualização de $\lambda(v)$ é $O(1)$, mas executada $k = d(u)$ vezes (no `for`)
- f) `Decrease_Key` é $O(1)$ (acesso direto)

Assim, a função $T(n)$ que mede a complexidade total do algoritmo é:

$$T(n) = O(n) + O(n) + O(n) \cdot O(n) + (O(1) + O(1)) \cdot (d(v_1) + d(v_2) + \dots + d(v_n))$$

Sabendo que o produto de dois termos lineares é quadrático, e usando o *Handshake Lemma*, que afirma que a soma dos graus de um grafo é igual a duas vezes o número de arestas ($\sum_i d(v_i) = 2m$), temos:

$$T(n) = O(n) + O(n^2) + O(1) \cdot O(m)$$

Como em todo grafo simples $m < n^2$, concluímos que o algoritmo tem complexidade:

$$\boxed{T(n) = O(n^2)}$$

Caso 2: Fila Q como um heap binário

Agora, considere que a fila de prioridades Q seja implementada como um *heap* binário (*min-heap*).

- a) Inicialização dos $\lambda(v)$: $O(n)$
- b) Inserção dos vértices em Q : $O(n)$ (todos os pesos iguais no início)
- c) O `while` é executado n vezes
- d) Extração do mínimo: $u = \text{ExtractMin}(Q)$ é $O(\log n)$ (operação de *dequeue*)
- e) Atualização de $\lambda(v)$ é $O(1)$, executada $k = d(u)$ vezes
- f) `Decrease_Key` é $O(\log n)$ (pode subir no *heap* até a raiz)

Assim, a complexidade total é:

$$T(n) = O(n) + O(n \log n) + (O(1) + O(\log n)) \cdot (d(v_1) + d(v_2) + \cdots + d(v_n))$$

Usando novamente $\sum_i d(v_i) = 2m$, obtemos:

$$T(n) = O(n) + O(n \log n) + O(m) + O(m \log n)$$

Como o termo com logaritmo domina os demais, temos:

$$T(n) = O((n + m) \log n)$$

Em grafos conexos, $m > n - 1$, portanto:

$$T(n) = O(m \log n)$$

Comparação entre as implementações

A eficiência relativa das duas implementações depende da densidade do grafo. A implementação baseada em array simples (Caso 1) é preferível quando:

$$m \log n > n^2 \implies \frac{m}{n^2} > \frac{1}{\log n}$$

ou, em termos da densidade de arestas $d = \frac{m}{n^2}$:

$$d > \frac{1}{\log n}$$

Portanto:

- Para **grafos densos**, o Caso 1 (array simples) é mais eficiente.
- Para **grafos esparsos**, o Caso 2 (min-heap) é preferível.

Exemplo: para $n = 1024$, temos:

$$\begin{cases} d > 0.1 & \Rightarrow \text{Caso 1 é melhor} \\ d < 0.1 & \Rightarrow \text{Caso 2 é melhor} \end{cases}$$

Para que $d = 0.1$, um grafo com $n = 1024$ vértices deve ter aproximadamente 104857 arestas.

6.4 Comparação entre os algoritmos de Kruskal e Prim

Tanto o algoritmo de **Kruskal** quanto o de **Prim** têm como objetivo comum a construção de uma *árvore geradora mínima* (MST, do inglês *Minimum Spanning Tree*) em um grafo ponderado e conexo.

Ambos se baseiam em uma **estratégia gulosa**, escolhendo iterativamente as arestas de menor peso que mantêm a propriedade de conectividade sem formar ciclos. Apesar de compartilharem o mesmo objetivo, suas abordagens, estruturas de dados e eficiências variam conforme as características do grafo.

6.4.1 Diferenças conceituais e estruturais

O algoritmo de **Kruskal** trabalha de forma **global** sobre as arestas do grafo. Ele considera todas as arestas em ordem crescente de peso e as insere na MST de maneira incremental, desde que não gerem ciclos. Para isso, utiliza-se comumente a estrutura de dados `Union-Find` (ou `Disjoint Set Union - DSU`), que permite determinar rapidamente se duas arestas pertencem a componentes distintas do grafo. O algoritmo termina quando todas as $n - 1$ arestas da árvore mínima forem selecionadas.

Por outro lado, o algoritmo de **Prim** adota uma abordagem **local e incremental**, partindo de um vértice inicial e expandindo a árvore vértice a vértice. A cada passo, é escolhida a aresta de menor peso que conecta um vértice ainda não visitado a algum vértice já incluído na MST. A eficiência do método depende fortemente da estrutura utilizada para armazenar e atualizar as chaves de menor custo (por exemplo, *heaps* ou filas de prioridade).

A Tabela 7 resume as principais diferenças entre os dois métodos.

6.4.2 Conclusão comparativa

Em síntese, o algoritmo de **Kruskal** é preferido quando o grafo é **esparso** e representado por uma **lista de arestas**, pois a ordenação dessas arestas é eficiente e o número de comparações é relativamente pequeno. Já o algoritmo de **Prim** tende a ser mais eficiente em grafos **densos** ou quando o grafo é representado por uma **matriz de adjacências**, uma vez que sua expansão incremental reduz o número de operações de verificação de ciclo.

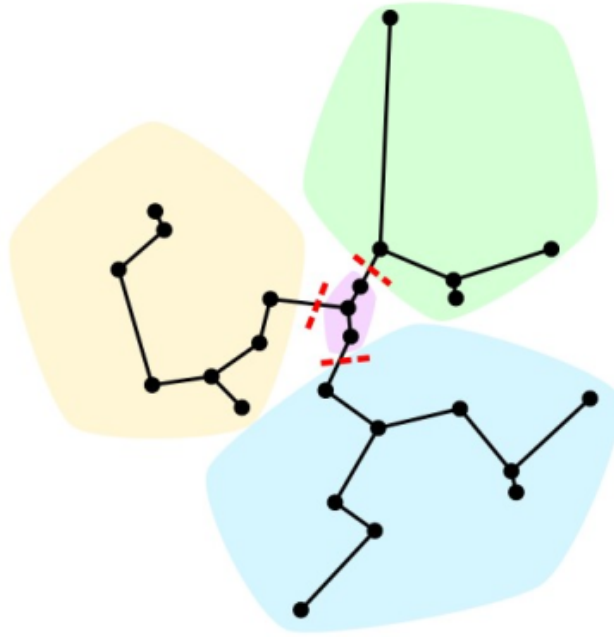
Ambos os algoritmos são corretos e ótimos: garantem a construção de uma MST mínima, e a escolha entre eles deve ser orientada principalmente pelas **características estruturais do grafo** e pelas **estruturas de dados disponíveis**.

Tabela 7: Comparação entre os algoritmos de Kruskal e Prim

Aspecto	Kruskal	Prim
Estratégia	Adiciona arestas em ordem crescente de peso, evitando ciclos.	Expande a MST a partir de um vértice inicial, adicionando sempre a aresta de menor peso incidente ao conjunto atual.
Visão do problema	Global (todas as arestas consideradas simultaneamente).	Local (crescimento progressivo a partir de uma raiz).
Estrutura de dados principal	Estrutura Union-Find (disjoint sets).	Fila de prioridade (min-heap) ou matriz de adjacências.
Complexidade típica	$O(m \log n)$ devido à ordenação das arestas.	$O(m \log n)$ usando min-heap; $O(n^2)$ com matriz de adjacências.
Adequado para	Grafos esparsos com muitas componentes inicialmente desconexas.	Grafos densos ou representados por matrizes de adjacências.
Vantagens	Simple de implementar; independente da representação do grafo; pode ser usado em grafos desconexos (gera floresta mínima).	Mais eficiente em grafos densos; fácil de adaptar para representar MSTs parciais.
Desvantagens	Menos eficiente em grafos densos devido à ordenação completa das arestas.	Depende da escolha da estrutura de dados; não é trivial adaptar para grafos desconexos.

6.5 Agrupamento baseado em MST's

A ideia básica dos algoritmos de agrupamento baseados em MST é remover arestas da MST para criar clusters. Por exemplo, ao remover uma aresta da MST, teremos a partição dos vértices em dois subconjuntos disjuntos, ou seja, 2 agrupamentos. Ao remover 2 arestas da MST, temos 3 agrupamentos, e assim sucessivamente, como ilustra a figura a seguir. Mas então, para obter K agrupamentos, basta remover as K-1 arestas de maior custo? Não, pois em conjuntos de dados reais, a presença de ruído e outliers faz com que os agrupamentos encontrados dessa forma contenham 1 único ponto, o que não é interessante do ponto de vista prático.



6.5.1 Algoritmos divisivos

A classe de algoritmos de agrupamento baseados em MST mais conhecida é composta pelos **algoritmos divisivos**, que utilizam a estratégia *single linkage*, na qual a distância entre dois agrupamentos é determinada por um único par de elementos: aquele formado pelos pontos mais próximos entre os dois grupos.

Um dos primeiros critérios divisivos para agrupamento de dados foi proposto por Calinski e Harabasz em 1974. No contexto de árvores geradoras, a ideia central dessa abordagem é remover iterativamente as arestas da MST de modo a **minimizar a soma dos espalhamentos intra-cluster** das partições, utilizando um critério análogo ao empregado no algoritmo k-médias. A função objetivo a ser minimizada é definida como:

$$\text{WCSS}(X_1, X_2, \dots, X_\ell) = \sum_{i=1}^{\ell} \sum_{\vec{x}_j \in X_i} \|\vec{x}_j - \vec{\mu}_i\|^2 \quad (18)$$

onde $\bar{\mu}_i$ denota o centróide do agrupamento X_i . Como é mais conveniente adotar um problema de **maximização**, define-se:

$$F(X_1, X_2, \dots, X_\ell) = -\text{WCSS}(X_1, X_2, \dots, X_\ell) \quad (19)$$

Assim, a busca consiste em encontrar a melhor forma de remover arestas da MST que maximize o valor de F , ou seja, que minimize a variabilidade intra-grupo.

Considere um conjunto de dados X representado por um grafo completo $G = (V, E, w)$, cujos pesos w correspondem às distâncias euclidianas ponto a ponto entre todos os pares de vértices. O procedimento para obter uma K -partição usando o método *single linkage* é descrito a seguir.

Algoritmo 16 Divisive_MST_Clustering(G, K)

Require: $G = (V, E, w)$: grafo completo ponderado; K : número desejado de agrupamentos

Ensure: Partição $C(V, E'')$ com K agrupamentos

- 1: $T \leftarrow \text{MST}(G) = (V, E')$
- 2: $E'' \leftarrow E'$
- 3: **for** $i = 1$ **to** $K - 1$ **do**
- 4: Encontre a aresta $(u, v) \in E''$ que maximiza:

$$(u, v) = \arg \max_{(u, v) \in E''} F(C(V, E'' - (u, v)))$$

- 5: Remova (u, v) de E''
- 6: **end for**
- 7: **return** $C(V, E'')$

▷ Partição resultante dos vértices

Em cada iteração, o algoritmo remove a aresta mais “fraca” da MST, ou seja, aquela cuja remoção leva à melhor separação das partições segundo o critério F . O processo se repete até que o número desejado de agrupamentos K seja alcançado.

6.5.2 Agrupamento baseado em teoria da informação

Essa abordagem de agrupamento combina o uso de estimativas não-paramétricas de entropia com estruturas de grafo baseadas em árvores geradoras mínimas (MST — *Minimum Spanning Tree*). A essência do método é tratar o agrupamento como a maximização da informação mútua entre as amostras e os rótulos de cluster atribuídos, e utilizar uma estimativa de entropia derivada do comprimento de uma MST construída sobre os dados.

Informação mútua

Sejam os dados consistindo em amostras $\mathbf{x} = (x_1, \dots, x_n)$, consideradas como i.i.d. provenientes de uma densidade contínua $p(X)$, e o vetor de rótulos de cluster $\mathbf{y} = (y_1, \dots, y_n)$, onde

$y_i \in \{1, \dots, k\}$. Define-se a informação mútua

$$I(X, Y) = H(X) - \sum_{y=1}^k p(Y = y) H(X | Y = y),$$

onde $H(\cdot)$ é a entropia diferencial da densidade contínua. Veja que $H(X)$ é independente da partição em clusters, e portanto a otimização recai sobre a minimização da $\sum_y p(Y = y) H(X | Y = y)$.

Como a densidade $p(X, Y)$ não é conhecida, os autores optam por estimar $H(X | Y = y)$ usando uma estimativa não-paramétrica baseada em MST. Pode-se mostrar que esse processo nos leva à seguinte medida:

$$F(X_1, X_2, \dots, X_k) = -d \sum_{i=1}^k \left(\frac{n_i}{n} \log \frac{L_i}{n_i} \right) - \sum_{i=1}^k \left(\frac{n_i}{n} \log \frac{n_i}{n} \right) \quad (20)$$

onde n_i denota o tamanho do i -ésimo agrupamento, d é o número de atributos (ou dimensões) dos padrões, e L_i representa a soma dos pesos das arestas na subárvore que define o i -ésimo agrupamento.

Essa formulação mostra que a função F combina, de forma implícita, dois aspectos desejáveis em um agrupamento: 1) minimizar a soma dos comprimentos das subárvores internas (refletindo a *compacidade* dos clusters) e 2) balancear o tamanho relativo de cada agrupamento (evitando clusters desproporcionalmente pequenos ou grandes).

Para otimizar essa função de agrupamento, devemos maximizar a função definida anteriormente. O algoritmo proposto (denominado ITM – *Information Theoretic MST-based clustering*) executa os seguintes passos principais:

1. Construir a MST $T(x)$ sobre o conjunto de dados (usando arestas ponderadas por distâncias euclidianas).
2. Inicializar $G \leftarrow T(x)$.
3. Repetir até obter k componentes:
 - Para cada componente atual, calcular o impacto de remover cada aresta e em termos de F .
 - Selecionar a aresta e^* cuja remoção maximiza F .
 - Remover e^* de G .
4. A partição correspondente às componentes de G é a solução de agrupamento final.

Vantagens e considerações

- O método é **não-paramétrico**: não pressupõe forma de distribuição ou número elevado de hiperparâmetros além de k .
- É capaz de lidar com clusters de forma arbitrária (não necessariamente convexos), graças à liberdade da MST e da estimativa de entropia.

- A estimativa da entropia baseada em MST incorpora tanto a *qualidade interna dos clusters* (via comprimentos das árvores) quanto o *equilíbrio entre tamanhos de clusters*, integrando-os em um único critério de informação.
- Do ponto de vista computacional, a construção da MST domina o custo ($O(n \log n)$ para algoritmos eficientes) e cada remoção de aresta pode ser feita em tempo linear ou próximo disso, resultando em complexidade global razoável para grandes bases de dados.
- Uma limitação apontada é a necessidade de que os dados sejam amostras de uma densidade absoluta contínua em $\mathbb{R}^d$; dados categóricos ou que residem em sub-variedades de menor dimensão podem violar os pressupostos da estimativa de entropia.

O método ITM é particularmente indicado quando se deseja um agrupamento que maximize informação mútua (em vez de apenas espalhamento intra-cluster), e onde se espera que os clusters possam ter formas complexas e não convexas. Se a aplicação exige poucos hiperparâmetros, ou se a representação dos dados permite uma MST eficaz (por exemplo, distância euclidiana ou outras métricas contínuas), então o método baseado em teoria da informação e MST se apresenta como uma alternativa elegante e poderosa frente a métodos clássicos como k -médias ou agrupamento por single linkage.

6.5.3 Derivação matemática

Partimos de

$$I(X, Y) = H(X) - \sum_{i=1}^k p_i H(X | Y = i), \quad \text{onde } p_i \equiv p(Y = i).$$

1. Estimativa de entropia condicional a partir da MST. Os autores usam uma estimativa não paramétrica da entropia diferencial baseada no comprimento da MST construída sobre as amostras de cada cluster. Uma forma frequentemente usada é

$$\hat{H}_{\text{mst}}(X_i) \approx d \log L_i - (d - 1) \log n_i + C_d,$$

onde:

- L_i é o comprimento (soma dos pesos) da MST do cluster i ,
- n_i é o número de amostras no cluster i ,
- d é a dimensão dos padrões,
- C_d é uma constante que depende apenas de d (e de termos numéricos que não dependem dos clusters).

Assim, para cada i substituímos $H(X | Y = i)$ por $\hat{H}_{\text{mst}}(X_i)$:

$$\sum_{i=1}^k p_i H(X | Y = i) \approx \sum_{i=1}^k p_i (d \log L_i - (d-1) \log n_i + C_d).$$

2. Inserir na expressão de $I(X, Y)$. Isso dá

$$I(X, Y) \approx H(X) - d \sum_{i=1}^k p_i \log L_i + (d-1) \sum_{i=1}^k p_i \log n_i - C'_d,$$

onde $C'_d = \sum_i p_i C_d = C_d$ (constante global).

3. Reescrever termos para obter uma forma prática. Queremos chegar a uma expressão que envolva termos como $\log(L_i/n_i)$ e também um termo relacionado a $-\sum p_i \log p_i$ (entropia discreta de Y). Observe que

$$-d \sum_i p_i \log L_i + (d-1) \sum_i p_i \log n_i = -d \sum_i p_i (\log L_i - \log n_i) + \sum_i p_i \log n_i.$$

Logo

$$I(X, Y) \approx H(X) - d \sum_i p_i \log \frac{L_i}{n_i} + \sum_i p_i \log n_i - C'_d.$$

Agora introduzimos e subtraímos $\sum_i p_i \log p_i$ (que é $-H(Y)$, a entropia discreta de Y), apenas para organizar a expressão em termos que são informativos para agrupamento:

$$I(X, Y) \approx \underbrace{H(X) + \sum_i p_i \log p_i - C'_d}_{\text{constante } C} - d \sum_i p_i \log \frac{L_i}{n_i} - \sum_i p_i \log p_i + \sum_i p_i \log n_i.$$

Agrupando constantes (tudo que não depende da partição em clusters) em uma constante C , e lembrando que $p_i = n_i/n$, temos

$$\begin{aligned} I(X, Y) &\approx C - d \sum_{i=1}^k \frac{n_i}{n} \log \frac{L_i}{n_i} - \sum_{i=1}^k \frac{n_i}{n} \log \frac{n_i}{n} \\ &\approx C - \left(d \sum_{i=1}^k \frac{n_i}{n} \log \frac{L_i}{n_i} + \sum_{i=1}^k \frac{n_i}{n} \log \frac{n_i}{n} \right) \end{aligned} \quad (21)$$

o que nos leva a conclusão de que devemos minimizar a expressão entre parêntesis.

4. Definição do objetivo prático F . Como C é independente da partição, maximizar $I(X, Y)$ é equivalente a maximizar o negativo da expressão entre parêntesis, ou seja:

$$F(X_1, \dots, X_k) := -d \sum_{i=1}^k \frac{n_i}{n} \log \frac{L_i}{n_i} - \sum_{i=1}^k \frac{n_i}{n} \log \frac{n_i}{n}.$$

Observações finais sobre aprox. e constantes

- A derivação usa explicitamente a estimativa de entropia baseada em MST; essa estimativa já contém constantes que dependem da dimensão d e de factores geométricos — esses termos são agrupados em C e descartados na optimização porque não dependem da partição.
- O termo $-\sum_i (n_i/n) \log(n_i/n)$ é simplesmente a entropia discreta de Y (prefere partições equilibradas); o primeiro termo $-d \sum (n_i/n) \log(L_i/n_i)$ vem da estimativa MST para a entropia diferencial condicional e penaliza subárvores longas (clusters pouco compactos).
- Portanto, F é uma forma prática (e consistente) de aproximar a informação mútua $I(X, Y)$ usando quantidades obtidas da MST. Maximizar F equivale (até constantes aditivas) a maximizar a informação mútua estimada.

6.5.4 Algoritmos aglomerativos

Os algoritmos aglomerativos partem de n *singletons* (agrupamentos contendo uma única amostra) e, a cada passo, adicionam a aresta que maximiza um critério de qualidade dos agrupamentos. O processo continua até que o número desejado de agrupamentos K seja atingido.

Algoritmo 17 Aglomerative_MST_Clustering(G, K)

Require: Grafo $G = (V, E, w)$ representando as amostras e suas distâncias; número de agrupamentos desejados K .

Ensure: Partição final $C(V, E'')$ com K agrupamentos.

- 1: $T = \text{MST}(G) = (V, E')$
- 2: $E'' = \emptyset$
- 3: **for** $i = 1$ **to** $n - K$ **do**
- 4: Encontre a aresta $(u, v) \in E' \setminus E''$ que maximiza

$$\max_{(u,v)} F(C(V, E'' \cup (u, v)))$$

- 5: onde $C(V, E'') = (X_1, X_2, \dots, X_\ell)$ denota a partição atual dos vértices.
 - 6: Adicione (u, v) a E''
 - 7: **end for**
 - 8: **return** Partição final $C(V, E'')$
-

Um dos principais desafios da abordagem aglomerativa está relacionado ao fato de que muitas medidas de qualidade de agrupamento não são bem definidas para *singletons*, ou seja, agrupamentos

contendo apenas uma amostra. Por essa razão, na prática, a abordagem divisiva tende a ser mais utilizada, especialmente em aplicações em que a interpretação do processo hierárquico é desejável e os critérios de qualidade são mais estáveis quando há mais de um elemento por grupo.

6.6 Considerações finais

Os métodos de agrupamento baseados em *árvores geradoras mínimas* (MSTs) constituem uma alternativa elegante e eficiente para a identificação de estruturas em dados complexos. Ao representar o conjunto de amostras como um grafo ponderado, no qual as arestas refletem a similaridade (ou dissimilaridade) entre pares de padrões, a MST permite uma modelagem global das relações entre os dados, preservando apenas as conexões mais relevantes.

Uma das principais vantagens desses métodos é sua simplicidade conceitual aliada à capacidade de capturar agrupamentos de formas arbitrárias, sem pressupor distribuições específicas ou formatos geométricos regulares. Além disso, as MSTs possibilitam tanto abordagens *divisivas* quanto *aglomerativas*, dependendo se o processo de agrupamento envolve a remoção ou a adição de arestas, respectivamente. Em ambos os casos, a estrutura hierárquica implícita na MST fornece uma visão clara sobre como os agrupamentos se formam e se separam à medida que o critério de corte é ajustado.

O critério de particionamento pode ser definido de diversas maneiras. Os métodos clássicos baseiam-se em medidas geométricas, como o comprimento das arestas, enquanto abordagens mais recentes empregam princípios de *teoria da informação*, associando o processo de agrupamento à maximização da informação mútua entre as variáveis dos dados e as partições resultantes. Essa última perspectiva permite uma interpretação estatística e fundamenta o uso de estimativas não paramétricas da entropia baseadas no comprimento das MSTs, proporcionando uma visão unificada entre geometria, estatística e teoria da informação.

Apesar de suas virtudes, os algoritmos baseados em MSTs também apresentam limitações. Por exemplo, o desempenho pode ser sensível à definição da métrica de similaridade e à presença de ruído, além de exigir cuidados em conjuntos de dados de alta dimensionalidade. Ainda assim, sua combinação de eficiência, robustez conceitual e interpretabilidade torna a abordagem baseada em MST uma ferramenta valiosa no arsenal de técnicas de agrupamento de dados.

Em síntese, o agrupamento por árvores geradoras mínimas destaca-se por sua capacidade de revelar a estrutura intrínseca dos dados de maneira não supervisionada, oferecendo uma ponte natural entre algoritmos clássicos de grafos e modernas abordagens de aprendizado de máquina baseadas em conectividade e informação.

7 Agrupamento de dados baseado em densidade

Os métodos de agrupamento baseados em densidade constituem uma importante classe de algoritmos não supervisionados capazes de identificar estruturas complexas em conjuntos de dados. Diferentemente de abordagens como o *k-médias* ou os modelos de mistura de Gaussianas, que assumem formas aproximadamente esféricas ou elípticas para os agrupamentos, os métodos baseados em densidade são capazes de detectar agrupamentos de formas arbitrárias, além de distinguir regiões densas de regiões de ruído ou baixa concentração de amostras.

A ideia fundamental consiste em definir agrupamentos como regiões do espaço de atributos onde a densidade local de amostras é significativamente maior do que em outras regiões. Intuitivamente, um agrupamento é formado por pontos que pertencem a uma mesma “montanha” na paisagem de densidade dos dados. Dessa forma, as fronteiras entre agrupamentos emergem naturalmente como regiões de densidade mais baixa, sem a necessidade de impor modelos paramétricos ou o número de grupos previamente.

Entre os algoritmos mais representativos dessa categoria estão o **DBSCAN** (*Density-Based Spatial Clustering of Applications with Noise*) e o **OPTICS** (*Ordering Points To Identify the Clustering Structure*). Ambos exploram o conceito de vizinhança e o número mínimo de amostras necessárias para caracterizar uma região densa. Esses métodos apresentam vantagens relevantes, como a capacidade de lidar com ruído e detectar agrupamentos de tamanhos e formatos variados, embora possam exigir ajustes cuidadosos dos parâmetros de densidade.

A abordagem baseada em densidade tem sido amplamente empregada em diversas áreas, incluindo análise de imagens, detecção de anomalias, mineração de trajetórias e bioinformática, entre outras. Sua robustez frente a dados ruidosos e a ausência de hipóteses rígidas sobre a forma dos agrupamentos tornam essa classe de métodos especialmente útil em cenários reais, onde as distribuições de dados são complexas e não seguem padrões geométricos simples.

Nos tópicos seguintes, serão apresentados os principais fundamentos teóricos que sustentam os métodos de agrupamento baseados em densidade, seguidos da descrição detalhada dos algoritmos DBSCAN e OPTICS, bem como suas propriedades, vantagens e limitações.

7.1 O algoritmo DBSCAN

O algoritmo **DBSCAN** (*Density-Based Spatial Clustering of Applications with Noise*) é um dos métodos mais populares de agrupamento baseado em densidade. O DBSCAN é capaz de identificar agrupamentos de forma arbitrária e detectar ruídos (outliers) sem a necessidade de especificar o número de grupos a priori, ao contrário de algoritmos como o *k-médias*.

A principal ideia do DBSCAN é que os pontos pertencentes a um mesmo agrupamento estão localizados em regiões de alta densidade, enquanto regiões de baixa densidade separam diferentes agrupamentos. O algoritmo depende de dois parâmetros fundamentais:

- ε (*epsilon*): raio de vizinhança que define o limite de proximidade entre os pontos;

- *MinPts*: número mínimo de vizinhos dentro da vizinhança ε para que um ponto seja considerado um **ponto núcleo**.

Com base nesses parâmetros, o DBSCAN classifica os pontos em três categorias:

1. **Pontos núcleo (core points)**: possuem ao menos *MinPts* vizinhos dentro do raio ε .
2. **Pontos de borda (border points)**: estão dentro da vizinhança de um ponto núcleo, mas não possuem vizinhos suficientes para serem núcleos.
3. **Ruídos (noise points)**: não pertencem a nenhuma vizinhança de núcleo e são considerados outliers.

O processo de agrupamento se inicia escolhendo aleatoriamente um ponto não visitado. Se o ponto for um núcleo, um novo agrupamento é iniciado e todos os pontos densamente conectados a ele são adicionados ao mesmo grupo, expandindo-se recursivamente a partir de seus vizinhos. Caso contrário, o ponto é marcado como ruído. Esse procedimento continua até que todos os pontos tenham sido processados.

Uma das principais vantagens do DBSCAN é sua robustez a ruídos e a capacidade de detectar agrupamentos de formatos arbitrários — uma limitação importante de métodos baseados em centroides. No entanto, a escolha apropriada dos parâmetros ε e *MinPts* é crucial para o bom desempenho do algoritmo, podendo variar significativamente conforme a densidade dos dados e o espaço de atributos analisado.

7.1.1 Fundamentação matemática

Para o algoritmo DBSCAN, uma pergunta crucial é: Como podemos medir a densidade ao redor de um ponto?

Medir a densidade ao redor de um ponto é um procedimento direto — define-se uma região em torno do ponto e avalia-se o número de pontos contidos nessa área designada. Essa abordagem serve como um método prático para estimar a densidade local em torno de um ponto específico.

Para determinar a densidade ao redor de um ponto, utilizamos círculos em duas dimensões (2-D), esferas em três dimensões (3-D) e hipersferas em espaços de dimensão n . Suponha que desenhemos um círculo de raio unitário ao redor de um ponto P , conforme ilustrado na figura anterior. Nesse contexto, estabelecemos o seguinte critério: uma região é considerada **esparsa** se contiver menos de três pontos, e **densa** se contiver três ou mais pontos.

O parâmetro *MinPts* (abreviação de *Minimum Points*) especifica o número mínimo de pontos necessários para que uma região seja considerada densa — isto é, para que ela forme um agrupamento (*cluster*).

Já o parâmetro ε (*epsilon*) é outro componente fundamental do algoritmo, responsável por definir o raio da vizinhança em torno de um ponto de dados. Especificamente, ε representa a **distância máxima** entre dois pontos para que ambos sejam considerados pertencentes à mesma vizinhança.

O **DBSCAN (Density-Based Spatial Clustering of Applications with Noise)** é um dos algoritmos mais importantes da categoria de agrupamento baseado em densidade, sendo capaz de descobrir **clusters de forma arbitrária e detectar ruído** de maneira eficiente. O método foi desenvolvido para lidar com grandes bancos de dados espaciais, onde os agrupamentos nem sempre possuem formas convexas ou tamanhos uniformes.

Ideia Fundamental

A principal hipótese do DBSCAN é que **regiões de alta densidade de pontos** correspondem a **agrupamentos**, enquanto **regiões de baixa densidade** correspondem a ruído.

Assim, um ponto p é dito **ponto núcleo** (*core point*) se o número de pontos dentro de sua vizinhança ε -vizinha for pelo menos MinPts. Pontos que não satisfazem esse critério podem ser **bordas** (*border points*) ou **ruído** (*noise points*).

Definições Formais

Seja D um conjunto de pontos em um espaço métrico (S, dist) .

1. Vizinhança ε de um ponto p :

$$N_\varepsilon(p) = \{q \in D \mid \text{dist}(p, q) \leq \varepsilon\}$$

2. Acessibilidade direta por densidade: Um ponto p é diretamente acessível por densidade a partir de q se:

$$p \in N_\varepsilon(q) \quad \text{e} \quad |N_\varepsilon(q)| \geq \text{MinPts}.$$

3. Acessibilidade por densidade: Um ponto p é acessível por densidade a partir de q se existe uma sequência de pontos:

$$p_1, p_2, \dots, p_n, \quad p_1 = q, \quad p_n = p,$$

tal que cada p_{i+1} é diretamente acessível por densidade a partir de p_i .

4. Conectividade por densidade: Dois pontos p e q são conectados por densidade se existe um ponto o tal que ambos são acessíveis por densidade a partir de o .

5. Cluster baseado em densidade: Um subconjunto $C \subseteq D$ é um cluster se:

$$\begin{cases} \forall p, q \in C, & p \text{ e } q \text{ são conectados por densidade;} \\ \forall p \in C, q \in D, & \text{se } q \text{ é acessível por densidade de } p, \text{ então } q \in C. \end{cases}$$

6. Ruído: Pontos que não pertencem a nenhum cluster:

$$\text{Noise} = D \setminus \bigcup_i C_i.$$

Essas definições garantem que os clusters sejam **conjuntos maximais de pontos densamente conectados**, enquanto regiões dispersas são tratadas como ruído.

7.1.2 Estrutura do algoritmo

O DBSCAN explora a ideia de “crescimento de clusters” a partir de pontos núcleo. O pseudocódigo básico pode ser descrito da seguinte forma:

Algoritmo 18 DBSCAN (Density-Based Spatial Clustering of Applications with Noise)

Require: D ▷ Conjunto de pontos
Require: ε ▷ Raio de vizinhança
Require: MinPts ▷ Número mínimo de pontos para uma região densa
Ensure: Conjunto de clusters $\{C_1, C_2, \dots, C_k\}$ e pontos de ruído

```

1: for all  $p \in D$  do
2:   if  $p$  não visitado then
3:     marcar  $p$  como visitado
4:      $N \leftarrow N_\varepsilon(p)$ 
5:     if  $|N| < \text{MinPts}$  then
6:       marcar  $p$  como ruído
7:     else
8:       criar novo cluster  $C$ 
9:       EXPANDCLUSTER( $C, p, N, \varepsilon, \text{MinPts}$ )
10:    end if
11:  end if
12: end for
```

A expansão de um cluster ocorre por meio da função `ExpandCluster`, que recursivamente adiciona à região todos os pontos alcançáveis por densidade a partir de um ponto núcleo.

Se o algoritmo é implementado com **índices espaciais** (como R*-trees), as buscas regionais têm custo médio $O(\log n)$, levando a uma complexidade global de:

$$O(n \log n)$$

No pior caso (sem estrutura de indexação), o custo pode chegar a $O(n^2)$.

7.1.3 Heurística para escolha de ε e MinPts

O algoritmo DBSCAN propõe o uso do gráfico *k-dist*, em que cada ponto é associado à distância até seu k -ésimo vizinho mais próximo. Ao ordenar essas distâncias de forma decrescente, o ponto de inflexão (“joelho”) do gráfico indica o valor apropriado de ε , enquanto MinPts é tipicamente fixado entre 4 e 6 (para dados 2D).

7.1.4 O conceito do gráfico k -dist

A ideia do gráfico k -dist é estudar a distribuição das distâncias aos vizinhos mais próximos de cada ponto, auxiliando na definição do parâmetro ε .

Para cada ponto $x_i \in D$, calcula-se:

$$d_k(x_i) = \text{distância do ponto } x_i \text{ ao seu } k\text{-ésimo vizinho mais próximo.}$$

O valor de k é tipicamente igual a $\text{MinPts} - 1$, pois queremos medir a densidade necessária para formar um cluster.

Isso gera uma lista de distâncias:

$$\{d_k(x_1), d_k(x_2), \dots, d_k(x_n)\}.$$

Essas distâncias são então ordenadas em ordem crescente, e o gráfico resultante é chamado de *gráfico k -dist*.

Ao plotar os pontos ordenados $(i, d_k(x_i))$, normalmente observamos um gráfico com três regiões distintas:

- **Região plana (baixa distância):** representa pontos dentro de clusters densos, com vizinhanças pequenas.
- **Transição (curva ascendente):** pontos próximos à fronteira dos clusters.
- **Região de crescimento acentuado (joelho):** pontos isolados ou ruído, com vizinhanças mais distantes.

O *joelho* (ou *knee*) do gráfico indica o valor ideal de ε :

- É o ponto onde o gráfico deixa de crescer lentamente e começa a crescer rapidamente;
- Esse ponto separa as densidades típicas dos clusters das densidades de ruído.

Matematicamente, o ponto de joelho é aquele onde a **segunda derivada** do gráfico (curvatura) é máxima — embora, na prática, ele seja identificado visualmente.

Procedimento prático passo a passo

1. Escolha um valor de $k = \text{MinPts} - 1$.

- Para dados 2D, normalmente $4 \leq \text{MinPts} \leq 6$.
- Em alta dimensão, recomenda-se $\text{MinPts} \approx d + 1$, onde d é o número de atributos.

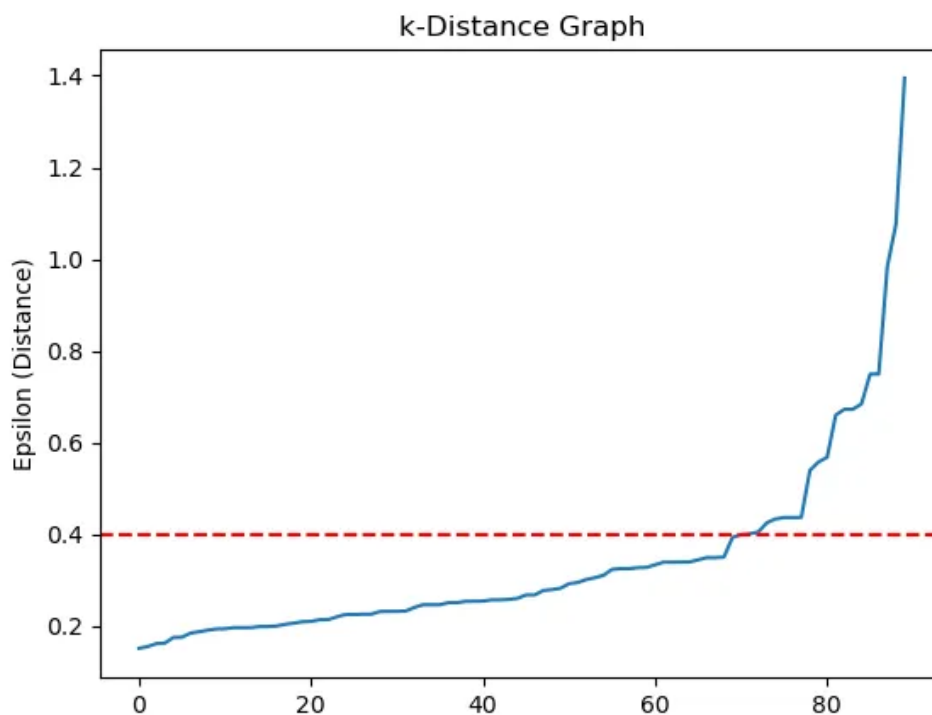
2. Para cada ponto x_i , calcule $d_k(x_i)$.

3. Ordene os valores $d_k(x_i)$ em ordem crescente.
4. Plote o gráfico k -dist (índice no eixo x , distância no eixo y).
5. Identifique o “joelho” do gráfico — esse será o valor recomendado para ε .

A interpretação geométrica indica que:

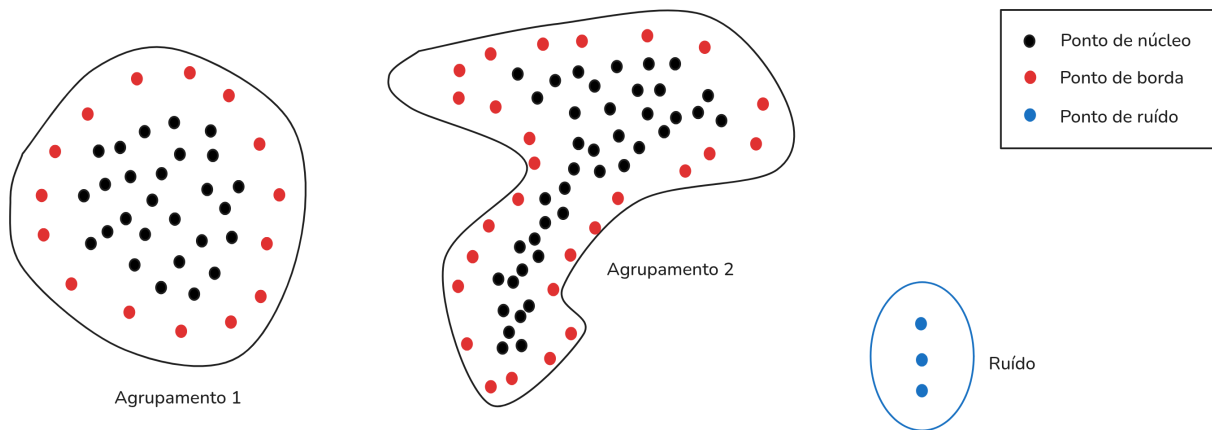
- Se ε for muito pequeno, a maioria dos pontos será marcada como ruído — o algoritmo fragmenta os clusters.
- Se ε for muito grande, clusters diferentes se fundem e a estrutura de densidade é perdida.

O valor do “joelho” reflete o limite natural de densidade do conjunto de dados, sendo portanto uma escolha intuitiva e robusta para o parâmetro ε .



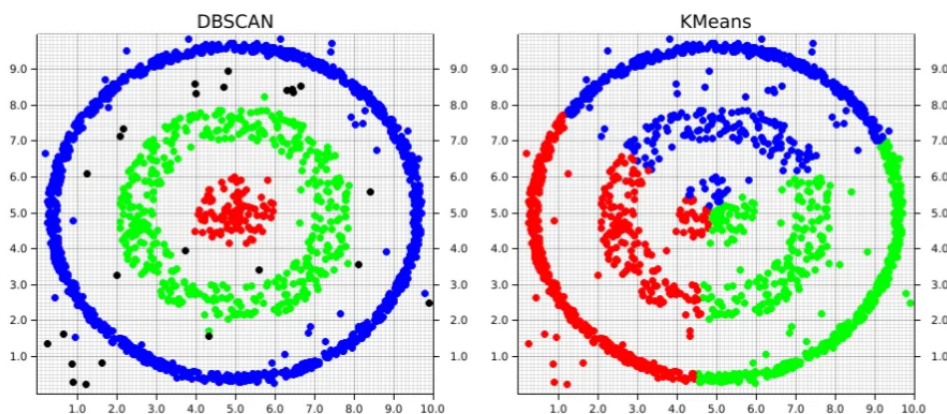
Em resumo, podemos dizer que:

- Pontos **núcleo**: localizados em regiões densas, iniciam clusters;
- Pontos de **borda**: estão próximos de regiões densas, mas não são densos o suficiente para iniciar um novo cluster;
- Pontos de **ruído**: não estão próximos o suficiente de qualquer cluster.



Vantagens do DBSCAN

- **Robustez a outliers:** O algoritmo é robusto a outliers, pois define agrupamentos com base em regiões densas de dados, tratando pontos isolados como ruído.
- **Não requer especificação prévia do número de clusters:** Diferentemente de alguns algoritmos de agrupamento, o DBSCAN não exige que o número de agrupamentos seja informado previamente pelo usuário, tornando-o mais flexível e aplicável a uma ampla variedade de conjuntos de dados.
- **Capacidade de detectar clusters de formas arbitrárias:** O DBSCAN é capaz de identificar agrupamentos com formas complexas, não estando limitado por suposições de formato (como esférico), sendo adequado para dados com estruturas irregulares.



- **Apenas dois hiperparâmetros principais:** O DBSCAN requer apenas o ajuste de dois hiperparâmetros principais: *eps* (limiar de distância para definir a vizinhança) e *min_samples* (número mínimo de pontos necessários para formar uma região densa). Essa simplicidade facilita o processo de ajuste de parâmetros.

Desvantagens do DBSCAN

- **Sensibilidade aos hiperparâmetros:** O desempenho do DBSCAN pode ser sensível à escolha dos hiperparâmetros, especialmente ao raio de vizinhança (*eps*) e ao número mínimo de pontos (*min_samples*). Uma seleção inadequada pode levar a uma segmentação insuficiente ou excessiva dos agrupamentos.
- **Dificuldade com clusters de densidades variáveis:** O DBSCAN tende a apresentar dificuldades quando os agrupamentos possuem diferentes densidades. Nessas situações, o algoritmo pode falhar em conectar regiões menos densas a agrupamentos mais densos, resultando em partições subótimas em conjuntos de dados com densidades heterogêneas.

7.2 O algoritmo OPTICS

O algoritmo **OPTICS** (“*Ordering Points To Identify the Clustering Structure*”), proposto por Ankerst et al. (1999), foi desenvolvido como uma extensão do *DBSCAN* para superar uma de suas principais limitações: a sensibilidade à escolha do parâmetro de vizinhança ε .

Enquanto o DBSCAN utiliza um único valor de ε para definir regiões densas, o OPTICS introduz um mecanismo capaz de capturar a estrutura hierárquica de densidade presente nos dados, permitindo a descoberta de agrupamentos com diferentes densidades sem a necessidade de escolher um único valor fixo de ε .

Motivação e intuição

A principal ideia do OPTICS é que, em vez de gerar diretamente uma partição dos dados, o algoritmo produz uma **ordenação dos pontos** (“*ordering*”) que reflete a estrutura de densidade do conjunto de dados. Essa ordenação é acompanhada de dois valores fundamentais para cada ponto:

- **Reachability-distance** (reachability-dist): distância mínima necessária para que um ponto possa ser alcançado a partir de outro ponto em uma região densa.
- **Core-distance** (core-dist): distância mínima que define o raio dentro do qual um ponto é considerado um *core point* (ponto de núcleo), isto é, um ponto suficientemente denso.

Ao processar os pontos em ordem crescente de densidade, o OPTICS constrói uma representação contínua da estrutura de agrupamento. O resultado é um **gráfico de distância de alcançabilidade** (*reachability plot*), no qual os vales (regiões de baixa distância) correspondem a agrupamentos densos, e os picos (altas distâncias) indicam separações entre clusters.

O DBSCAN pode ser interpretado como uma versão particular do OPTICS: se cortarmos o gráfico de distância de alcançabilidade em um determinado nível de densidade (ε), obtemos exatamente o mesmo resultado que o DBSCAN produziria com esse parâmetro. Assim, o OPTICS generaliza o DBSCAN, fornecendo uma visão mais rica e detalhada da estrutura de densidade dos dados.

Em síntese, o OPTICS não retorna diretamente um conjunto de clusters, mas uma **descrição ordenada da estrutura de densidade** do conjunto de dados, a partir da qual é possível extrair agrupamentos de diferentes níveis de granularidade. Essa característica faz do OPTICS uma ferramenta poderosa para a análise exploratória de dados e para a detecção de estruturas complexas em espaços de alta dimensionalidade.

Vantagens do OPTICS

O OPTICS apresenta diversas vantagens em relação ao DBSCAN:

- Detecta agrupamentos com diferentes densidades sem precisar ajustar ε manualmente.
- Gera uma representação visual intuitiva da estrutura dos dados (gráfico de alcançabilidade).
- Mantém a robustez do DBSCAN em relação a ruídos e outliers.

Desvantagens e Considerações

Por outro lado, o OPTICS é computacionalmente mais custoso, pois requer o cálculo e armazenamento das distâncias de alcançabilidade entre os pontos. Além disso, a interpretação do gráfico pode exigir análise visual ou heurísticas automáticas para definir os limites entre agrupamentos.

7.2.1 Fundamentação teórica

O algoritmo **OPTICS** (*Ordering Points To Identify the Clustering Structure*) foi proposto por Ankerst et al. (1999) como uma generalização do *DBSCAN*. Seu principal objetivo é identificar a estrutura de densidade de um conjunto de dados sem depender de um único valor de vizinhança ε .

Diferentemente do DBSCAN, que produz diretamente uma partição dos dados, o OPTICS gera uma **ordenação dos pontos** que reflete a hierarquia de densidades e as relações de proximidade entre eles. A partir dessa ordenação, é possível extrair agrupamentos com diferentes níveis de densidade, ajustando ε de maneira flexível.

Seja um conjunto de dados $D = \{x_1, x_2, \dots, x_n\}$ em um espaço métrico $(\mathbb{R}^d, \text{dist})$. São definidos dois conceitos centrais:

Definição 2 (Distância de núcleo (*core-distance*)). *Dado um ponto $p \in D$, um raio $\varepsilon > 0$ e um valor $\text{MinPts} \in \mathbb{N}$, define-se a **distância de núcleo** de p como:*

$$\text{core-dist}_{\varepsilon, \text{MinPts}}(p) = \begin{cases} \text{dist}_{\text{MinPts}}(p), & \text{se } |N_\varepsilon(p)| \geq \text{MinPts} \\ \text{indefinida}, & \text{caso contrário} \end{cases}$$

onde $N_\varepsilon(p) = \{q \in D \mid \text{dist}(p, q) \leq \varepsilon\}$ é o conjunto de vizinhos de p , e $\text{dist}_{\text{MinPts}}(p)$ é a distância entre p e seu MinPts -ésimo vizinho mais próximo.

Definição 3 (Distância de alcançabilidade (*reachability-distance*)). Seja $p, o \in D$ dois pontos e $core-dist(p)$ definida. A **distância de alcançabilidade** de o em relação a p é dada por:

$$reach-dist_{\varepsilon, MinPts}(p, o) = \max\{core-dist_{\varepsilon, MinPts}(p), dist(p, o)\}.$$

A distância de alcançabilidade representa a menor distância necessária para alcançar um ponto o a partir de um ponto núcleo p dentro de uma região de densidade. Se p não for um ponto núcleo, a distância de alcançabilidade é indefinida.

O OPTICS constrói uma ordenação linear dos pontos do conjunto D , onde a sequência reflete a estrutura de densidade dos dados. Cada ponto p recebe dois atributos:

- sua *distância de núcleo*;
- sua *distância de alcançabilidade* (em relação ao ponto anterior na ordenação).

Essas distâncias são usadas para gerar o **gráfico de alcançabilidade** (*reachability plot*), que revela os agrupamentos como “vales” de baixa distância e separações entre eles como “picos”.

O funcionamento do OPTICS pode ser resumido nos seguintes passos:

1. Para cada ponto $p \in D$, inicializar $reach-dist(p) = \text{undefined}$ e marcar todos os pontos como não visitados.
2. Selecionar um ponto p não visitado e marcar como visitado.
3. Obter seu conjunto de vizinhos $N_{\varepsilon}(p)$ e calcular $core-dist(p)$.
4. Se $core-dist(p)$ for definida, inserir todos os vizinhos elegíveis em uma *Priority Queue*, ordenada pela menor $reach-dist$.
5. Processar recursivamente os pontos da fila, expandindo o cluster implicitamente e registrando as distâncias de alcançabilidade.
6. Repetir até que todos os pontos tenham sido visitados.

O resultado é uma ordenação dos pontos e seus valores de alcançabilidade, a partir dos quais diferentes partições podem ser extraídas.

Algoritmo 19 OPTICS (Ordering Points To Identify the Clustering Structure)

Require: D : conjunto de pontos; ε : raio máximo de busca; MinPts: número mínimo de pontos

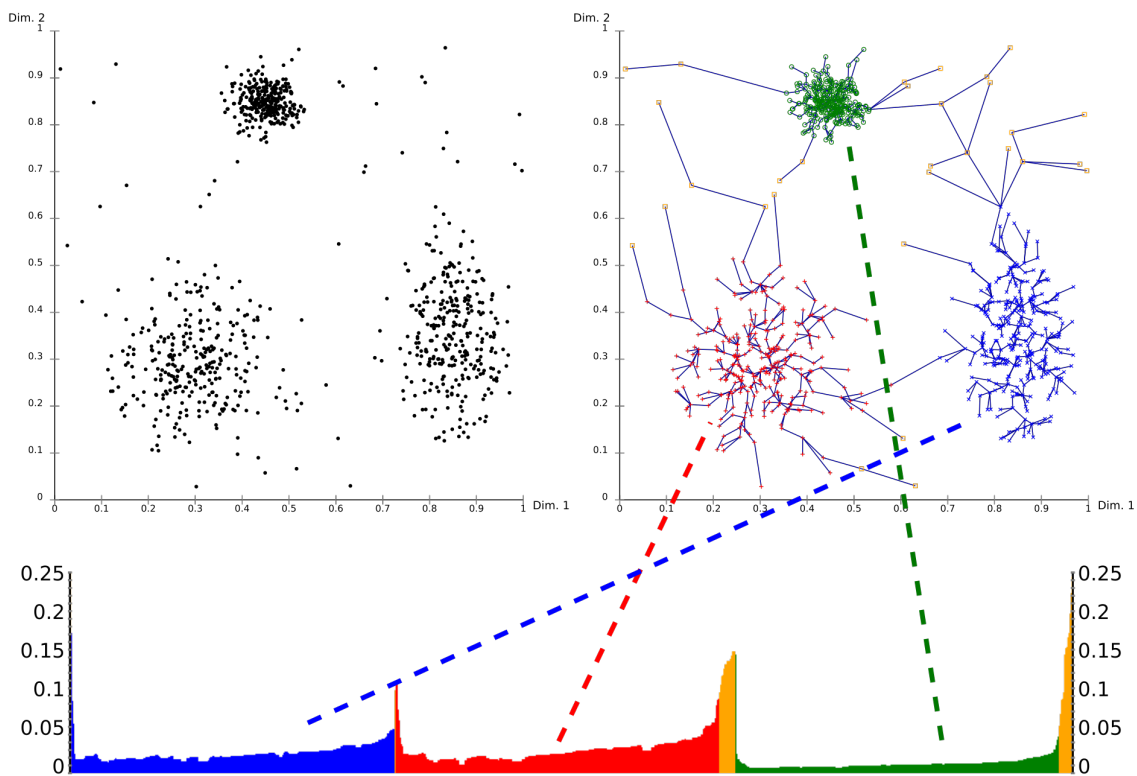
Ensure: Ordenação dos pontos e respectivas distâncias de alcançabilidade

```
1: for all  $p \in D$  do
2:   marcar  $p$  como não visitado
3:    $\text{reach-dist}(p) \leftarrow \text{undefined}$ 
4: end for
5: for all  $p \in D$  do
6:   if  $p$  não visitado then
7:     marcar  $p$  como visitado
8:      $N \leftarrow N_\varepsilon(p)$ 
9:     adicionar  $p$  à ordenação
10:    if  $|N| \geq \text{MinPts}$  then
11:       $\text{core-dist}(p) \leftarrow \text{dist}_{\text{MinPts}}(p)$ 
12:      ExpandClusterOrder( $p, N, \varepsilon, \text{MinPts}$ )
13:    else
14:       $\text{core-dist}(p) \leftarrow \text{undefined}$ 
15:    end if
16:  end if
17: end for

18: Função EXPANDCLUSTERORDER( $p, N, \varepsilon, \text{MinPts}$ ):
19:   criar uma fila de prioridade  $Q$  (ordenada por reach-dist)
20: for all  $o \in N$  do
21:   if  $o$  não visitado then
22:     marcar  $o$  como visitado
23:      $N' \leftarrow N_\varepsilon(o)$ 
24:     adicionar  $o$  à ordenação
25:     if  $|N'| \geq \text{MinPts}$  then
26:        $\text{core-dist}(o) \leftarrow \text{dist}_{\text{MinPts}}(o)$ 
27:     else
28:        $\text{core-dist}(o) \leftarrow \text{undefined}$ 
29:     end if
30:   end if
31:   if  $\text{core-dist}(p)$  é definida then
32:      $\text{new-reach-dist} \leftarrow \max(\text{core-dist}(p), \text{dist}(p, o))$ 
33:     if  $\text{reach-dist}(o)$  não definida ou  $\text{new-reach-dist} < \text{reach-dist}(o)$  then
34:       atualizar  $\text{reach-dist}(o)$  e inserir  $o$  em  $Q$ 
35:     end if
36:   end if
37: end for
38: while  $Q$  não vazia do
39:   retirar  $o$  com menor reach-dist de  $Q$ 
40:    $N' \leftarrow N_\varepsilon(o)$ 
41:   adicionar  $o$  à ordenação
42:   if  $|N'| \geq \text{MinPts}$  then
43:      $\text{core-dist}(o) \leftarrow \text{dist}_{\text{MinPts}}(o)$ 
44:     ExpandClusterOrder( $o, N', \varepsilon, \text{MinPts}$ )
45:   end if
46: end while
```

Complexidade

A complexidade do OPTICS é semelhante à do DBSCAN, sendo $O(n \log n)$ quando se utiliza uma estrutura de indexação eficiente (como *kd-tree*) para as buscas de vizinhança, e $O(n^2)$ no pior caso. O gráfico de distância de alcançabilidade resultante fornece uma representação visual da estrutura de densidade dos dados. Os **vales** correspondem a agrupamentos de alta densidade, enquanto os **picos** indicam transições entre regiões ou ruído. Assim, o OPTICS combina os benefícios do DBSCAN com uma representação contínua da hierarquia de densidades, tornando-se uma poderosa ferramenta para análise exploratória de dados complexos.



7.2.2 O gráfico de alcançabilidade

Utilizando um *reachability plot* (um tipo especial de dendrograma), a estrutura hierárquica dos agrupamentos pode ser obtida de forma bastante intuitiva. Trata-se de um gráfico bidimensional (2D), em que o eixo x representa a ordenação dos pontos conforme processados pelo algoritmo OPTICS, e o eixo y mostra a distância de alcançabilidade (*reachability distance*).

Como os pontos pertencentes a um mesmo agrupamento apresentam pequenas distâncias de alcançabilidade em relação aos seus vizinhos mais próximos, os agrupamentos aparecem no gráfico como *vales*. Quanto mais profundo o vale, mais denso é o agrupamento correspondente.

A figura acima ilustra esse conceito. No canto superior esquerdo, é mostrado um conjunto de dados sintético de exemplo. Na parte superior direita, visualiza-se a árvore de conexões (*spanning tree*) produzida pelo OPTICS, e na parte inferior encontra-se o gráfico de alcançabilidade calculado pelo algoritmo. As cores indicadas nesse gráfico representam rótulos atribuídos para fins ilustrati-

vos — elas não são geradas automaticamente pelo algoritmo —, mas é possível observar claramente como os vales do gráfico correspondem aos agrupamentos presentes no conjunto de dados. Os pontos amarelos na figura são considerados ruído, uma vez que não há vale correspondente no gráfico de alcançabilidade. Esses pontos geralmente não são atribuídos a nenhum agrupamento, exceto ao agrupamento trivial “todos os dados”, que aparece em resultados hierárquicos.

A extração de agrupamentos a partir desse gráfico pode ser feita manualmente, selecionando intervalos ao longo do eixo x após inspeção visual, ou estabelecendo um limiar (*threshold*) sobre o eixo y . Nesse último caso, o resultado é semelhante ao obtido pelo algoritmo DBSCAN com os mesmos parâmetros ε e *MinPts* — por exemplo, um valor de $\varepsilon = 0.1$ pode gerar bons resultados.

Alternativamente, diferentes algoritmos podem ser empregados para detectar automaticamente os vales, baseando-se na inclinação, em técnicas de detecção de “joelho” (*knee detection*) ou em máximos locais. Um intervalo do gráfico que se inicia com uma queda acentuada e termina com uma subida igualmente acentuada é interpretado como um vale e corresponde a uma região contígua de alta densidade.

É importante ter cuidado ao tratar os pontos localizados na extremidade de um vale, decidindo se eles pertencem ao agrupamento interno ou externo. Essa decisão pode ser tomada levando em consideração o predecessor de cada ponto. Os agrupamentos obtidos dessa forma são, em geral, de natureza hierárquica e não podem ser produzidos por uma única execução do DBSCAN.

7.3 O algoritmo HDBSCAN

O algoritmo **HDBSCAN** (*Hierarchical Density-Based Spatial Clustering of Applications with Noise*) é uma extensão hierárquica do DBSCAN e do OPTICS, projetada para lidar de forma mais eficiente com dados que apresentam **densidades variáveis** entre os agrupamentos. Ele combina os princípios de agrupamento baseado em densidade com técnicas de análise hierárquica, oferecendo uma forma mais robusta e automática de identificar estruturas complexas em conjuntos de dados.

Enquanto o DBSCAN exige a definição prévia de um único parâmetro de densidade (ε), o HDBSCAN elimina essa limitação ao construir uma **árvore de densidade hierárquica**, que descreve como os agrupamentos emergem e se dissolvem em diferentes níveis de densidade. Em seguida, essa hierarquia é condensada por meio de um processo de otimização que seleciona automaticamente os agrupamentos mais estáveis, resultando em uma partição final dos dados sem a necessidade de escolher manualmente o valor de ε .

Motivação e intuição

Em muitos conjuntos de dados reais — como dados espaciais, imagens, biologia computacional e aprendizado de representações — os agrupamentos podem apresentar **densidades muito distintas**. Nesses casos, o DBSCAN tende a falhar, pois um único valor de ε não consegue capturar adequadamente tanto as regiões densas quanto as regiões esparsas. O HDBSCAN foi proposto justamente para superar essa limitação, utilizando um modelo de densidade adaptativo.

A ideia central é reinterpretar o conceito de densidade utilizado pelo DBSCAN em termos de uma **distância de alcance ajustada pela densidade local**, denominada *mutual reachability distance*. Essa métrica redefine a noção de vizinhança de forma mais estável e menos sensível a flutuações locais na distribuição dos dados.

Etapas fundamentais do HDBSCAN

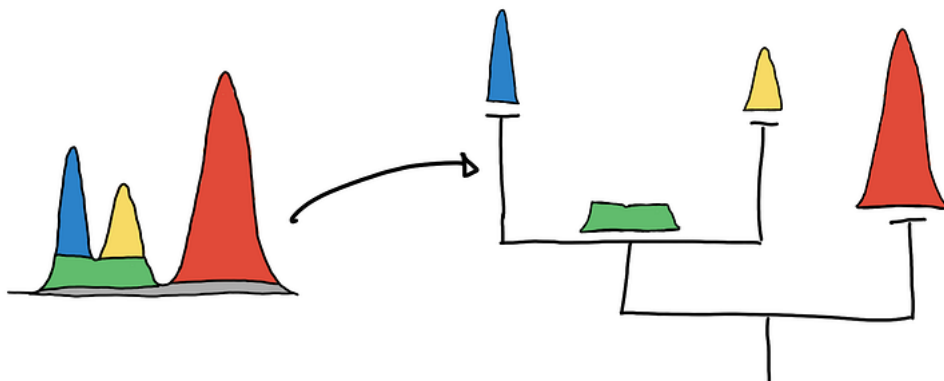
O funcionamento geral do HDBSCAN pode ser resumido nas seguintes etapas principais:

1. **Cálculo da distância de alcançabilidade mútua:** Define-se para cada ponto x_i uma *distância núcleo* (*core distance*) igual à distância até seu MinPts-ésimo vizinho mais próximo. A *distância de alcançabilidade mútua* entre dois pontos x_i e x_j é então dada por:

$$d_{\text{mreach}}(x_i, x_j) = \max \{ \text{core}_k(x_i), \text{core}_k(x_j), d(x_i, x_j) \}.$$

Essa redefinição suaviza as variações locais de densidade, estabilizando a análise.

2. **Construção da árvore geradora mínima (MST):** Uma árvore geradora mínima é construída sobre o grafo completo dos pontos, usando a distância de alcançabilidade mútua como peso das arestas. Essa árvore representa a conectividade das regiões densas em diferentes escalas.
3. **Geração da hierarquia de agrupamentos:** A MST é processada em ordem crescente de distância, simulando o “afrouxamento” progressivo da densidade. Isso resulta em uma hierarquia de agrupamentos — similar a um dendrograma — que mostra como os grupos se unem e se separam à medida que a densidade diminui.



4. **Condensação da hierarquia e extração de agrupamentos estáveis:** Em vez de escolher um nível fixo da hierarquia (como o corte em dendrogramas tradicionais), o HDBSCAN utiliza um critério de **estabilidade** baseado na persistência de cada agrupamento ao longo da hierarquia. Os agrupamentos mais estáveis — isto é, que permanecem coesos em um grande intervalo de densidades — são selecionados como resultado final.

Vantagens do HDBSCAN

O HDBSCAN oferece várias vantagens em relação ao DBSCAN e ao OPTICS:

- Não requer o parâmetro ε , reduzindo a necessidade de calibração manual.
- Detecta automaticamente agrupamentos de diferentes densidades.
- Produz uma representação hierárquica útil para análise exploratória.
- Identifica ruído de maneira mais robusta e adaptativa.
- É eficiente e escalável, com complexidade próxima à do DBSCAN.

Algoritmo 20 Hierarchical Density-Based Spatial Clustering of Applications with Noise

Require: D : conjunto de pontos; MinPts: número mínimo de pontos

Ensure: Conjunto de agrupamentos $\{C_1, C_2, \dots, C_k\}$ e pontos de ruído

```
1: for all  $p \in D$  do
2:   calcular a distância núcleo  $\text{core}_k(p)$ , isto é, a distância até o MinPts-ésimo vizinho mais próximo
3: end for
4: Construir o grafo de alcançabilidade mútua  $G = (V, E)$  com pesos:
    $w(p, q) = d_{\text{mreach}}(p, q) = \max\{\text{core}_k(p), \text{core}_k(q), d(p, q)\}$ 
5: Construir a árvore geradora mínima (MST) de  $G$ 
6: Ordenar as arestas da MST em ordem crescente de  $w(p, q)$ 
7: Inicializar uma hierarquia de agrupamentos  $\mathcal{H}$ 
8: for all aresta  $(p, q)$  em ordem crescente de  $w(p, q)$  do
9:   if a remoção de  $(p, q)$  divide um agrupamento  $C$  em  $C_1$  e  $C_2$  then
10:    adicionar  $(C_1, C_2)$  como filhos de  $C$  em  $\mathcal{H}$ 
11:   end if
12: end for
13: Condensar a hierarquia  $\mathcal{H}$ , removendo agrupamentos de baixa densidade relativa
14: for all agrupamento  $C_i$  em  $\mathcal{H}$  do
15:   calcular a estabilidade:
   
$$\text{Stability}(C_i) = \sum_{p \in C_i} \left( \frac{1}{\lambda_{\text{birth}}(p)} - \frac{1}{\lambda_{\text{death}}(p)} \right)$$

16:   onde  $\lambda = 1/\text{distância de alcançabilidade}$ 
17: end for
18: Selecionar os agrupamentos mais estáveis que não se sobrepõem
19: Rotular os pontos que não pertencem a nenhum agrupamento como ruído
20: return  $\{C_1, C_2, \dots, C_k\}$  e pontos de ruído
```

De forma intuitiva, o HDBSCAN pode ser visto como um processo de *filtragem da estrutura de densidade* dos dados: inicia-se com todos os pontos conectados em uma única rede e, conforme a densidade mínima exigida aumenta, as conexões mais fracas se rompem, revelando regiões coesas de alta densidade — os agrupamentos. Assim, os agrupamentos mais persistentes ao longo desse processo são considerados estruturalmente mais significativos.

O algoritmo HDBSCAN pode ser descrito de forma geral pelo pseudocódigo a seguir. Ele utiliza conceitos do DBSCAN e do OPTICS, mas adiciona um processo de hierarquização e seleção baseada em estabilidade de agrupamentos.

Comentários sobre o algoritmo

O HDBSCAN pode ser interpretado como uma versão hierárquica e adaptativa do DBSCAN. As principais diferenças conceituais são:

- Substitui o parâmetro ε por um processo contínuo de variação de densidade, evitando a escolha manual.
- Utiliza a **distância de alcançabilidade mútua** para suavizar variações locais de densidade.
- Constrói uma hierarquia de agrupamentos análoga a um dendrograma.
- Seleciona automaticamente os agrupamentos mais **estáveis** e representativos.

Essa abordagem permite capturar estruturas complexas em dados com **densidades heterogêneas**, além de produzir uma visualização hierárquica informativa da estrutura de agrupamento. A fase de condensação garante que apenas os agrupamentos mais persistentes — aqueles que permanecem coesos em uma ampla faixa de densidades — sejam incluídos no resultado final.

7.4 Considerações finais

Os métodos de *agrupamento baseado em densidade* representam uma das abordagens mais robustas e flexíveis dentro da análise não supervisionada de dados. Diferentemente de técnicas como o *k*-médias ou o *spectral clustering*, esses algoritmos não assumem uma forma pré-definida para os agrupamentos, sendo capazes de identificar regiões arbitrárias de alta densidade separadas por áreas de baixa densidade. Essa característica os torna especialmente adequados para conjuntos de dados complexos, nos quais os agrupamentos podem apresentar geometrias irregulares ou distribuições não lineares.

O algoritmo DBSCAN foi um marco importante ao introduzir o conceito de *pontos núcleo*, *bordas* e *ruído*, além de eliminar a necessidade de definir previamente o número de agrupamentos. Apesar de sua simplicidade e eficiência, o DBSCAN apresenta limitações na identificação de clusters com densidades variáveis — cenário em que algoritmos mais avançados, como o OPTICS e o HDBSCAN, oferecem soluções mais eficazes.

O OPTICS, ao produzir uma ordenação dos pontos e uma representação contínua das densidades locais por meio do *reachability plot*, fornece uma visão hierárquica dos agrupamentos, permitindo identificar estruturas de diferentes escalas. O HDBSCAN, por sua vez, estende essa ideia ao introduzir uma abordagem estatística de condensação hierárquica, na qual a estabilidade de cada agrupamento serve como critério objetivo para determinar sua relevância. Isso confere ao HDBSCAN uma grande

robustez, reduzindo a sensibilidade aos parâmetros e proporcionando resultados consistentes mesmo em dados com densidade heterogênea.

Em síntese, os métodos de agrupamento baseados em densidade consolidam-se como ferramentas poderosas para a descoberta de padrões complexos, especialmente em contextos em que os agrupamentos não seguem distribuições globais uniformes. Seu uso é amplamente indicado em aplicações de mineração de dados, visão computacional, análise de trajetórias e detecção de anomalias. A escolha entre DBSCAN, OPTICS e HDBSCAN deve levar em conta o equilíbrio entre simplicidade, interpretabilidade e robustez, conforme as características e a natureza do conjunto de dados em estudo.

Informações sobre o autor

Alexandre L. M. Levada é Bacharel em Ciência da Computação pela Universidade Estadual Paulista (UNESP) em 2002. Concluiu seu Mestrado em Ciência da Computação pela Universidade Federal de São Carlos (UFSCar) em 2006 e seu Doutorado em Física Computacional pela Universidade de São Paulo (USP) em 2010. No mesmo ano, ingressou no Departamento de Computação da Universidade Federal de São Carlos. De 2010 a 2024, foi professor assistente no Departamento de Computação, onde lecionou diversas disciplinas de graduação e pós-graduação como lógica matemática, teoria dos grafos, matemática discreta, processamento digital de imagens, análise de sinais e sistemas, reconhecimento de padrões, visão computacional, algoritmos e estruturas de dados 1 e 2, projeto e análise de algoritmos, programação científica e otimização matemática. Desde 2024 é professor associado do mesmo departamento. Seus principais temas de pesquisa incluem filtragem de ruído em sinais e imagens, reconhecimento de padrões e aprendizado de máquina, além da aplicação da geometria da informação na análise da dinâmica de campos aleatórios. É bolsista produtividade nível C do CNPq.

“Se você tiver um pão e eu tiver um euro, e eu uso o meu euro para comprar o seu pão, no final da troca eu terei o pão e você o euro. Parece um equilíbrio perfeito, não? A tem um euro, B tem um pão; depois, A tem o pão e B o euro. É uma transação justa, mas meramente material. Agora, imagine que você tem um soneto de Verlaine ou conhece o teorema de Pitágoras, e eu não tenho nada. Se me ensinar, no final dessa troca, eu terei aprendido o soneto e o teorema, mas você ainda os terá também. Nesse caso, não há apenas equilíbrio, mas crescimento. No primeiro, trocamos mercadorias. No segundo, compartilhamos conhecimento. E enquanto a mercadoria se consome, a cultura se expande infinitamente.”

Michel Serres, filósofo francês.

Referências

- Lloyd, S. P. Least squares quantization in PCM. *IEEE Transactions on Information Theory*, 28(2), 129-137, 1982.
- R. O. Duda, P. E. Hart, and D. G. Stork. *Pattern Classification*, 2nd ed., Wiley-Interscience, 2000.
- S. Theodoridis and K. Koutroumbas. *Pattern Recognition*, 4th ed., Academic Press, 2008.
- T. Hastie, R. Tibshirani, and J. Friedman. *The Elements of Statistical Learning*, 2nd ed., Springer, 2009.
- A. R. Webb and K. D. Copsey. *Statistical Pattern Recognition*, 3rd ed., Wiley, 2011.
- C. M. Bishop. *Pattern Recognition and Machine Learning*, Springer, 2011.
- C. C. Aggarwal and C. K. Reddy. *Data Clustering: Algorithms and Applications* (1st. ed.). Chapman & Hall/CRC, 2013.
- D. Arthur and S. Vassilvitskii. K-means++: the advantages of careful seeding. In *Proceedings of the eighteenth annual ACM-SIAM symposium on Discrete algorithms (SODA '07)*. Society for Industrial and Applied Mathematics, USA, 1027–1035, 2007.
- J. C. Bezdek, R. Ehrlich, W. Full. FCM: The fuzzy c-means clustering algorithm, *Computers & Geosciences*, Volume 10, Issues 2–3, Pages 191-203, 1984.
- V. Cohen-addad, V. Kanade, F. Mallmann-trenn, and C. Mathieu. Hierarchical Clustering: Objective Functions and Algorithms. *J. ACM* 66, 4, Article 26, 42 pages, 2019.
- Murtagh, F., Legendre, P. Ward's Hierarchical Agglomerative Clustering Method: Which Algorithms Implement Ward's Criterion?. *J Classif* 31, 274–295, 2014.
- Caliński, Tadeusz; Harabasz, Jerzy. "A dendrite method for cluster analysis". *Communications in Statistics*. 3 (1): 1–27, 1974.
- Dempster, A.P.; Laird, N.M.; Rubin, D.B. "Maximum Likelihood from Incomplete Data via the EM Algorithm". *Journal of the Royal Statistical Society, Series B*. 39 (1): 1–38, 1977.
- Akaike, H. A new look at the statistical model identification, *IEEE Transactions on Automatic Control*, 19 (6): 716–723, 1974.
- Schwarz, Gideon E. "Estimating the dimension of a model", *Annals of Statistics*, 6 (2): 461–464, 1978.
- von Luxburg, U. A tutorial on spectral clustering. *Stat Comput* 17, 395–416, 2007.

M. Belkin and P. Niyogi. Laplacian Eigenmaps for Dimensionality Reduction and Data Representation, in *Neural Computation*, vol. 15, no. 6, pp. 1373-1396, 1 June 2003..

Gagolewski, M., Cena, A., Bartoszek, M. et al. Clustering with Minimum Spanning Trees: How Good Can It Be?. *J Classif* 42, 90–112, 2025.

Campello, Ricardo JGB, Davoud Moulavi, and Jörg Sander. Density-based clustering based on hierarchical density estimates. *Pacific-Asia conference on knowledge discovery and data mining*. Springer, Berlin, Heidelberg, 2013.

Ankerst, Mihael; Breunig, Markus M.; Kriegel, Hans-Peter; Sander, Jörg. OPTICS: Ordering points to identify the clustering structure. *ACM SIGMOD Record*. 28 (2): 49–60, 1999.

McInnes, L., Healy, J. and Astels, S. HDBSCAN: Hierarchical density based clustering, *Journal of Open Source Software*, 2(11), 205, 2017.